



ספר פרויקט סייבר

שם בית הספר : תיכון אלון רמת השרון

שם העבודה : Anti PhishIdo

שם התלמיד : עידו דהן

ת.ז : 330944885

שם המנחה : עידן פלדברג, רום רפסון, עודי רז, יעקב שוצמן

שם החלופה : הגנת סייבר

תאריך הגשה : 24.05.2025





תוכן עניינים

4	מבוא.....
4	י"זום:.....
6	פירוט תיאור המערכת (אפיון):.....
9	תיאור תחום הידע – פרק מילולי.....
9	פירוט היכולות בצד השרת:.....
11	פירוט היכולות בצד הלקוח:.....
13	פירוט היכולות בצד האדמין (AdminClient).....
15	מבנה / ארכיטקטורה של הפרויקט.....
15	תיאור הארכיטקטורה של המערכת המוצעת:.....
17	תיאור הטכנולוגיה הרלוונטית:.....
17	תיאור זרימת המידע במערכת:.....
20	תיאור האלגוריתמים מרכזיים בפרויקט:.....
22	תיאור סביבת הפיתוח:.....
23	תיאור פרוטוקול התקשורת.....
25	תיאור מסכי המערכת:.....
37	תיאור מבני הנתונים:.....
41	סקירת חולשות והאיומים:.....
42	אילו חולשות קיימות ואיך טופלו:.....
43	מימוש הפרויקט.....
43	חלק א'- סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם.....
70	חלק ב'- הבעיה האלגוריתמית.....
73	חלק ג'- מסמך בדיקות מלא.....
76	מדריך למשתמש.....
76	התקנת המערכת.....
77	משתמשי המערכת:.....
81	סיכום אישי.....
81	תודות:.....
82	ביבליוגרפיה.....
83	נספחים.....
83	1. הסבר מעמיק על סיבות החשד כלפי קישור.....
84	ההצפנות בפרויקט.....



85	ערכים חשובים
86	הקוד
86	Server.py
92	client.py
95	communication.py
97	AdminClient.py
104	GUI.py
109	SQLITE.py
113	URL_Validators.py
115	RSA.py
118	Rating.py
120	Hash.py



מבוא

ייזום:

תיאור ראשוני של המערכת: המערכת תאפשר למשתמשים לבדוק אם קישור שהם מספקים שייך לאתר פשינג – אתר הונאה. כאשר המשתמש מזין קישור, המערכת מבצעת סדרה של בדיקות טכניות ואלגוריתמיות כדי לזהות אם מדובר באתר שנועד להונות את המשתמש או לגנוב מידע אישי. המערכת תספק תשובה מיידית האם הקישור שנבדק הוא פשינג או לא. הפרויקט נבחר בשל העלייה בשימוש של אתרי פשינג שמנסים להונות משתמשים ברחבי האינטרנט בשנים האחרונות. אתרים אלו משתמשים בטכניקות מתקדמות כדי להיראות כחוקיים, מה שמקשה על משתמשים לזהות אותם בצורה ידנית. מטרת המערכת היא לספק פתרון אוטומטי שיכול לעזור למשתמשים לזהות אתרי פשינג במהירות וביעילות. האתגרים אשר צופים לי בפרויקט הם:

1. פיתוח של אלגוריתם יעיל ומדויק לבדיקת קישור וזיהוי פשינג.
2. מימוש תקשורת רבת משתמשים ומאובטחת בצורה הטובה ביותר.
3. יצירת ממש משתמש נוח, שיוכל לשמש את הקהל הרחב.
4. מניעת מתקפת SQL Injection או DDoS.
5. שילוב הצפנה.

הגדרת הלקוח: עיקר קהל היעד הוא משתמשים, פרטיים או לא, אשר רוצים לוודא את בטיחות אתרי האינטרנט בהם הם גולשים, ולמנוע את היותם קורבני הונאה.

הגדרת יעדים/מטרות: המטרות העיקריות של המערכת הן לספק למשתמשים כלים כדי לזהות אם האתר שבו רוצים לגלוש הוא אתר פשינג. כל זה, יתבצע בשילוב של שירות פשוט, נגיש וחינמי, שיאפשר שימוש נוח לכולם, תוך הגנה על בטיחותו ופרטיותו של המשתמש באתר.

בעיות תועלות וחסכונות: מה הבעיה? אתרי פשינג מהווים איום על משתמשי אינטרנט רבים, וכוללים שיטות מתקדמות להונות את המשתמשים ולהשיג גישה למידע רגיש כמו סיסמאות, מיקום ופרטי כרטיסי אשראי. ברוב המקרים, המשתמשים אינם יודעים איך לזהות את האתרים המזויפים. מה אני מנסה להשיג? המערכת תספק כלי אוטומטי שמסייע למשתמש לזהות אתרי פשינג בזמן אמת, ומפחית את הסיכון להיכנס לאתרים מזיקים. מה התועלות שסביר לצפות מהמערכת?

1. שירות מהיר ויעיל - המשתמש יקבל תשובה מיידית על האם האתר חשוד או לא.
2. הגנה על המידע האישי - המשתמשים ימנעו מלספק את פרטיהם האישיים לאתרים מזויפים.
3. נוחות - המערכת פועלת ישירות דרך דפדפן, כך שאין צורך בהתקנה או תצורה מורכבת.

מה החסכונות? המערכת יכולה למנוע נזק כלכלי ואובדן מידע אישי, דבר שמוביל לחיסכון בכסף ומאמץ. בנוסף, חסכון בזמן עבור עסקים או מוסדות שמתמודדים עם המון קישורים חשודים.



סקירת פתרונות קיימים: כיום, קיימים מספר אתרים, כלים ושרתים שמטרתם לזהות אתרי פישנינג. לדוגמה Google Safe Browsing, אשר מתריע מפני סכנת הונאה בכניסה לאתר. בנוסף, קיים האתר PhishTank, אשר מתקיים בצורה מאוד דומה לפרוייקט שלי.

סקירת טכנולוגיות הפרוייקט: כפי שתיארתי, זוהי אינה טכנולוגיה חדשה. ישנם כבר מספר אתרים ותוכנות אשר מסוגלים לבצע את אותה המטרה בצורה כזו או אחרת.

תיחום הפרוייקט:

python – פיתון היא שפת התכנות המרכזית בפרוייקט. התוכנה מספקת פשטות לחלק מקטעי הקוד שלא תתקבל בשפות אחרות. היא תספק פשטות במשימות כגון קישור בין שרת ולקוח, ושילוב אלגוריתמים מתקדמים.

PyCharm – סביבת הפיתוח שבה יפותח הפרוייקט, המאפשרת כתיבה יעילה וניהול קוד נכון.

SQLite3 – הספרייה בפייתון בה משתמשים כדי לגשת לdatabase, כלומר מסדי הנתונים.

DB Browser for SQLite – בסיס הנתונים בה תשתמש המערכת לצורך של שמירת נתונים באמצעות טבלה. לדוגמה, תשמור קישורים שנבדקו בעבר ותוצאתם. בסיס נתונים זה קל לשימוש ותורם הרבה בשמירת הנתונים.

RSA – שימוש במנגנון הצפנת RSA לצורך הגנה על נתוני המשתמשים והקישורים, ובשילוב עם טכניקות אבטחה מתקדמות כדי למנוע חשיפה של מידע.

Hashing – המערכת תשתמש בטכניקות Hashing כמו SHA-256 כדי ליצור אבטחה גבוהה כדי שאף גורם לא מורשה לא יכול לשנות את הנתונים שנשמרו.

Client-server – המערכת תפעל בצורה של שרת – לקוח, שתאפשר למשתמשים בה להעביר מידע ולהשתמש בה בצורה מאובטחת.

Multi-threading – המערכת תשתמש באמצעות ה-multi-threading במספר תהליכים בו זמנית, מכיוון שיתכן שמספר משתמשים באתר ישלחו בקשות במקביל, והמערכת צריכה להבטיח שכל בקשה תטופל בזמן אמת ללא עיכובים, ובקשה אחת לא תעכב את האחרון (כלומר, יתחילו רק כשתיגמר).

TKINTER – המערכת תכלול ממשק משתמש גרפי (GUI) – זוהי ספרייה מובנית בפייתון שתאפשר יצירת אפליקציה בצורה נוחה, שתהיה פשוטה לשימוש. הממשק יאפשר למשתמש להזין קישורים לבדיקה באמצעות חלון קלט, ולקבל תוצאות באופן ברור ומובהק עם הודעה מתאימה. בנוסף, TKINTER יספק עיצוב פשוט באמצעות כפתורים, תיבות טקסט ותוויות. בקיצור, הממשק יעוצב כך שיכלול אלמנטים ויזואליים כמו צבעים, חלונות הודעה והתראות, כדי לשפר את חוויית המשתמש ולהפוך את השימוש בו לפשוט ויעיל, בנוסף לפשטות שבה.



Cyber Security(sql injection/DDoS/multi-factor authentication) - המערכת בעלת דגש משמעותי על אבטחת סייבר, כדי להבטיח הגנה מלאה על נתוני המשתמשים ולמנוע מתקפות פוטנציאליות כמו SQL Injection, מתקפות DDOS ועוד. אחד מהאיומים המרכזיים הוא SQL Injection, שבו תוקף מנצל חולשות בקלט כדי להריץ פקודות SQL זדוניות על מסד הנתונים. כדי להתמודד עם איום זה, המערכת תעשה שימוש בפרמטרים מוכנים מראש (Prepared Statements) ותוודא שאימות וסינון קלט ייעשה לכל נתון שהמשתמש מזין, ובכך תימנע הרצת פקודות זדוניות.

בנוסף, כדי להתמודד עם מתקפות DDoS (Distributed Denial of Service), המערכת תשלב מנגנוני הגנה מגוונים, כגון הגבלת מספר הבקשות שניתן לשלוח מכתובת IP מסוימת בפרק זמן נתון. אמצעים אלה יאפשרו למערכת להמשיך לפעול באופן יציב, גם במקרים של עומס חריג.

פירוט תיאור המערכת (אפיון):

תיאור מפורט יותר של המערכת. מה היא אמורה לעשות- המערכת היא בעצם כלי פשוט, המתבסס על משלל תחומים בפייתון, SQL, ועוד. מטרת המערכת לבדוק את אמינות הקישורים לאתרי אינטרנט. בעצם, המשתמש מזין את הקישור לבדיקה, והמערכת מבצעת סדרת בדיקות טכניות המתבססות על משלל דרכים, כדי לקבוע האם האתר הנתון חשוד. בסיום של התהליך, שיתבצע במהירות, יעילות, ביטחון ואמינות, המערכת תחזיר למשתמש תשובה המצביעה על בטיחות הקישור.

פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת (ברמת כותרות)-

1. משתמשים פרטיים:

-בדיקת קישורים מהירה ובטוחה.

-קבלת תשובה בזמן אמת.

-נגישות קלה באמצעות TKINTER.

2. משתמשים עסקיים/חברות:

-ניתוח של מספר קישורים בו זמנית.

-בדיקה של האם כתובת אתר החברה נתפס כהונאתי.



פירוט הבדיקות (קופסא שחורה) שמתוכננות לפרויקט-

1. בדיקת מבנה הurl:

- בדיקה אם הקישור מכיל מבנה חריג (תווים חשודים, מבנה חשוד).
- אימות שם הקישור מול רשימת קישורים בטוחים או רשימת אתרים שאובחנו חשודים.

2. בדיקות סייבר:

- בדיקה למניעת SQL Injection באמצעות ניהול שאילתות מאובטח. תכנון הבדיקה: הזנת בשדות הקלט טקסט שמכיל ניסיון הזרקה, לדוגמה '1'='1' OR
- הגבלת בקשות למניעת תקיפת DDos. תכנון הבדיקה: המערכת תזהה ותגביל את מספר הבקשות שניתן לשלוח מאותה כתובת IP בזמן נתון, ותנקה את הכתובות החשודות.

תכנון וניהול לו"ז זמנים לפיתוח המערכת

תכנון לוז ראשוני – לפני התחלת הפרוייקט תכננתי לעצמי לו"ז להמשך, כלומר, סדר המשך משימות שאני צריך לבצע כדי להתקדם הלאה. ראשית, רציתי לייצר מנגנון שעובד ובודק כתובות אתרים כראוי. לאחר מכן, רציתי לייצר איזשהו ממשק גרפי, בו נכניס קישור ונקבל תוצאת האם הוא בטוח או לא, ובנוסף הסיבות. לאחר מכן, הצבתי לעצמי מטרה לייצר טבלאות DB, אך ללא ידיעה מה יהיה בהן. חשבתי כי במהלך העבודה על הפרוייקט יעלו לי רעיונות, איזה טבלאות יהיו משמעותיות ונכונות לשימוש. על פי הלו"ז, לאחר שכל אלו עבדו, המטרה הבאה שלי הייתה לייצר תקשורת נכונה בין משתמשים, ולאחריה הצפנה, threading ואבטחה, אך אלו דברים שלא חשבתי עליהם מספיק לעומק ונראו רחוקים.

לוז בפועל- בפועל, התחלתי את הלו"ז כמתוכנן, שבוטא באמצעות ממשק גרפי ובדיקות שפועלים כראוי. הגעתי לשלב של רשימות הDB. זהו שלב שהיה יחסית ארוך מבחינתי, בו ניסיתי לגלות אילו טבלאות יהיו הכי טובות כדי לשרת את המטרה. זה היה עוד בתחילת הפרוייקט, לכן לא היה לי עוד חזון. חשבתי רבות עד שהגעתי למסקנה שאני רוצה רשימה שחורה-blacklist, אשר תשמור את הקישורים שכבר אובחנו כחשודים. לאחר מכן, חשבתי על טבלת הסיבות, כך שלכל קישור יהיו את הסיבות מדוע הוא חשוד מתועדות, וכך לא אצטרך לעשות בדיקה בכל פעם, כלומר, אם הקישור בעבר כבר אובחן כחשוד, ניתן להדפיס את סיבותיו על פי הטבלאות, דבר זה יחסוך בדיקה חוזרת של קישור זהה משם נוצרו טבלאות, reasons, links_reasons. לאחר מכן, יצרתי תקשורת בין לקוח לשרת אחד. לאחר שהכל עבד כמו שצריך, יצרתי תקשורת מרובת משתמשים אשר עובדת באמצעות multi-threading. בקטע הזה, נגמר הפן הפרקטי של הקוד. יש תכנית אשר עובדת. כעת יש לאבטח את הקוד. יצרתי הגנות כגון הגנה DDos ו SQL injection. בנוסף יצרתי הצפנה – RSA – אשר תשומש בכל העברת מידע בין הלקוח לשרת, כדי למנוע דליפת מידע. מכאן והלאה הוספתי עוד מגוון פיצ'רים על מנת להעמיק את הקוד כגון דירוג בטיחות קישור, פידבק משתמש, גישה ישירה ללינק בטוח, משתמש אדמין ועוד.



ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם:

הסיכון	כיצד תוכנן להתמודד	מה בוצע בפועל
SQL_Injection	שימוש ב-prepared statements	שימוש ב-prepared statements
DDos	הגבלת כמות התחברויות לפי IP	הגבלת כמות התחברויות מסוימת במשך זמן נתון (10 התחברויות בעשר שניות)
MITM (האזנה למידע שעובר בין השרת ללקוח במטרה לגנוב מידע)	הצפנת RSA מקצה לקצה	הצפנת RSA מקצה לקצה
שימוש במידע אישי	לא היה ידוע	שמירה מוצפנת של הקו ומידע רגיש במסדי הנתונים, כמו כן הצפנת מידע מקצה לקצה.
גישה לא מורשית לממשק ניהול	סיסמה כלשהי לאגף הניהול	אימות סיסמה למנהל בעזרת מנגון HASH והשוואה לערך מסוים של סיסמה מוצפנת אשר מוגדר כמשתנה סביבה (environment vairable)

תיאור תחום הידע – פרק מילולי

פירוט היכולות בצד השרת:

שם היכולת: בדיקת קישורים

מהות היכולת: היכולת של צד השרת לקבל לינק מסוים, לבצע עליו סדרת בדיקות ולהחזיר תשובה לגביו שכוללת את רמת סיכנו.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: קבלת קישור מוצפן, פיענוחו, בדיקתו (האם הכתובת תקינה, האם האתר משתמש בהצפנת SSL, האם הקישור מסוג https, תקינות הקישור, מילות מפתח חשודות בקישור, האם האתר בכלל קיים, חישוב דירוג הבטיחות של האתר, עדכון הרשימה השחורה וקשריה, הצפנת התשובה, שליחתה ללקוח.

אובייקטים נחוצים: על מנת לממש את היכולת, נשתמש באובייקטים הבאים: מחלקת communication, אשר היא מחלקת תקשורת שיושרת את תכונותיה אל השרת. Databases – מחלקת הטבלאות. RSA – מחלקה כתובה עצמאית אשר מבצעת את פעולת ההצפנה/פיענוח. Requests – ספרייה בפיתוח אשר מאפשרת לתקשר עם אתרי אינטרנט בעזרת פרוטוקול http. (בקוד שלי הספרייה משמשת כדי לדעת האם האתר קיים). URL_Validator – מחלקה כתובה ידנית, אשר נועדה כדי להחליף את ספריית validators – משמשת לביצוע הבדיקות על הקישור הנתון (אם הקישור תקין).

שם היכולת: ניהול רשימה שחורה

מהות היכולת: היכולת של השרת לנהל מאגר של קישורים מסוכנים שהתגלו במהלך פעילות המשתמשים. ניהול זה כולל שמירה, צפייה, עדכון והסרה של קישורים מתוך הרשימה השחורה, במטרה להגן על המשתמשים ולהציג התרעות רלוונטיות בזמן אמת.

אלו הוזנו ע"י מנהל המערכת, או התגלו בעת בדיקות המשתמשים.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: צפייה בקישורים החשודים, הסרת קישור מהרשימה, איפוס כולל של הרשימה, שליפה או עדכון הרשימה, שליחת תגובה מוצפנת ללקוח.

אובייקטים נחוצים: communication, אשר אחראי על כל התקשורת בין השרת ללקוח. RSA, אשר מצפין את ההודעות בין השרת ללקוח. Databases, בתוך מחלקה זו נמצאת גם הרשימה השחורה המתנהלת.



שם היכולת: ניהול רשימת דירוגים

מהות היכולת: קליטה של דירוג מן המשתמש, הצגה ואיפוס טבלת דירוגים.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: קבלת דירוג משתמש ושמירתו במסד הנתונים, צפייה בכל הדירוגים הקיימים בטבלה, חישוב ממוצע דירוגים, שליחת תוצאות ללקוח.

אובייקטים נחוצים: communication – בשביל התקשורת עם הלקוח, קבלת דירוג ושליחת תשובות. RSA- הצפנה ופיענוח של התקשורת בין השרת ללקוח. Databases – שם מאוחסנים כל הדירוגים של המשתמשים.

שם היכולת: ניהול היסטוריית בדיקות

מהות היכולת: שמירה ומעקב אחרי היסטוריית בדיקות הקישורים שהתבצעו דרך המערכת שלי, תוך כדי שמירת פרטיות המשתמש.

אוסף היכולות הנדרשות למימוש היכולת: הצפנת כתובת הקל של הלקוח הנתון, שמירת פרטי הבדיקה (ip,port), קישור שנבדק, זמן שנבדק, האם חשוד) במסד נתונים מסודר, שליפת 10 הבדיקות האחרונות (היחיד שיכול לראות באמת את הקל הוא האדמין אשר בידו מספר סיסמאות), שליחת נתונים ללקוח וקבלתם (מוצפנים).

אובייקטים נחוצים: communication, אשר אחראי על כל התקשורת בין השרת ללקוח. RSA, אשר מצפין את ההודעות בין השרת ללקוח. Databases, בתוך מחלקה זו נמצאת גם הרשימה השחורה המתנהלת.



פירוט היכולות בצד הלקוח:

שם היכולת: שליחת קישור לבדיקה בשרת

מהות היכולת: היכולת של הלקוח לשלוח קישור מוצפן לשרת, לקבל בחזרה ניתוח מוצפן של הקישור, לפענחו ולהציגו למסך – הצגת האם הקישור תקין, סיבות מדוע חשוד, גישה לאתר ועוד.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: הכנסת הקישור בממשק הגרפי, ולחיצה על כפתור לבדיקתו. הצפנתו של הקישור ושליחתו לשרת. קבל תוצאה מוצפנת מן השרת, פענוח הסיבות והצגתן למסך (דבר זה קורה גם כשהאתר לא זמין, על מנת לאפשר למשתמש מענה תיאורטי). שמירת תוצאות הבדיקה גם במסד הנתונים של הלקוח.

אובייקטים נחוצים: GUI, הממשק הגרפי של המשתמש אשר יתן לו חוויה נוחה ופשוטה בה יוכל לקבל מענה. RSA, הצפנה ופיענוח בעת התקשורת בין שרת ללקוח. Communication – כל התקשורת בין השרת ללקוח. Databases- ניהול ושמירת התוצאות גם במסד הנתונים המקומי של הלקוח.

שם היכולת: דירוג חווית משתמש

מהות היכולת: היכולת של המשתמש בתוכנה לדרג את חווית שימוש. הדירוג כולל שם(אופציונלי), ציון (בין 1-5), ופידבק.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: מילוי פרטי הדירוג על ידי המשתמש, הצפנתם ושליחה לשרת.

אובייקטים נחוצים: GUI, שיהיה נוח ויגרום למשתמש לדרג בצורה נוחה את חווית השימוש שלו. RSA – שומר על פרטיותו של המדרג, פרטיו ישארו מוצפנים לאורך התקשורת. Communication – כל התקשורת בין השרת ללקוח.

שם היכולת: ניהול החיבור ואימותו מול השרת

מהות היכולת: יצירת חיבור מאובטח לשרת ושימוש במערכת, עם החלפת מפתחות הצפנה

אוסף היכולות/פעולות הנדרשות למימוש היכולת: יצירת מפתחות הצפנה, שליחת המפתח הציבורי לשרת, קבל מפתח ציבורי מהשרת (לשרת וללקוח יש מפתח פרטי שונה ויחודי לו, השרת מצפין הודעות עם המפתח הפרטי של הלקוח, והלקוח עם של השרת)

אובייקטים נחוצים: RSA, אחראי על המפתחות, communication, כל התקשורת בין השרת ללקוח.



שם היכולת: ניתוק החיבור

מהות היכולת: יש לסיים את החיבור בצורה תקינה – ניתוק הלקוח מהשרת וסגירת שאר הפרמטרים

אוסף היכולות/פעולות הנדרשות למימוש היכולת: ניתוק החיבור של הלקוח אל השרת, סגירת מסד הנתונים, סגירת הממשק. כל זאת, כדי שהחיבור יסגר בצורה מסוגרת, כדי למנוע תקלות ואו דליפות ולמוע חיבורים שישארו פתוחים.

אובייקטים נחוצים: GUI, כל הממש הגרפי. Databases, מבני הנתונים שצריכים להיסגר. Communication, התקשורת בין השרת לבין הלקוח אשר צריכה להתנתק בסיום השימוש.



פירוט היכולות בצד האדמין (AdminClient)

שם היכולת: התחברות למערכת באמצעות סיסמת ניהול

מהות היכולת: האדמין חייב להזדהות ע"י סיסמה לפני שיוכל לגשת אל הנתונים הנגישים אליו.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: קבלת סיסמה מהמשתמש, אימות מול משתנה סביבה (environment variable) אשר מאובטח באמצעות הצפנת HASH. במידה וערך הצפנת הHASH של הסיסמה שהזין המשתמש, שווה לערך המוצפן אשר מוגדר במשתנה הסביבה, אותו האדמין רשאי להיכנס למערכת. אחרת, הוא אינו מורשה.

אובייקטים נחוצים: HashManager, אשר אחראי על הצפנת הHash. GUI - חלון עם קלט לאימות האדמין.

שם היכולת: ניהול הרשימה השחורה

מהות היכולת: צפייה, הסרה, עדכון או רענון של כל הקישורים המסוכנים במערכת.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: צפייה בכל הקישורים, הסרת קישור מסוים, עדכון סיבות של קישור מסוים, קבלת תגובה מהשרת, הצגת המידע.

אובייקטים נחוצים: GUI - הממשק הגרפי, הצגת מידע וכפתורי ניהול. RSA - הצפנה ופיענוח הודעות בתקשורת. Communication - תקשורת מול השרת.

שם היכולת: צפייה בטבלת הדירוגים

מהות היכולת: הצגה נוחה של טבלת דירוגי המשתמשים לאדמין בממשק, אשר כוללת שמות ציונים ומשובים.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: שליחת בקשה לשרת לקבל הדירוגים, קבלתם מוצפנים מהשרת, פיענוחם והצגתם בממשק

אובייקטים נחוצים: GUI, הצגת טבלה גרפית ונוחה בממשק. RSA - הצפנה בתקשורת. Communication, התקשורת מול השרת.



שם היכולת: צפייה בממוצע הדירוגים

מהות היכולת: חישוב ממוצע הדירוגים של המשתמשים והצגתם למסך.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: שליחת בקשה לשרת לקבל הדירוגים, קבלת הממוצע המוצפן, פענוח והצגת המידע בצורה נוחה בממשק.

אובייקטים נחוצים: הצגת ממוצע בצורה נוחה בממשק. RSA - הצפנה בתקשורת. Communication, התקשורת מול השרת.

שם היכולת: צפייה בהיסטוריית הבדיקות – עשר הבדיקות האחרונות

מהות היכולת: לאפשר לאדמין לראות בצורה נוחה בממשק את עשר הבדיקות האחרונות שהתבצעו, כולל כתובת IP, Port, הקישור שנבדק, האם הוא בטוח ושעת הבדיקה.

אוסף היכולות/פעולות הנדרשות למימוש היכולת: קבלת מפתח פרטי מהאדמין, ביצוע פעולת hash עליו, ובדיקה האם ערך זה תואם למשתנה הסביבתי המוגדר. ברגע שכן, כל כתובות ה IP מופענחות באמצעות אותו המפתח המוזן, ומוצגות לאדמין באופן גלוי. דבר זה מאפשר רק לאדמין לראות את כתובות ה IP של המשתמשים, מה ששומר על בטיחותם.

אובייקטים נחוצים: HashManager - אימות המפתח אל מול משתנה הסביבה. RSA, פיענוח כתובת ה IP. GUI – תצוגת ההיסטוריה בממשק. Communication - תקשורת מול השרת

שם היכולת: ניהול מסדי נתונים – איפוס רשימות

מהות היכולת: היכולת של האדמין לאפס את מסדי הנתונים בלחיצת כפתור (הרשימה השחורה וקשריה או טבלת הדירוגים).

אוסף היכולות/פעולות הנדרשות למימוש היכולת: שליחת פקודה לשרת למחיקת תוכן הרשימה השחורה/ טבלת הדירוגים. קבלת אישור מהשרת על הצלחה. הצגת הודעה למנהל.

אובייקטים נחוצים: GUI - הצגת כפתורי איפוס ואישורים. RSA - הצפנה ופענוח של הפקודות והתגובות. Communication – תקשורת מול השרת.



מבנה / ארכיטקטורה של הפרויקט

תיאור הארכיטקטורה של המערכת המוצעת:

תיאור החומרה- רכיבים שונים והקשרים ביניהם: המערכת מורכבת ממודל של שרת לקוח, הכולל את הרכיבים הבאים:

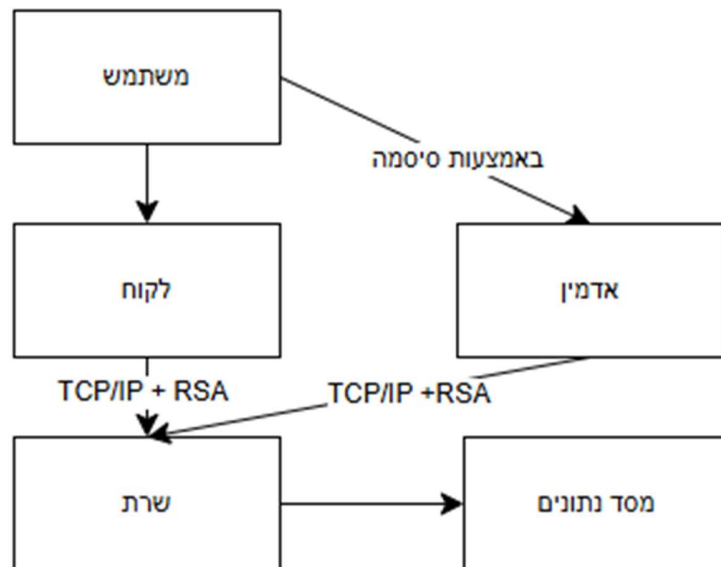
שרת – מקבל קישורים או דירוגים מן הלקוח באמצעות תקשורת, מבצע ניתוח של הקישורים באמצעות מחלקת הבדיקות (URL_Validators), מחשב את דירוג הבטיחות של האתר לפי משקלים שהוגדרו מראש, מנהל מסד נתונים לשמירת נתונים חשופים כגון רשימות שחורות, היסטוריה, דירוגים וכו'. מנהל הצפנה או פענוח לפני קבלת או שליחת הודעות וקישורים דרך התקשורת על מנת לשמור על תקשורת מאובטחת. לאחר העיבוד השרת מחזיר את התוצאה אל הלקוח.

לקוח- שולח בקשות לשרת (דירוגים, קישורים) על ידי הכנסת הקישור או הדירוג לממשק הגרפי. לאחר מכן מקבל את התשובות מן השרת ומציג אותן על המסך הגרפי. לכל אורך התקשורת מתבצעת הצפנה על מנת לשמור על המידע, בדיוק כמו בצד השרת.

מסד הנתונים – מנוהל בצד השרת. מכיל טבלאות שונות כגון רשימה שחורה, רשימת דירוגים, היסטוריה ורשימות מקושרות (באמצעות נרמול).

תקשורת- כל ההודעות העוברות בין השרת לבין הלקוח, מתבצעות באמצעות תקשורת אסימטרית – תקשורת RSA. ישנה מחלקת הצפנה ופענוח, אשר מתבצעת לפני כל שליחת הודעה או אחרי קבלת הודעה בין השרת ללקוח. בכל פעם המפתחות משתנים על מנת להבטיח אבטחה ברמה גבוהה. התקשורת נעשית באמצעות socket, ונעשית דרך פרוטוקול IP, אשר זהו פרוטוקול המשתמש להעברת מידע ברשת בין מחשבים שונים. המערכת שלי משתמשת בכתובות IP על מנת לאפשר חיבור ישיר בין השרת ללקוח – המשמעות היא שהמערכת פועלת ברשת מקומית ולא דרך האינטרנט הציבורי. בנוסף, מתבצעת תקשורת באמצעות פרוטוקול TCP, אשר מבטיח שהמידע המועבר יגיע בשלמותו ולפי הסדר הנכון (הפרוטוקול מנהל את פתיחת החיבור, ניהול זרימת ההודעות וסגירת החיבור) כלומר, אצלי במערכת השרת מאזין לפניות על פורט 8002 כרגע, הלקוח יוזם חיבור TCP לשרת באמצעות הפורט וכתובת ה-IP, וכל ההודעות בין השרת ללקוח נשלחות על גבי חיבור TCP אחד מתמשך, כך שאין צורך לפתוח ולסגור את החיבורים בין השרת ללקוח עבור כל הודעה נשלחת. (בפועל בפרוייקט, הלקוח פותח חיבור TCP לכתובת ולפורט של השרת, ההודעות מוצפנות באמצעות הצפנה אסימטרית ונשלחות על גבי אותו ה-TCP, השרת קורא ושולח את ההודעות באמצעות send ו recv. המערכת עושה שימוש ב chunking, עם סימון של <END> כדי לאפשר שליחת הודעות גדולות יותר מהמגבלה של 1024 בתים)

הקשרים בין הרכיבים – הלקוח מקבל קלט מהמשתמש ושולח בקשה לשרת. השרת בודק את הבקשה ושולח תוצאה חזרה ללקוח. השרת מנהל את מסדי הנתונים. כל התקשורת ביניהם מוצפנת מקצה לקצה.

שרטוט המציג קשרים אלו:



תיאור הטכנולוגיה הרלוונטית:

שפת תכנות – המערכת נכתבה בשפת פייתון (גרסה 3.10).

מערכת הפעלה – המערכת פותחה ב-WINDOWS.

תקשורת – הלקוח והשרת מתקשרים באמצעות socket, בפרוטוקול IP/TCP המספק חיבור אמין בין השרת לבין הלקוח. כל התקשורת מוצפנת באמצעות RSA לשמירה על אבטחת המידע.

תחומי עניין – אבטחת מידע: המערכת בודקת את בטיחות הקישורים ומזהירה מפני קישורים חשודים על מנת להבטיח למשתמש בטיחות בעת גלישתו באינטרנט. תקשורת: תקשורת מאובטחת בין הלקוח לשרת. חווית משתמש: חווית משתמש נוחה וקלילה באמצעות tkinter בעלת מגוון פיצ'רים (דירוגים, פידבק, תפריט מידע). ניהול נתונים: המערכת שומרת ומנהלת רשימות שחורות, סיבות חסימה, רשימת היסטוריה על מנת לאפשר מעקב נכון על המידע הנצבר דרך המשתמשים (בנוסף קיימת גם טבלת דירוג משתמש).

תיאור זרימת המידע במערכת:

בשרת:

1. בדיקת קישורים
 - הלקוח שולח לשרת כתובת URL מוצפנת.
 - השרת מפענח את הכתובת.
 - השרת מריץ סדרת בדיקות טכניות על הכתובת.
 - השרת מחשב את דירוג הבטיחות של האתר.
 - השרת מעדכן את הרשימה השחורה במסד הנתונים במידת הצורך.
 - השרת מצפין את תוצאות הבדיקה ושולח ללקוח.
2. ניהול רשימה שחורה
 - הלקוח (או האדמין) שולח בקשה לצפייה, עדכון או איפוס של הרשימה השחורה.
 - השרת מקבל את הבקשה ומבצע את הפעולה במסד.
 - השרת שולח תשובה מוצפנת ללקוח.
3. ניהול רשימת דירוגים
 - הלקוח שולח לשרת דירוג משתמש מוצפן הכולל שם (אופציונלי) דירוג ומשוב.
 - השרת שומר את הדירוג במסד הנתונים.
 - הלקוח או האדמין מבקשים לצפות בדירוגים או במסד.
 - השרת מחשב או מעביר את המידע ושולח אותו מוצפן ללקוח או לאדמין.
4. ניהול היסטוריית בדיקות
 - בכל בדיקה, השרת מצפין את כתובת הקו של הלקוח ושומר את פרטי הבדיקה במסד הנתונים.
 - האדמין מבקש את עשר הבדיקות האחרונות.
 - השרת שולח את ההיסטוריה (10) המוצפנת ללקוח שמציג אותה לאדמין.



בלקוח:

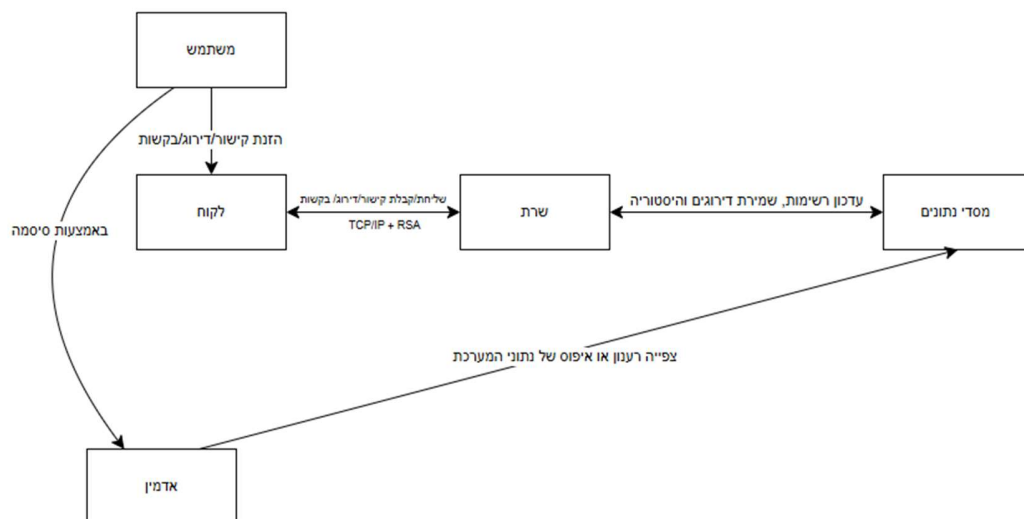
1. שליחת קישור לבדיקה בשרת
 - המשתמש מזין כתובת URL בממשק ולוחץ על כפתור הבדיקה.
 - הלקוח מצפין את הכתובת ושולח לשרת.
 - הלקוח מקבל תוצאה מוצפנת מהשרת, מפענח אותה ומציג בממשק.
2. דירוג חווית משתמש
 - המשתמש לוחץ על כפתור הדירוג בממשק ונפתח לו חלון דירוגים.
 - המשתמש ממלא את הטופס (שם אופציונלי, דירוג ומשוב) ושולח.
 - הלקוח מצפין את המידע ושולח אותו לשרת.
3. ניהול החיבור ואימותו מול השרת
 - הלקוח והשרת מבצעים החלפת מפתחות ציבוריים בהתחברות הראשונה.
 - כל התקשורת לאחר מכן מתבצעת בעזרת המפתחות שהוחלפו.
4. ניתוק החיבור
 - הלקוח סוגר את החיבור לשרת.
 - הלקוח סוגר את הממשק.

באדמין:

1. התחברות למערכת באמצעות סיסמת ניהול
 - האדמין מזין סיסמה בממשק.
 - המערכת בודקת אם ערך הצפנת hash של הסיסמה שהוזנה שווה לערך המוצפן אשר מוגדר כמשתנה סביבה במחשב.
 - אם האימות מצליח, האדמין נכנס למערכת.
2. ניהול הרשימה השחורה
 - האדמין שולח בקשה (מוצפנת) לניהול הרשימה (צפייה רענון או הסרה).
 - השרת מעבדת את הבקשה ושולח תגובה מוצפנת.
 - התגובה מפוענחת ומוצגת על הממשק.
3. צפייה בטבלת דירוגים
 - האדמין מבקש לצפות במסד הדירוגים.
 - השרת שולח את הנתונים המוצפנים בחזרה.
 - הלקוח מפענח אותן ומציג את המידע בממשק.
4. צפייה במוצע דירוגים
 - האדמין מבקש את ממוצע הדירוגים.
 - השרת מחשב אותו ושולח את המידע המוצפן.
 - המידע מוצג בממשק.
5. צפייה בהיסטוריית הבדיקות
 - האדמין מבקש את עשר הבדיקות האחרונות.
 - השרת שולח את המידע המוצפן.
 - האדמין מזין את המפתח הפרטי לפענוח כתובות הקן של המשתמשים.

- המערכת מבצעת הצפנת hash לאותו המפתח. אם ערך הצפנת hash של המפתח זהה לערך המוצפן בתור משתנה הסביבה, אז הקו מפוענח באמצעות אותו מפתח והטבלה מוצגת. במקרה הערך לא זהה מוצגת הודעה מתאימה.
- 6. ניהול מסדי נתונים – איפוס רשימות
- האדמין שולח פקודה לשרת לאיפוס טבלאות (הרשימה השחורה או טבלת דירוגים).
- השרת מאפס את התוכן ושולח אישור חזרה לאדמין.

הצגת זרימת המידע בין הרכיבים המרכזיים במערכת באמצעות שרטוט:



השרטוט מציג את זרימת המידע המרכזית בין רכיבי המערכת – משתמש, לקוח, שרת, אדמין ומסדי הנתונים. המשתמש מזין בקשות (קישורים לבדיקה, דירוגים ועוד) דרך הלקוח, אשר מועברת לשרת בצורה מאובטחת באמצעות תקשורת TCP/IP אשר מוצפנת בהצפנת RSA. השרת מטפל בבקשות, ניגש למסד הנתונים, ומחזיר ללקוח תשובות מוצפנות. האדמין, מתקשר עם השרת לצורכי ניהול המידע – צפייה רענון או איפוס של הנתונים שלו. זרימת המידע העובר בין הרכיבים המרכזיים במערכת מוצגים בתרשים הנ"ל בצורה ברורה ומרכזת את הפעולות המרכזיות במערכת.

תיאור האלגוריתמים מרכזיים בפרוייקט:

ניסוח וניתוח של הבעיה האלגוריתמית:

הבעיה המרכזית שהמערכת נועדה לפתור היא הערכת רמת הסיכון של כתובת אינטרנטית מסויימת לצורך הגנה על המשתמש מפני אתרי פישניג וקישורים אשר נועדו למטרות זדוניות. המטרה היא לבצע בדיקה מהירה ואוטומטית שתזהה סימנים חשודים בקישור ותספק למשתמש המלצות על בטיחותו, אשר דרכן יוכל לבחור האם לגלוש באותו האתר או לא. הבדיקות מתבצעות על בסיס מאפיינים חיצוניים של האתר.

תיאור אלגוריתמים קיימים לפתרון הבעיה:

1. בדיקת פורמט הקישור ותקינות - אלגוריתמים קיימים אשר נועדו להחליף את ספריית validator שנמצאת בpython, המבצעת בדיקות תחביריות בסיסיות על הקישור כדי לראות אם תקין בהתאם לכללים תחביריים מקובלים
2. בדיקת שימוש בפרוטוקול HTTPS – בדיקה פשוטה שבודקת האם כתובת האתר מתחילה ב-<https://> מה שמצביע על כך שהאתר מספק חיבור מאובטח בין הדפדפן לאתר. שימוש ב<http> בלבד מראה לרוב על שימוש באתר לא מאובטח
3. בדיקה קיימת של תעודת ssl תקפה – בדיקה שמנסה לפתוח חיבור מאובטח עם האתר באמצעות ספריית requests, ובדיקת תקפות ה-SSL- מסמך דיגיטלי המאמת את זהותו של אתר אינטרנט ומאפשר הצפנה של התקשורת בינו לבין המשתמש. בעצם, התעודה מבטיחה שהמידע המועבר בין הדפדפן לאתר מוצפן, הכי האתר הוא אכן הבעלים החוקי של הקישור, ולא מתחזה. היעדר תעודת SSL או תעודה לא תקפה היא סימן לאתר לא בטוח
4. חיפוש מילות מפתח חשודות – סריקה של כתובות URL לאיתור מחרוזות נפוצות באתרי הונאת פישניג כגון bank,prize,bonus,password,login ועוד. שימוש במילים כאלה עלול להצביע על ניסיון להטעות את המשתמש.
5. בדיקת תווים חשודים בכתובת – בדיקה שמחפשת תווים חשודים כגון %, &, * , @ ועוד. תווים אלה נפוצים ביצירת כתובות מזוייפות או מבלבלות שמנסות להטעות את המשתמש.
6. בדיקה אם הדומיין קיים בפועל. המערכת מנסה להתחבר לאתר באמצעות שליחת בקשה HTTP. אם השרת מחזיר תגובה תקינה – status code 200, האתר ככל הנראה קיים. אתרים שלא קיימים או מחזירים שגיאות חיבור מסומנים כחשודים.



הפנייה למקורות רלוונטיים:

1. בדיקת תקינות כתובת URL – ספריית validators של python – לבדיקת תקינות תחבירית של כתובות אינטרנט.
לינק למידע נוסף: [/https://pypi.org/project/validators](https://pypi.org/project/validators)
2. בדיקת קיומו של האתר ותעודת SSL- ספריית requests של python- מאפשרת לשלוח בקשות HTTP ובדיקת SSL.
לינק לתיעוד: [/https://requests.readthedocs.io/en/latest](https://requests.readthedocs.io/en/latest)
3. זיהוי קישורים חשודים על בסיס תווים ומילים – אין ספרייה רשמית לכך, הבדיקות מתבצעות על בסיס רשימות שהגדרנו ידנית בקוד.
לינק למילים לדוגמה: https://www.researchgate.net/figure/Suspicious-words-to-detect-phishing-URLs_fig6_364265241

סקירת הפתרון הנבחר:

לאחר בחינות שיטות קיימות לזיהוי קישורי פישג, בחרתי ליישם מערכת מבוססת כללים פשוטה, המתמקדת בניתוח כתובת ה-URL על בסיס מספר מאפיינים נגשים ובסיסיים. המערכת שלי לא כוללת למידת מכונה או ניתוח סטטיסטי מתקדם, אלא משתמשת בשיטות הבאות:

- בדיקת תקינות התחביר של הכתובת: באמצעות מחלקה שכתבתי בעצמי, URL_Validators, אשר היא גרסה משופרת של הספרייה הקיימת validators.
- בדיקת קיומו של האתר ובדיקת תעודת SSL תקפה: באמצעות שליחת בקשת HTTP עם ספריית requests ובדיקת תקינות ה-SSL
- חיפוש תווים ומילות חשודות: על בסיס רשימות שהוגדרו בקוד בהשראת מקורות מחקריים.
- בדיקת קיום: בדיקת קיום של האתר בפועל על ידי ניסיון להתחבר לאתר ולוודא שהוא זמין.

נימוק הבחירה:

-פשטות ויעילות- הפתרון מהיר ועצמאי. בנוסף אינו דורש תשתיות מתקדמות או מודלים מאומנים מראש.

-שליטה מלאה- מאפשר לי להתאים את הכללים לצרכים של הפרוייקט מבלי להיות תלוי בשירות חיצוני.

-יכולת להרחבה עתידית- ניתן להוסיף בעתיד עוד שכבות מתקדמות כגון למידת מכונה.



שלילת פתרונות אלטרנטיביים:

-שימוש בלמידת מכונה- נדחה בשלב זה עקב מורכבות המימוש, הצורך בנתונים לאימון ועומס חישובי, אשר לוקח זמן רב.

-שימוש בשירותי צד שלישי – שירות צד שלישי הוא שירות חיצוני, שאינו חלק מן המערכת ומספק פונקצייה מוכנה מראש דרך חיבור לאינטרנט, למשל שירות שמקבל URL ונחזיר את דירוג סיכנו (למשל Google Safe Browsing). דבר זה לא מומש כיוון שאין לי שליטה על איך השירות פועל. בנוסף ישנה דרישן חיבור לאינטרנט, כלומר השירות דורש תקשורת חיצונית תמידית. בנוסף, לעיתים השירות דורש תשלום או מגבלות שימוש אחרות.

תיאור סביבת הפיתוח:

פירוט כלי הפיתוח הדרושים לפיתוח:

-שפת תכנות – python 3.11

-סביבת פיתוח PyCharm

-מערכת הפעלה – windows 11

-ספריות – requests, tkinter, hashlib, sqlite3

פירוט הסביבה והכלים הנדרשים לבדיקות:

-הרצת המערכת: מחשב עם python 3.11 או גרסה חדשה יותר.

-מערכת הפעלה להרצה – windows 11

-חיבור אינטרנטי – אשר מהווה חובה עבור תקשורת בין הלקוח לשרת ובדיקת אתרים קיימים.

-הגדרת משתנה סביבה: ADNIN_HASH- ערך hash של סיסמת הניהול לאימות זהות האדמין. ADMIN_PRIVATE_KEY-המפתח הפרטי של האדמין המשמש לפענוח כתובות ip מוצפנות שנשמרו בהיסטוריית הבדיקה. (על מנת להגדיר משתנה סביבה, ניכנס להגדרות המחשב, נכתוב בחיפוש "ערוך את משתני הסביבה של המערכת". לאחר מכן, נלחץ על הכפתור "משתני סביבה..." שם, נלחץ על הכפתור "חדש..." אשר נמצא תחת משתני המשתמש ולא משתני המערכת. נזין שם משתנה וערך משתנה. כדי לסגור נלחץ "אישור" על כל החלונות). (תדריך מלא בהמשך).

תיאור פרוטוקול התקשורת

תיאור מילולי של פרוטוקול התקשורת: המערכת מבוססת על פרוטוקול TCP/IP לתקשורת אמינה ובטוחה בין השרת ללקוח. כל ההודעות הנשלחות בין השרת ללקוח מוצפנות על ידי הצפנה אסימטרית-RSA, על מנת להבטיח את פרטיותו וביטחונו של המשתמש. מבנה הודעה: כל הודעה במערכת בנויה כמחרוזת טקסטואלית מוצפנת (באמצעות RSA). אין הגדרה של כמות בתים קבועה, מכיוון שההודעה מוצפנת, כלומר אורך ההודעה משתנה בהתאם לאורך התוכן לפני ואחרי ההצפנה. לאחר ההצפנה ההודעה נשלחת כבלוק אחד של בתים דרך חיבור TCP/IP.

תהליך העברת ההודעה: ראשית, הלקוח והשרת מחליפים מפתחות ציבוריים לפני התחלת התקשורת המוצפנת. שנית, כל אחד מצפין את ההודעות במפתח הציבורי של הצד השני. שלישית, כל אחד מפענח את ההודעות במפתח הפרטי שלו.

פירוט של כל ההודעות הזורמות במערכת:

מבנה השדות בהודעה	נשלחת אל-	נשלחת מ-	תיאור/תוכן ההודעה
הלקוח שולח כתובת אתר לבדיקה	שרת	לקוח	טקסט הכולל כתובת URL מוצפנת
הלקוח שולח את חווית דירוג שימוש	שרת	לקוח	טקסט הכולל שם, ציון ופידבק מוצפנים
האדמין שולח בקשה לקבלת טבלת הדירוגים	שרת	אדמין	"GET RATINGS" מוצפן
האדמין שולח בקשה לקבלת ממוצע הדירוגים	שרת	אדמין	"GET AVRAGE_RATINGS" מוצפן
האדמין מבקש את עשר הבדיקות האחרונות	שרת	אדמין	"GET_HISTORY" מוצפן
האדמין מבקש את הרשימה השחורה	שרת	אדמין	"GET_BLACKLIST" מוצפן
האדמין מבקש להסיר קישור מהרשימה	שרת	אדמין	"REMOVE" מוצפן
האדמין מבקש לאפס את הרשימה השחורה	שרת	אדמין	"RESET_BLACKLIST" מוצפן
האדמין מבקש לאפס את רשימת הדירוגים	שרת	אדמין	"RESET_RATINGS" מוצפן
בקשה לניתוק מהשרת	שרת	לקוח	"DISCONNECT" מוצפן

המשך בעמוד הבא



מבנה השדות בהודעה	נשלחת אל-	נשלחת מ-	תיאור/תוכן ההודעה
השרת מחזיר ללקוח את תוצאות הקישור ששלח	לקוח	שרת	טקסט מוצפן עם תוצאות בדיקות הקישור
השרת מחזיר ללקוח אישור כי קלט את הדירוג	לקוח	שרת	טקסט מוצפן עם אישור קליטת הדירוג
השרת שולח לאדמין את טבלת הדירוגים	אדמין	שרת	טקסט מוצפן עם טבלת הדירוגים
השרת שולח לאדמין את ממוצע הדירוגים המחושב	אדמין	שרת	טקסט מוצפן עם ממוצע הדירוגים
השרת שולח לאדמין את עשר הבדיקות האחרונות	אדמין	שרת	טקסט מוצפן עם עשר הבדיקות האחרונות
השרת שולח לאדמין את הרשימה השחורה	אדמין	שרת	טקסט מוצפן עם הרשימה השחורה
השרת שולח לאדמין טקסט המאשר את עדכון/איפוס הרשימה	אדמין	שרת	טקסט מוצפן המאשר עדכון/איפוס
השרת מבשר על סיום תקין של החיבור	לקוח/אדמין	שרת	טקסט מוצפן המאשר ניתוק

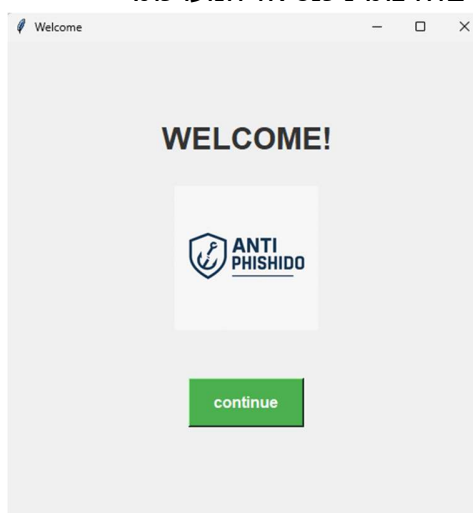


תיאור מסכי המערכת:

צד הלקוח

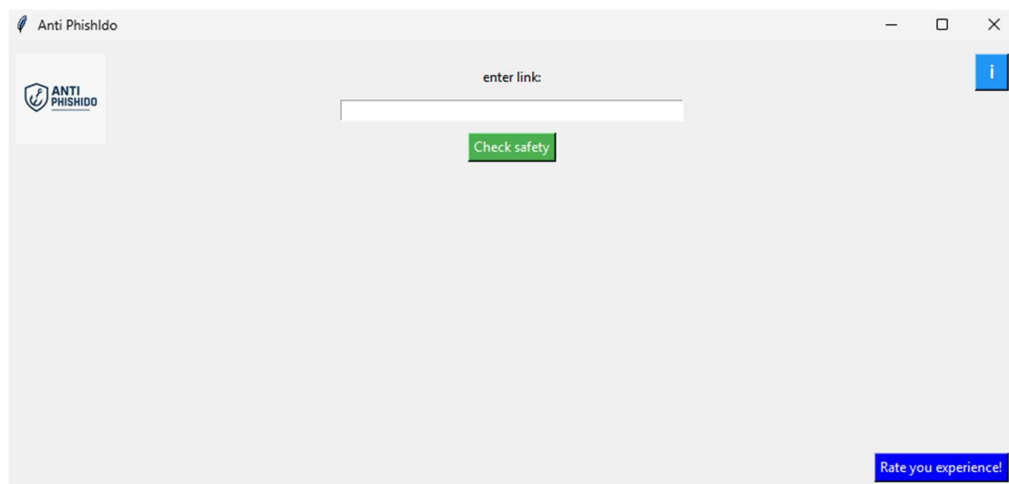
כותרת/ שם/ תפקיד המסך: Welcome-מסך פתיחה

תיאור המסך: בראש המסך טקסט – "WELCOME BACK" מתחת לטקסט, לוגו התוכנה. מתחתיו, כפתור "continue" אשר בלחיצתו ניכנס אל המערכת.



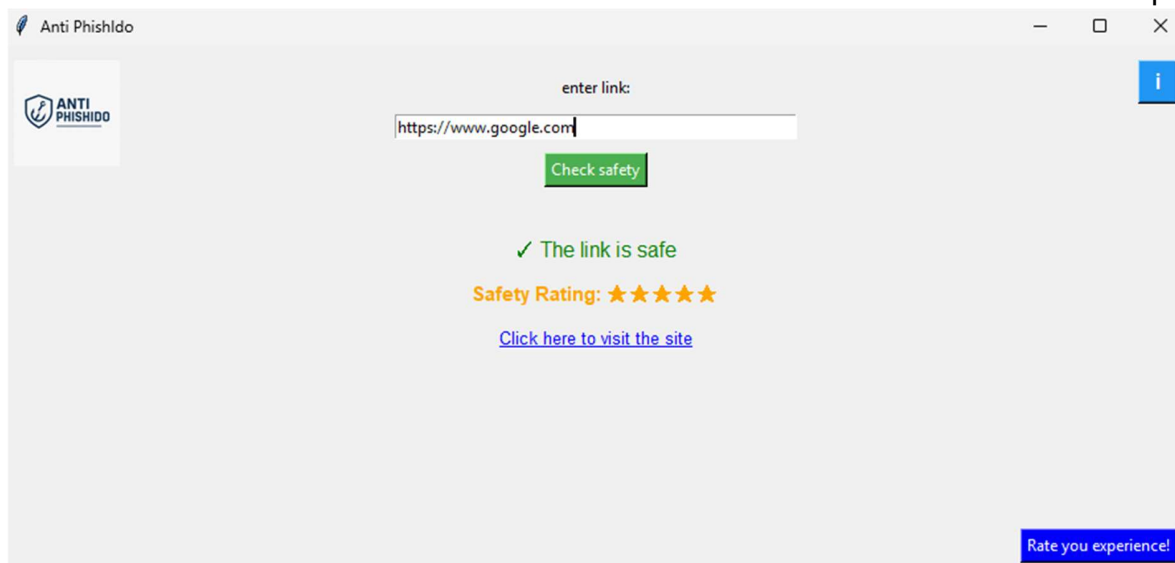
כותרת/ שם/ תפקיד המסך: Anti PhishIdo- המסך הראשי של המערכת

תיאור המסך: ניתן לראות מסך ובו שלל אופציות: מקום להקלדת קישור לבדיקה במרכז המסך. מתחתיו כפתור לבדיקת הקישור. מצד שמאל למעלה לוגו המערכת. מצד ימין למעלה כפתור i אשר מספק מידע (information) על המערכת בעת לחיצה. מצד ימין למטה כפתור "Rate me!" אשר בלחיצה עליו יוכל המשתמש לדרג את חווית שימוש

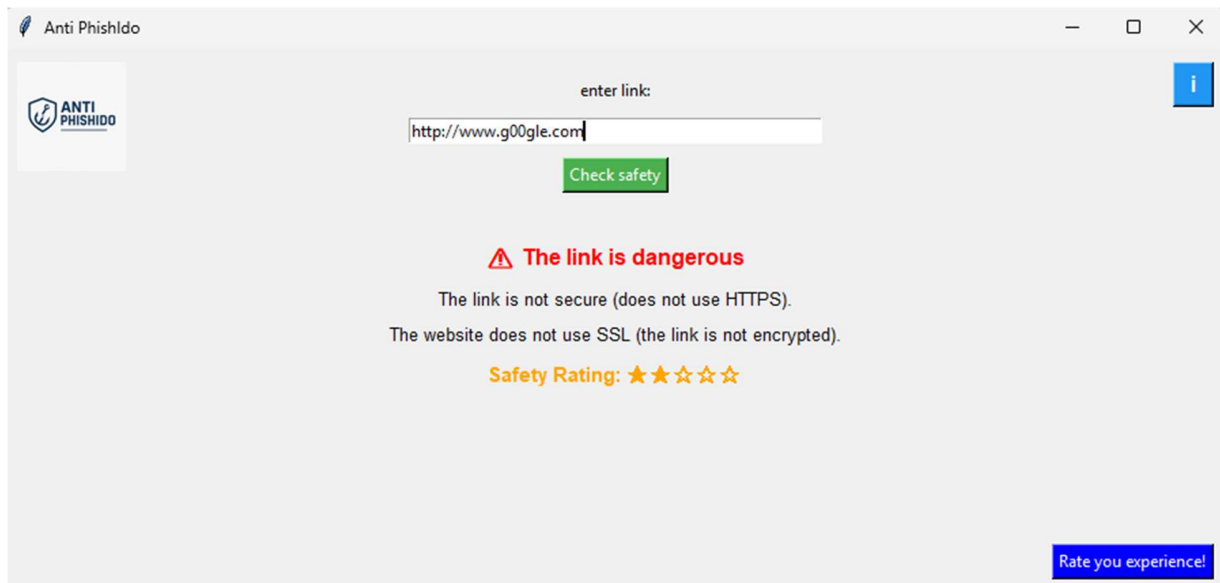


כאשר המשתמש יזין דירוג וילחץ על כפתור הבדיקה, ישנן כמה אופציות:

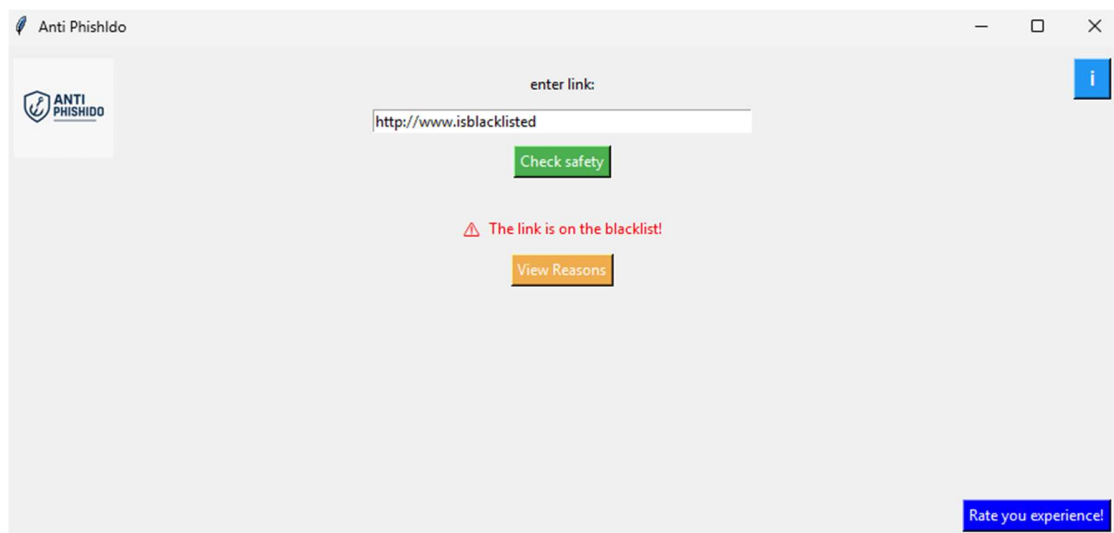
הקישור בטוח:



הקישור מסוכן:

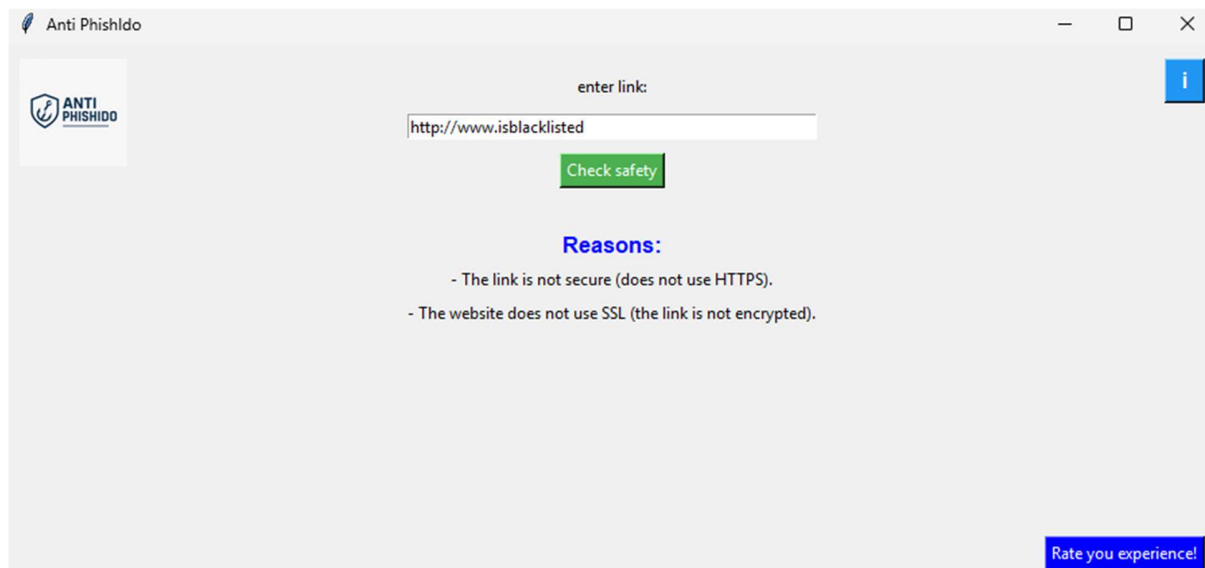


הקישור ברשימה שחורה:



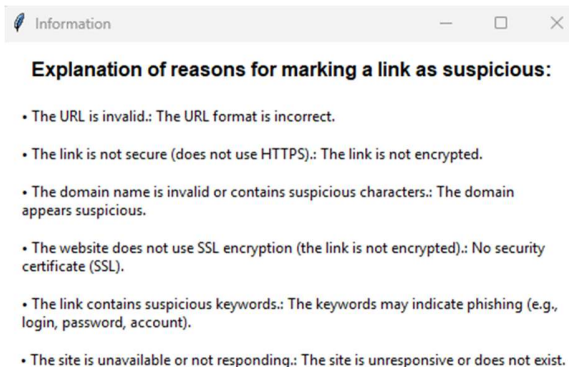


וכאשר הכפתור ילחץ:



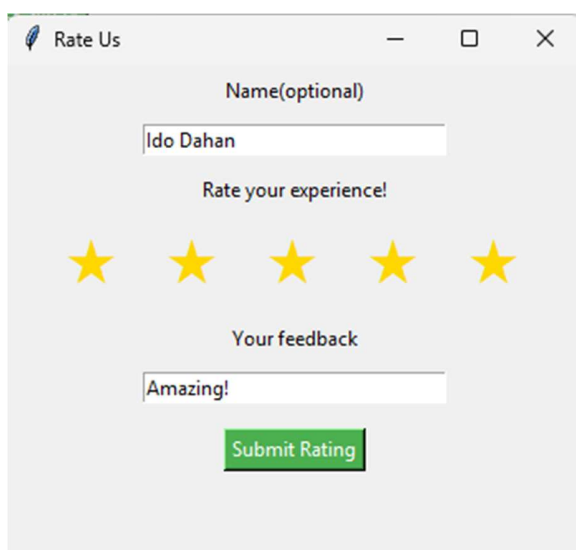
כותרת/ שם/ תפקיד המסך: Information- מידע על סיבות מדוע קישור יכול להיות חשוד.

תיאור המסך: ניתן לראות, בראשית המסך כותרת: "Explanation of reasons for marking link as suspicious". מתחת לכותרת, כמובן, מוצג בקצרה הסבר בקצרה על משמעות כל סיבה אשר הופכת קישור לחשוד ויכולה להתקבל במערכת, כלומר המסך משמש כ"מילון מושגים" למשתמש. המסך יתקבל בלחיצה על הכפתור "ז", אשר נמצא במסך הראשי בפינה הימנית העליונה.



כותרת/ שם/ תפקיד המסך: Rate Us- דירוג המשתמש

תיאור המסך – ניתן לראות את מערכת דירוג חווית המשתמש. ראשית, המשתמש יוכל להזין שם, אך ערך זה איננו חובה כמצוין (Name(optional)). שנית, המשתמש ידרג את חווית שימוש במערכת, מ-1 ועד 5 (Rate your experience!). שלישית, המשתמש ישאיר פידבק, כלומר, משוב מילולי על שימוש במערכת. כאשר ילחץ על הכפתור "Submit Rating", ישלח הדירוג.



Rate Us

Name(optional)

Ido Dahan

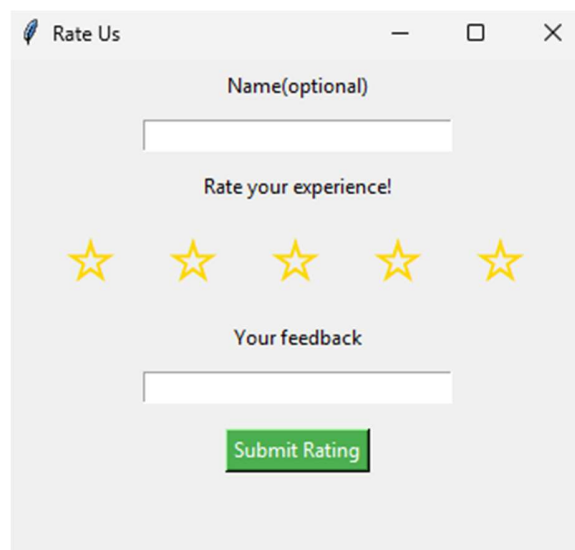
Rate your experience!

★★★★★

Your feedback

Amazing!

Submit Rating



Rate Us

Name(optional)

Rate your experience!

★★★★★

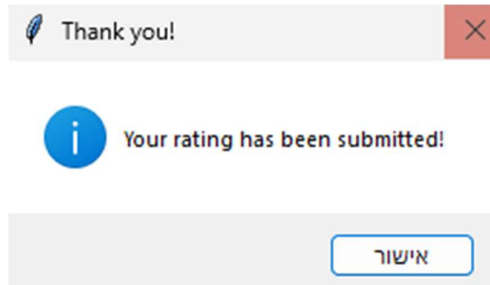
Your feedback

Submit Rating



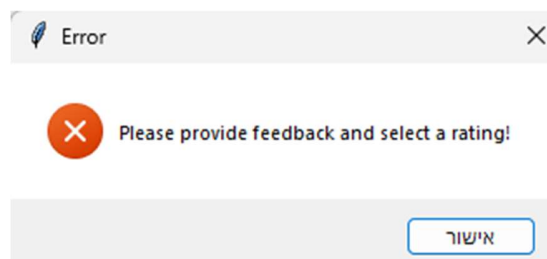
דרך מסך הדירוג, יתכנו שני תרחישים:

1. אישור "Your rating has been submitted!"



2. כותרת/ שם/ תפקיד המסך: Error - שגיאה בדירוג (חוסר נתונים)

תיאור המסך: הודעת "Please provide feedback and select a rating!", כלומר, המערכת מודיעה למשתמש שיש למלא דירוג ופידבק

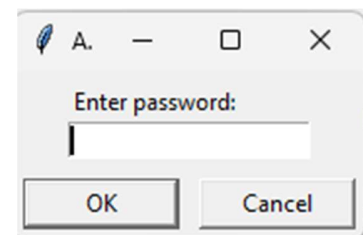




צד האדמין

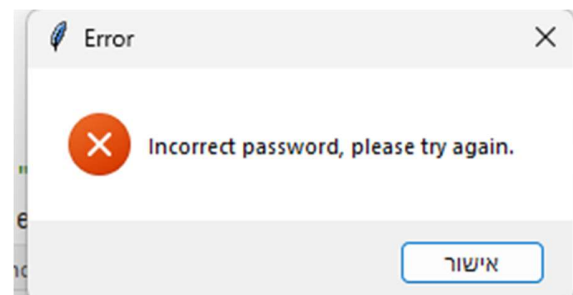
כותרת/ שם/ תפקיד המסך: Authentication- אימות האדמין.

תיאור המסך: המתחבר למערכת ידרש ראשית להזין סיסמה, הידוע רק לאדמין, על מנת לאמת את כניסתו למערכת.

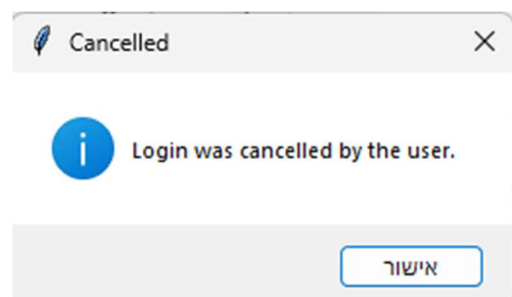


במידה ואותו המשתמש יזין את הסיסמה הנכונה, תיפתח בפניו המערכת.

במידה ויזין סיסמה שגויה:



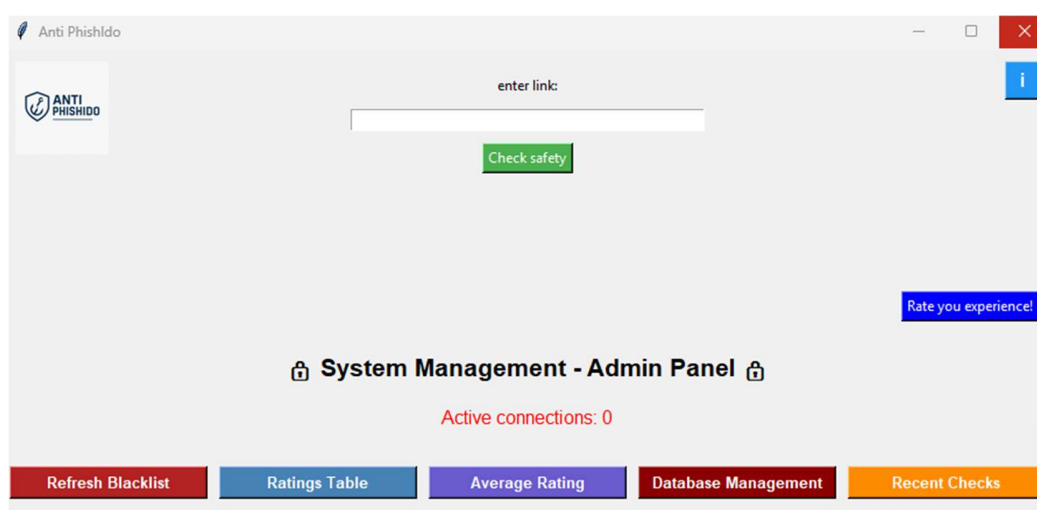
ובמידה ויסגור את החלון:





כותרת/ שם/ תפקיד המסך: מסך הניהול של האדמין

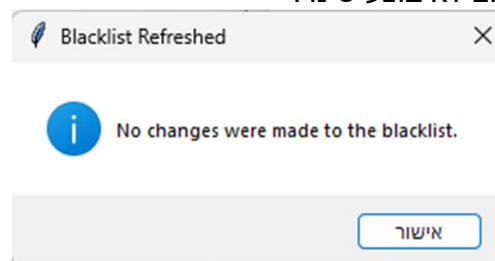
תיאור המסך: ניתן לראות את מסך הלקוח הרגיל, אך מתחתיו, נמצא פאנל האדמין, אשר דרכו יוכל האדמין לבצע ניהול של התוכנה. מעל הכפתורים מתנוססת הכותרת: "System management – Admin Panel". מתחת אל הכותרת, מספר אלמנטים. ראשית, נוכל לראות את מספר החיבורים הפעילים כרגע, שנית, נוכל לראות מספר יכולות ניהוליות של האדמין, כגון צפייה בטבלת הדירוגים, רענון הרשימה השחורה וכו'.



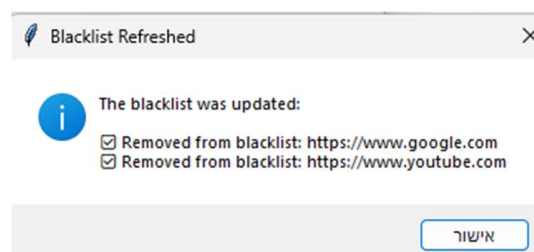
כותרת/ שם/ תפקיד המסך: Blacklist refreshed - תוצאות רענון הרשימה השחורה

תיאור המסך: בעת לחיצה על כפתור "Refresh Blacklist", הרשימה השחורה תעודכן בהתאם, ויוצגו הודעות מתאימות למסך:

אם לא בוצע שינוי:

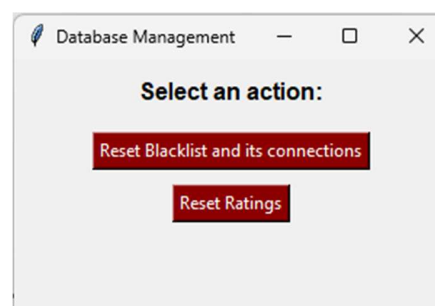


אם בוצע:

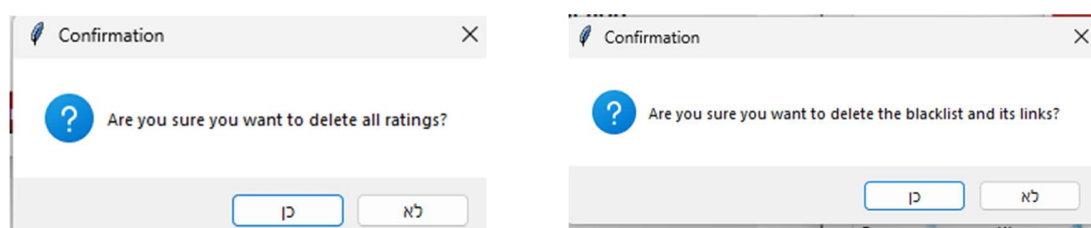


כותרת/ שם/ תפקיד המסך: Database Management- ניהול מסדי הנתונים

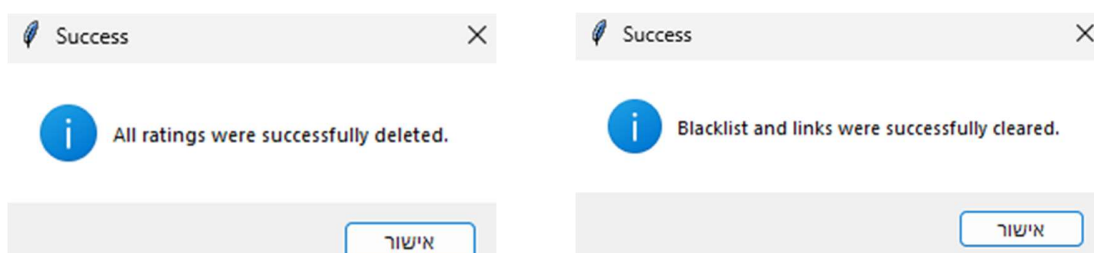
תיאור המסך: בעת הלחיצה על הכפתור בשם "Database Management", שבמסך הניהול, יפתח המסך הבא, שבראשיתו הכותרת "Select an action:". מתחתיה, שני כפתורים: הראשון מביניהם, "Reset Blacklist and its connections", כלומר איפוס הרשימה השחורה וקשריה. השני, "Reset Ratings", כלומר איפוס טבלת הדירוגים.



כאשר נלחץ על אחד מהכפתורים, יפתח המסך הבא בשם confirmation, כלומר, אישור. במסך מוצגת הודעה – "Are you sure you want to delete the blacklist and its links?", או ההודעה "Are you sure you want to delete all ratings?", ובנוסף כפתורים של "כן" או "לא".

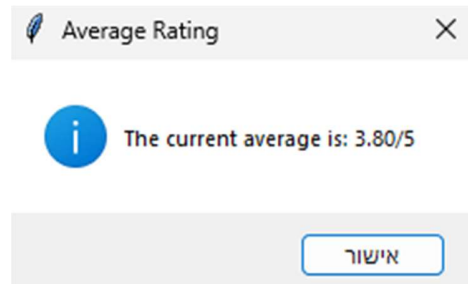


במידה ונלחץ "לא", המסך חיסגר. במקרה ונלחץ "כן":





כותרת/ שם/ תפקיד המסך: Average Rating - ממוצע דירוגי המשתמשים
תיאור המסך: המסך יפתח בעת הלחיצה על כפתור "Average Rating" במסך הניהול. במסך מופיעה הודעה על ממוצע הדירוגים הנוכחי של טבלת הדירוגים.



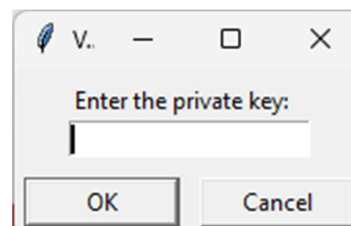
כותרת/ שם/ תפקיד המסך: Ratings Table - טבלת הדירוגים.
תיאור המסך: נוכל לראות במסך את טבלת דירוגי המשתמשים, הכוללת ID, שם, דירוג ופידבק.

ID	Name	Rating	Feedback
1	Ido Dahan	5	Amazing!
2	Idan Feldberg	5	Wow!
3	Rom Rapson	3	meh
4	Anonymous	1	it sucks!
5	Galit Barzam	5	Beautiful!!

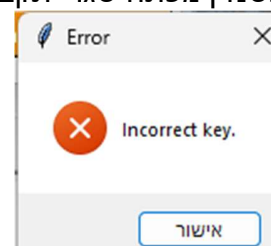


כותרת/ שם/ תפקיד המסך: Verify Private Key- אימות מפתח פרטי.

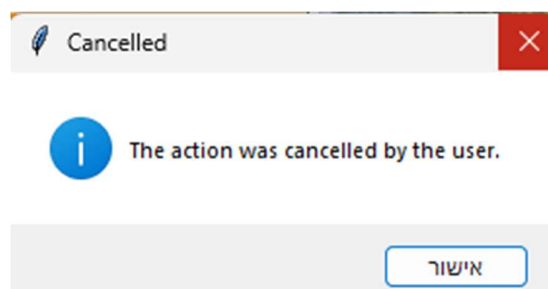
תיאור המסך: כדי לצפות בטבלת ההיסטוריה (10 הבדיקות האחרונות), יש להזין מפתח פרטי, כיוון שבטבלה כתובות קו של משתמשים מוצפנות, והמפתי יוכל לפענח אותן. זוהי בעצם "הגנה כפולה" על פרטיות המשתמשים. במסך כפתור "ok" לאישור, וכפתור "cancel" לביטול.



כשנזין מפתח שגוי יתקבל המסך הבא:



כשנסגור את המסך:



כותרת/ שם/ תפקיד המסך: Recent 10 Checks - עשר הבדיקות האחרונות שבוצעו

תיאור המסך: במסך ניתן לראות את טבלת עשר הבדיקות האחרונות שבוצעו, הכוללת כתובת IP מפוענחת של המשתמש, פורט, הקישור שנבדק, האם הוא חשוד, ומתי הוא נבדק

Recent 10 Checks				
IP	Port	Link	Suspicious	Timestamp
192.168.8.143	52693	https://www.whatsapp.com	False	2025-05-22 20:00:00
192.168.8.143	52693	www.phishid0	True	2025-05-22 19:59:47
192.168.8.143	52693	http://www.yout0be	True	2025-05-22 19:59:23
192.168.8.143	52693	www.hello	True	2025-05-22 19:59:04
192.168.8.143	52582	https://www.youtube.com	False	2025-05-22 19:21:49
192.168.8.143	52582	https://www.google.com	False	2025-05-22 19:21:49
192.168.8.143	52582	http://www.login	True	2025-05-22 19:21:48
192.168.8.143	52582	http://www.isblacklisted	True	2025-05-22 19:21:48
192.168.8.143	52582	http://www.isblacklisted?	True	2025-05-22 19:21:48
192.168.8.143	52582	http://www.g00gle.com	True	2025-05-22 19:21:48



תיאור מבני הנתונים:

פירוט מבני הנתונים- מסדי נתונים מסוג SQLite:

-מסד נתונים בשרת – אשר משמש לאחסון המידע הרשמי של המערכת

פירוט מאגרי המידע של המערכת:

1. BLACKLIST- רשימה שחורה, מכילה את הקישורים שסומנו כחשודים במערכת. ישנו מזהה אישי לכל קישור
2. REASONS- רשימת הסיבות, מכילה את כל הסיבות המוגדרות במערכת לסימון קישור כחשוד, וספירה של כמה פעמים הופיעה כל סיבה. ישנו מזהה יחודי לכל סיבה.
3. LINKS_REASONS- מאגר של קשרים בין קישורים לסיבות, כלומר אילו סיבות משוייכות לאילו קישורים. המאגר יוצר קשה של רבים-לרבים (N:N) בין הרשימה השחורה לטבלת הסיבות
4. RATINGS- טבלת הדירוגים, מאגר של דירוגי חוויות המשתמשים במערכת. כוללת שם המדרג(אופציונלי), ציון (1-5) ופידבק
5. HISTORY- מאגר היסטוריית הבדיקות של קישורים שהתבצעו במערכת. כוללת את כתובת הקו של המשתמש(מוצפן), פורט, כתובת שנבדקה, סטטוס(חשוד או לא) וזמן הבדיקה.



מסדי נתונים:

1. טבלת BLACKLIST.

שדה: LINK – כתובת האתר

טיפוס: TEXT

אורך: ללא הגבלה

ערך אפשרי: <https://www.google.com>

שדה: LINK_ID - מזהה יחודי לקישור

טיפוס: INTEGER

אורך: ללא הגבלה

ערך אפשרי: 99

2. טבלת REASONS

שדה: REASON_ID – מזהה יחודי לסיבה

טיפוס: INTEGER

אורך: ללא הגבלה

ערך אפשרי: 1/2/3/4/5

שדה: NAME - תיאור הסיבה

טיפוס: TEXT

אורך: ללא הגבלה

ערך אפשרי: ה-URL לא תקין/ הקישור לא מאובטח (לא משתמש ב-HTTPS). / שם הדומיין לא תקין או מכיל תווים חשודים. / האתר לא משתמש בהצפנת SSL (הקישור לא מוצפן). / הקישור מכיל מילות מפתח חשודות (לדוגמה: login, password, account).



שדה: COUNT – מספר הפעמים שהסיבה הופיעה

טיפוס: INTEGER

אורך: ללא הגבלה

ערך אפשרי: 10

3. טבלת LINKS_REASONS

שדה: LINK_ID - מזהה הקישור

טיפוס: INTEGER

אורך: ללא הגבלה

ערך אפשרי: 111

שדה: REASON_ID - מזהה הסיבה

טיפוס: INTEGER

אורך: ללא הגבלה

ערך אפשרי: 5

4. טבלת RATINGS

שדה: NAME

טיפוס: TEXT

אורך: ללא הגבלה

ערך אפשרי: Ido Dahan

שדה: RATING

טיפוס: INTEGER

אורך: ללא הגבלה

ערך אפשרי: 5



שדה: FEEDBACK
 טיפוס: TEXT
 אורך: ללא הגבלה
 ערך אפשרי: you saved me!!!

5. טבלת history

שדה: ip (מוצפן)
 טיפוס: TEXT
 אורך: ללא הגבלה
 ערך אפשרי: 2906,1175,538,2825,2906,1962,1794,2825,1794,2825,2906,538,2906

שדה: Port
 טיפוס: INTEGER
 אורך: ללא הגבלה
 ערך אפשרי: 8002

שדה: link
 טיפוס: TEXT
 אורך: ללא הגבלה
 ערך אפשרי: <https://www.facebook.com>

שדה: is_suspicious
 טיפוס: TEXT
 אורך: ללא הגבלה
 ערך אפשרי: Yes/No



שדה: timestamp

טיפוס: TEXT

אורך: פורמט של HH:MM:SS YYYY-MM-DD

ערך אפשרי: 20:04:54 2025-05-14

סקירת חולשות והאיומים:

שכבת האפליקציה:

1. עבודה עם בסיס נתונים - SQL Injection (למניעת הזרקת קוד למסד):
המערכת משתמשת בparameterized queries, כלומר, שימוש בסימני שאלה (?) בשאילתות, כדי להפריד בין קוד לקלט ולמנוע הזרקת קוד עוין.
2. עבודה עם אתרי web- ניתוח URL חשודים (למניעת הונאות פישנינג):
המערכת בודקת כתובות URL על בסיס מאפיינים שונים כדי למנוי פישנינג
3. תהליך login- לאדמין (למניעת גישה לא מורשית):
המערכת מאמתת את האדמין באמצעות סיסמה מוצפנת בhash ומאוחסת כ environment variable כלומר משתנה סביבה. (בכניסה למערכת הניהול האדמין מזין סיסמה. היא עוברת הצפנת hash. אם היא זהה לערך משתנה הסביבה (שהוא ערך מסוים של הצפנת hash של הסיסמה המקורית), המנהל נכנס למערכת)
4. MITM (להגנה על פרטיות המידע): המערכת מבצעת הצפנה מקצה לקצה באמצעות RSA, כך שגם אם התקשורת תיורט, המידע אינו ניתן לקריאה.
5. DDOS- עומס על השרת (לשמירה על זמינות המערכת): השרת מגביל את כמות בקשות החיבור למקסימום של 10 התחברויות ב10 שניות, כדי למנוע עומס.
6. HASH (לאימות זהות המנהל): המערכת אינה תומכת בהעלאת קבצים, אך מבצעת שימוש בHASH כדי לאמת את סיסמת האדמין.

שכבת התעבורה:

המערכת משתמשת בפרוטוקול TCP/IP לניהול התקשורת בין השרת לבין הלקוח. TCP מספק חיבור אמין, הכולל לחיצת יד תלת שלבית – three-way handshake:

1. SYN- הלקוח שולח בקשה לפתיחת חיבור
2. SYN-ACK – השרת מאשר ומחזיר תגובה
3. ACK- הלקוח מאשר את אישור השרת והחיבור נפתח

מנגנון זה מבטיח סנכרון ואמינות בקבלת ושליחת ההודעות.



הצפנה:

המערכת משתמשת בהצפנה אסימטרית, הנקראת גם הצפנת RSA. ההצפנה מתבצעת לפני שליחת כל הודעה העוברת בין השרת לבין הלקוח:

-הלקוח שולח לשרת את המפתח הציבורי שלו

-השרת שולח את המפתח הציבורי שלו ללקוח

*כך כל צד מצפין אך ורק עם המפתח של הצד השני, מה שמבטיח שרק הצד המתאים יוכל לראות את המידע

הצפנה זו מגנה מפני MITM – יירוט, ומבטיחה את סודיות המידע

הפעלת המערכת:

בצד השרת: השרת מופעל, מאזין לחיבורים ומבצע החלפת מפתחות RSA עם הלקוח. טוען את מסדי הנתונים וממתין לפעולות מהלקוחות או מהאדמין

בצד הלקוח: הלקוח יוצר חיבור TCP לשרת, מחליף מפתחות RSA, ופותח את הממשק הגרפי. המשתמש יכול לבדוק קישורים, לדרג חוויה או להיכנס כמנהל. כל הנתונים נשלחים בהצפנה והשרת מגיב עם בקרת עומס (מנגנון שבדוק כמה בקשות מתקבלות בפרק זמן) אשר מונעת מהשרת לקרוס בשל ריבוי של חיבורים זדוניים.

בסיום השימוש המערכת סוגרת את החיבור והמשאבים בצורה תקינה.

אילו חולשות קיימות ואיך טופלו:

1. SQL_Injection: טופל באמצעות parameterized queries, כלומר שימוש בסימני שאלה למניעת הזרקת קוד למסדי הנתונים
2. פשינג וקישורים מסוכנים: טופל באמצעות ניתוח URL על בסיס מאפיינים חשודים
3. גישה לא מורשית למנהל: טופל באמצעות סיסמת שהמנהל צריך להזין
4. יירוט תקשורת (MITM): טופל באמצעות הצפנת RSA מקצה לקצה
5. DDoS: טופל על ידי מגבלה של
6. 10 התחברויות ב-10 שניות.

מימוש הפרוייקט

חלק א'- סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם

מודולים/מחלקות מיובאים:

שם המודול המיובא	למה מיועד המודול
socket	ניהול חיבורי TCP/IP בין הלקוח לשרת.
threading	הפעלת חיבורים מרובים ובמקביל לשרת.
os	קריאת משתני סביבה. (environment variables)
hashlib	חישוב ערכי HASH לסיסמאות.
datetime	מציאת זמן מדויק (למשל לטבלת history)
time	השהיית פעולות ובקרת קצב התחברויות
re	בדיקות על טקסט באמצעות ביטויים רגולריים (לדוגמה לאיתור מילים חשודות בקישורים)
math	חישובים מתמטיים (לצורך RSA)
random	יצירת מספרים אקראיים (לצורך RSA)
sqlite3	ניהול מסדי נתונים ב-SQLite
requests	שליחת בקשות HTTP ובדיקות זמינות אתרים ו-SSL
urllib.parse(urllparse)	פירוק כתובת URL לחלקיה
tkinter	בניית הממשק הגרפי של התוכנה
tkinter.simpledialog	פתיחת חלונות קלט למשתמש (למשל לבקשת סיסמה)
tkinter.messagebox	הצגת הודעות/אזהרות/שגיאות למשתמש
tkinter.Toplevel	יצירת חלון עצמאי נוסף (לדוגמה צפייה בטבלאות)
Text	יצירת שדה טקסט גדול (למשל תצוגת טבלאות)
WORD, BOTH	קביעת אפשרויות גלילה ועיצוב בשדה טקסט



מודולים/מחלקות שפיתחתי לכל מחלקה:

שם המחלקה: Server

תפקיד המחלקה: תפקיד מחלקת השרת הוא להפעיל את השרת הראשי של המערכת, לנהל חיבורים נכנסים מלקוחות הרוצים להשתמש במערכת או מן המנהל, לקבל בקשות לבדיקה או ניהול, לבצע את הבדיקות על הקישורים שנשלחים לשרת, לנהל את מסדי הנתונים של המערכת ולהחזיר תשובות מוצפנות ללקוח.

תכונות המחלקה:

שם תכונה: self.ip

-תפקיד התכונה: כתובת הIp של השרת

שם תכונה: self.port

-תפקיד התכונה: מספר הפורט בוא השרת מאזין לחיבורים

שם תכונה: self.socket

-תפקיד התכונה: אובייקט socket המאזין לחיבורים נכנסים

שם תכונה: self.encryption

-תפקיד התכונה: אובייקט RSA האחראי להצפנה ולפענוח של הודעות

שם תכונה: self.db_name

-תפקיד התכונה: שם קובץ מסד הנתונים בו מאוחסן המידע

שם תכונה: self.issue

-תפקיד התכונה: מילון המכיל משקל של כל סיבת חשד לקישור (לחישוב דירוג הבטיחות)

שם תכונה: self.url_validator

-תפקיד התכונה: אובייקט URL_Validator שמבצע בדיקות תקינות וחשדנות על כתובות URL

שם תכונה: self.connection_count



-תפקיד התכונה: ספירת מספר החיבורים הפעילים לשרת (לניהול עומס על השרת ובקרת התחברויות)

*מחלקת Server יורשת ממחלקת Communication ולכן בנוסף לתכונות הייחודיות לה כמנהל צד השרת, היא כוללת גם את כל תכונות התקשורת הכלליות כמו כתובת ה IP, הפורט, ניהול חיבור ה-TCP, סטטוס החיבור (מחובר/מנותק) ועוד שיפורטו בהמשך. הירושה מאפשרת לשרת לקבל ולשלוח הודעות בצורה מאובטחת ללקוחות המחוברים אליו, תוך שימוש במנגנוני ההצפנה והתקשורת שהוגדרו במחלקת Communication.

פעולות במחלקה:

שם פעולה: __init__

טענת כניסה:

1. ip - כתובת ה-ip של השרת
2. Port - פורט החיבור של השרת

טענת יציאה: פעולה בונה, אשר מאתחלת את כל תכונות השרת: יצירת אובייקט socket להאזנה, יצירת מנגנוני הצפנה (RSA), טעינת שם מסד הנתונים, הגדרת משקלים לחשדות, יצירת אובייקט לבדיקת קישורים והגדרת מונה החיבורים.

שם פעולה: send_large_message

טענת כניסה:

1. Client_socket – החיבור הפעיל עם לקוח
2. message -ההודעה המיועדת לשליחה.

טענת יציאה: הפעולה בעצם שולחת הודעה מפוצלת למקטעים (צ'אנקים) בגודל של 1024 בתים ללקוח.

שם פעולה: check_url

טענת כניסה:

1. url - כתובת ה URL לבדיקה
2. Database - אובייקט מסד הנתונים
3. Update_database - ערך בוליאני שקובע האם לעדכן את מסד הנתונים (ברירת מחדל TRUE)

טענה יציאה: הפעולה מבצעת סדרת בדיקות על הקישור, מחשבת את רמת סיכנו ומחזירה את הסיבות ודירוג בטיחותו של הקישור.



שם פעולה: `rate_url`

טענת כניסה:

1. Reasons - רשימת הסיבות שזוהו עבור הקישור

טענה יציאה: הפעולה מחזירה את דירוג הבטיחות של הקישר (ציון בין 1 ל-5) לפי משקלי הסיבות.

שם פעולה: `handle_client`

טענת כניסה:

1. `Client_socket` – החיבור ללקוח

2. `Address` - כתובת ה-IP והפורט של הלקוח.

טענה יציאה: הפעולה מטפלת בתקשורת עם הלקוח, כלומר: החלפת מפתחות RSA, קבלת בקשות מהלקוח, עיבודן ושליחתן באמצעות הפונקציות המתאימות, שולח תגובות ללקוח ומעכן את המסד.

שם פעולה: `start_server`

טענת כניסה: אין

טענת יציאה: הפעולה מתחילה את פעולת השרת – מאזינה לחיבורים נכנסים ומטפלת בכל חיבור חדש בעזרת `thread` נפרד, על מנת לאפשר ניהול של מספר לקוחות במקביל מבלי לעצור את ההאזנה.

שם המחלקה: `Client`

תפקיד המחלקה: מחלקת הלקוח מייצגת את הצד של המשתמש במערכת. היא אחראית על כמה דברים: יצירת חיבור מאובטח לשרת, שליחת כתובות URL לבדיקה, הצפנה ושליחת דירוג חווית המשתמש לשרת, קבלת תגובות מוצפנות מהשרת, פיענוחן ושליחתן למשתמש. שמירה על היסטוריית הבדיקות והדירוגים במסד נתונים מקומי, כדי לאפשר עבודה גם כאשר השרת אינו זמין. כמו כן הצגת הממשק הגרפי למשתמש

תכונות המחלקה

שם התכונה: `self.root`

תפקיד התכונה: חלון `tkinter` הראשי לממשק המשתמש.

שם התכונה: `self.encryption`



תפקיד התכונה: מופע של מחלקת RSA, מייצר מפתחות הצפנה ציבוריים ופרטיים.

שם התכונה: `self.server_e`

תפקיד התכונה: המפתח הציבורי של השרת

שם התכונה: `self.server_n`

תפקיד התכונה: המודולו הציבורי של השרת

שם התכונה: `self.database`

תפקיד התכונה: מופע של Databases, מטפל במסד הנתונים המקומי של הלקוח.

שם התכונה: `self.gui`

תפקיד התכונה: מופע של GUI, הממשק הגרפי שנבנה עבור המשתמש.

* מחלקת Client יורשת ממחלקת Communication, ולכן בנוסף לתכונות הייחודיות לה, היא כוללת גם את כל תכונות התקשורת הכלליות כמו כתובת ה IP-הפורט, ניהול חיבור ה TCP-סטטוס החיבור (מחובר/מנותק) ועוד שיפורטו בהמשך. ירושה זו מאפשרת ללקוח לנהל את התקשורת מול השרת בצורה פשוטה וישירה.

פעולות במחלקה:

שם פעולה: `__init__`

טענת כניסה:

1. Root- מופע tkinter ליצירת הממשק

טענת יציאה: הפעולה מאתחלת את הלקוח: יוצרת מפתחות RSA (פרטי וציבורי), בונה את הממשק הגרפי ומבצעת התחברות לשרת כולל החלפת מפתחות הצפנה.

שם פעולה: `connect_to_server`

טענת כניסה: אין



טענט יציאה: הפעולת יוצרת חיבור TCP לשרת ומבצעת תהליך החלפת מפתחות הצפנה: ראשית, שולחת לשרת את המפתח הציבורי של הלקוח. שנית, מקבלת מהשרת את המפתח הציבורי של השרת. לבסוף, שומרת את המפתחות שהתקבלו לשימוש עתידי להצפנת ההודעות.

שם פעולה: receive_large_message

טענת כניסה:

1. Timeout - זמן מקסימלי לקריאת הודעה מהשרת

טענת יציאה: קורא הודעה מהשרת עד לתגית <END> ומחזיר את תוכן ההודעה, או NONE במקרה של כשל.

שם פעולה: send_url_check_request

טענת כניסה:

1. url – כתובת ה URL לבדיקה

טענת יציאה: הפעולה שולחת את כתובת ה URL כשהיא מוצפנת לשרת, מקבלת תשובה מוצפנת עם תוצאות ניתוח ומחזירה אותן.

שם הפעולה: request_link_reasons

טענת כניסה:

1. Link - כתובת URL שמבקשים לקבל את סיבות הסימון של כחשודה.

טענת יציאה: הפעולה שולחת בקשה לשרת לקבלת הסיבות, מפענחת את התשובה ומחזירה רשימת סיבות.

שם פעולה: submit_rating

טענת כניסה:

1. Name – שם המדרך

2. Rating - ציון בין 1 ל-5

3. Feedback - משוב טקסטואלי

טענת יציאה: הפעולה שולחת את פרטי הדירוג לשרת כשהם מוצפנים

שם פעולה: disconnect_and_exit



טענת כניסה: אין

טענת יציאה: הפעולה סוגרת את החיבור לשרת, סוגרת את מסד הנתונים המקומי של הלקוח וסוגרת את הממשק הגרפי

שם המחלקה: AdminClient

תפקיד המחלקה: תפקיד המחלקה הוא לאפשר למנהל המערכת לבצע פעולות יעודיות שיבחר דרך ממשק יעודי לכך. המנהל יכול לצפות בטבלת עשר הבדיקות האחרונות, לאפס רשימות, לצפות בטבלת הדירוגים וכן בממוצע הדירוגים ולרענן את הרשימה השחורה. בנוסף המנהל יכול לראות את מספר החיבורים הפעילים ברגע נתון. למנהל קיימות כל הפיצ'רים שקיימת במחלקה client.

תכונות המחלקה:

שם התכונה: self.root

תפקיד התכונה: חלון tkinter הראשי של ממשק הניהול

שם התכונה: self.connection_count_label

תפקיד התכונה: תווית אשר מציגה את מספר החיבור הפעילים בממשק הגרפי. (נוצרת כחלה מהפעלת add_admin_controls() במהלך האתחול.

*מחלקת AdminClient יורשת את מחלקת Client ו communication (יורשת את מחלקת הלקוח אשר יורשת את מחלקת התקשורת), ולכן כוללת גם את כל תכונות התקשורת, ההצפנה וניהול החיבור שהוגדרו במחלקות אלו.

פעולות המחלקה:

שם הפעולה: __init__

טענת כניסה:

1. root - מופע tkinter של המסך הראשי

טענת יציאה: הפעולה מאמתת סיסמת מנהל בשימוש HashManager. במקרה של אישור קוראת ללקוח לאתחול תכונות נוספות. מוסיפה רכיבי ניהול יחודיים לממשק (כפתורי הניהול)



שם הפעולה: add_admin_controls

טענת כניסה: אין

טענת יציאה: הפעולה בונה את ממשק הניהול: כותרת, תצוגת חיבורים פועלים וכפתורי הניהול.

שם הפעולה: open_database_menegement_window

טענת כניסה: אין

טענת יציאה: הפעולה פותחת חלון לבחירת פעולת איפוס, של הרשימה השחורה או של רשימת הדירוגים

שם הפעולה: reset_blacklist

טענת כניסה: אין

טענת יציאה: הפעולה מאפסת את הרשימה השחורה במסד הנתונים המקומי ומציג הודעת אישור או שגיאה.

שם הפעולה: reset_ratings

טענת כניסה: אין

טענת יציאה: הפעולה מאפסת את טבלת הדירוגים במסד הנתונים המקומי ומציגה הודעת אישור או שגיאה.

שם הפעולה: confirm_reset_blacklist

טענת כניסה: אין

טענת יציאה: הפעולה מציגה חלון לאישור לפני מחיקת הרשימה. אם הכפתור נלחץ, הפעולה קוראת ל reset_blacklist שמאפסת

שם הפעולה: confirm_reset_ratings

טענת כניסה: אין

טענת יציאה: הפעולה מציגה חלון לאישור לפני מחיקת הרשימה. אם הכפתור נלחץ, הפעולה קוראת ל reset_ratings שמאפסת



שם הפעולה: refresh_blacklist

טענת כניסה: אין

טענת יציאה: הפעולה שולחת בקשה לקבלת הרשימה השחורה מהשרת. לאחר מכן, בודקת בבדיקה חיה את כל אחד מן הקישורים דרך הבדיקות בשרת. במידה וקישור נמצא כתקין, הוא יוסר מן הרשימה וגם קשריו ימחקו.

שם הפעולה: update_connection_count

טענת כניסה: אין

טענת יציאה: הפעולה שולחת בקשה לשרת לקבל כמות חיבורים פעילים בכל חמש שניות ומציגה את התוצאה בממשק.

שם הפעולה: check_url_direct

טענת כניסה:

1. Url - כתובת ה URL לבדיקה ישירה מול השרת

טענת יציאה: שולחת את כתובת ה URL-לשרת, מקבל ומחזיר את תוצאות הבדיקה. פעולה זו מיועדת לשימוש פנימי במנגנון רענון הרשימה השחורה בלבד, ואינה זמינה למשתמש הרגיל.

שם הפעולה: show_rating_window

טענת כניסה: אין

טענת יציאה: מציגה חלון גרפי עם דירוגי המשתמשים מהשרת.

שם הפעולה: show_average_rating

טענת כניסה: אין

טענת יציאה: מבקש מהשרת את ממוצע הדירוגים ומציגה אותו בחלון הודעה.



שם הפעולה: `show_last_10_checks`

טענת כניסה: אין

טענת יציאה: ראשית מתבצע אימות מפתח פרטי של המנהל. במקרה שהמפתח נכון, נשלחת בקשה לשרת לקבלת עשר הבדיקות האחרונות. לאחר מכן כתובות הקן מפוענחות באמצעות אותו המפתח שהוזן ועשר הבדיקות האחרונות המפוענחות מוצגות למנהל, כאשר הוא היחיד שיכול לראות את כתובות הקן האמיתיות.

שם המחלקה : communication

תפקיד המחלקה: לספק תשתית בסיסית של תקשורת בין שרת ללקוח באמצעות פרוטוקול TCP.

המחלקה אחראית על פתיחת חיבורים, שליחת הודעות, קבלת הודעות, ניתוק מהשרת וניהול סטטוס החיבור.

תכונות המחלקה:

שם תכונה: `self.ip`

תפקיד התכונה: כתובת הקן אליה יתחבר הלקוח או שעליה יאזין השרת.

שם תכונה: `self.port`

תפקיד התכונה: הפורט שישמש את החיבור או את ההאזנה.

שם תכונה: `self.is_server`

תפקיד התכונה: מצין האם האובייקט מייצג שרת או לקוח (ערך בוליאני).

שם תכונה: `self.socket`

תפקיד התכונה: אובייקט מסוג `socket` לניהול החיבור.

שם תכונה: `self.connected`

תפקיד התכונה: סטטוס החיבור (`true` אם מחובר, `false` אם אחרת)

פעולות המחלקה:

שם הפעולה: `__init__`

טענת כניסה:

1. `ip` - כתובת הקן

2. `Port` - מספר הפורט

3. `is_server` - ערך המסמן האם מדובר בשרת או בלקוח (ברירת המחדל `false`)

טענת יציאה: פעולה בונה, מאתחלת את כתובת הקן, הפורט, מצב השרת/לקוח, אובייקט `socket` וסטטוס החיבור.



שם הפעולה: connect

טענת כניסה: אין

טענת יציאה: הפעולה מנסה להתחבר לשרת אם זהו לקוח ומעדכנת את סטטוס החיבור בהתאם לתוצאה.

שם הפעולה: send_message

טענת כניסה:

1. Message- ההודעה המיועדת לשליחה

טענת יציאה: הפעולה שולחת את ההודעה לכתובת החיבור. במידה ולא קיים חיבור תנסה להתחבר מחדש.

שם הפעולה: receive_message

טענת כניסה:

1. Buffer_size- גודל חבילת הנתונים לקריאה (ברירת המחדל 1024 בתים)

טענת יציאה- הפעולה קוראת את ההודעה מהחיבור הקיים ומחזירה אותה כמחרוזת, במקרה של כשל תחזיר None.

שם הפעולה: send_public_key

טענת כניסה:

1. Public_key- מפתח ציבורי (נכתב בפורמט $((e,n))$)

טענת יציאה: שולחת את המפתח הציבורי ללקוח במקרה של שרת.

שם הפעולה: receive_public_key

טענת כניסה: אין

טענת יציאה: הפעולה קוראת את המפתח הציבורי מהשרת במקרה של לקוח ומחזירה אותו בפורמו המתאים $((e,n))$.



שם הפעולה: disconnect

טענת כניסה: אין

טענת יציאה: הפעולה סוגרת את החיבור ומעדכנת את סטטוס החיבור כמנותק

שם המחלקה: Databases

תפקיד המחלקה: מחלקת Databases מספקת ממשק לניהול בסיס הנתונים של המערכת באמצעות תוכנת SQLite. היא אחראית על יצירה ותחזוקה של טבלאות, ניהול רשימה שחורה (Blacklist), ניהול סיבות לסכנה, שמירת דירוגי המשתמשים ושמירה של היסטוריית הפעולות של המשתמשים.

תכונות המחלקה:

שם התכונה: self.conn

תפקיד: אובייקט החיבור למסד הנתונים של SQLite

שם התכונה: self.cursor

תפקיד: אובייקט דרכו מבוצעות שאלות למסד הנתונים.

שם התכונה: VALID_REASONS

תפקיד: מילון קבוע של סיבות אפשריות לסימון קישור כחשוד (משמש לאימות תקינות הסיבות).

פעולות המחלקה:

שם הפעולה: __init__
טענת כניסה:

1. db_name - שם קובץ מסד הנתונים

טענת יציאה: פעולה בונה, יוצרת חיבור למסד הנתונים, מאתחלת את הטבלאות במידת הצורך, ומעדכנת את רשימת הסיבות התקניות.

שם הפעולה: create_tables

טענת כניסה: אין

טענת יציאה: יוצרת את טבלאות המידע הדרושות: Blacklist, Reasons, Links_Reasons, Ratings, History.



שם הפעולה: initialize_reasons

טענת כניסה: אין

טענת יציאה: מוודאת כי טבלת הסיבות במסד הנתונים מכילה את הסיבות התקניות בלבד.

שם הפעולה: add_to_blacklist

טענת כניסה:

1. url - הקישור להוספה לרשימה השחורה
2. Reasons - רשימת הסיבות לסימון הקישור כחשוד

טענת יציאה: מוסיפה את הקישור לרשימה השחורה ומעדכנת את הסיבות המתאימות במסד הנתונים.

שם הפעולה: is_url_blacklisted

טענת כניסה:

1. Url - הקישור לבדיקה

טענת יציאה: מחזירה True אם הקישור ברשימה השחורה, אחרת- False

שם הפעולה: get_all_blacklisted_links

טענת כניסה: אין

טענת יציאה: מחזירה רשימה של כל הקישורים ברשימה השחורה.

שם הפעולה: get_link_reasons

טענת כניסה:

1. Link – הקישור לבדיקה

טענת יציאה: מחזירה את הסיבות שסומנו עבור הקישור הנתון, אם קיימות.



שם הפעולה: insert_rating

טענת כניסה:

1. Name- שם המדרג (אופציונלי)
 2. Rating- ציון חווית השימוש מ-1 עד 5
 3. Feedback- המשוב שניתן על ידי אותו המשתמש
- טענת יציאה: הפעולה מוסיפה את הדירוג למסד הנתונים.

שם הפעולה: remove_from_blacklist

טענת כניסה:

1. url- הקישור להסרה

טענת יציאה: הפעולה מסירה את הקישור ואת כל הסיבות הקשורות אליו ברשימת links_reasons .

שם הפעולה: clear_blacklist

טענת כניסה: אין

טענת יציאה: מוחקת את כל הקישורים מהרשימה השחורה ואת קשריהם.

שם הפעולה: clear_ratings

טענת כניסה: אין

טענת יציאה: מוחקת את כל הדירוגים ממסד הנתונים.

שם הפעולה: get_all_ratings

טענת כניסה: אין

טענת יציאה: מחזירה את כל הדירוגים שנשמרו במערכת – שם, ציון, פידבק.

שם הפעולה: get_average_rating

טענת כניסה: אין

טענת יציאה: מחזירה את ממוצע הדירוגים במערכת.



שם הפעולה: `get_last_10_history`

טענת כניסה: אין

טענת יציאה: מחזירה את עשר הפעולות האחרונות שבוצעו במערכת (מטבלת `history`) הכוללות כתובת IP, פורט, קישור שנבדק, האם ברשימה השחורה ומתי נעשה הבדיקה.

שם הפעולה: `log_to_history`

טענת כניסה:

1. `ip` - כתובת הIP של המשתמש
2. `Port` - פורט המשתמש
3. `Link` - הקישור שנבדק
4. `is_suspicious` - האם חשוד (yes/no)

טענת יציאה: מוסיפה רשומה חדשה להיסטוריה עם פרטי פעולה של משתמש: כתובת IP, פורט, קישור, האם היה חשוד ותאריך הבדיקה (בדיקת הזמן מתבצעת עצמאית בפעולה).

שם הפעולה: `close`

טענת כניסה: אין

טענת יציאה: סוגרת את החיבור למסד הנתונים ומונעת איבוד נתונים.

שם המחלקה: RSA

תפקיד המחלקה: לספק את מנגנון ההצפנה והפענוח של המערכת באמצעות אלגוריתם RSA. המחלקה מאפשרת יצירה של זוג מפתחות ציבורי ופרטי, הצפנת הודעות באמצעות המפתח הציבורי, ופענוח הודעות באמצעות המפתח הפרטי. זוהי הצפנה א-סימטרית, כלומר נדרש מפתח אחד להצפנה, ומפתח שני לפענוח.

תכונות המחלקה:

שם התכונה: `self.key_size`

תפקיד: הגדרת גודל המפתח (ברירת מחדל 16 ביט)



שם התכונה: self.p, self.q

תפקיד: המספרים הראשוניים הנבחרים לבניית המפתח.

שם התכונה: self.n

תפקיד: מכפלת המספרים הראשוניים, משמש כמודולוס במפתח.

שם התכונה: self.phi_n

תפקיד: פונקציית אוילר של n, נדרשת לחישוב המפתחות.

שם התכונה: self.e

תפקיד: המפתח הציבורי

שם התכונה: self.d

תפקיד: המפתח הפרטי

פעולות המחלקה:

שם הפעולה: __init__

טענת כניסה:

1. Key_size – גודל המפתח (16 ביט כברירת מחדל)

טענת יציאה: פעולה בונה, מאתחלת את תכונות המחלקה ומגדירה את גודל המפתח ליצירה עתידית.

שם הפעולה: is_prime

טענת כניסה:

1. Num – מספר לבדיקה האם הוא ראשוני

טענת יציאה: מחזירה True אם המספר ראשוני, אחרת – False.

שם הפעולה: generate_prime



טענת כניסה: אין

טענת יציאה: מחזירה מספר ראשוני בגודל שנקבע על ידי `key_size`.

שם הפעולה: `gcd` (קיצור של `greatest common divisor`)

טענת כניסה:

1. `a` - מספר ראשון

2. `b` - מספר שני

טענת יציאה: הפעולה את המחלק המשותף הגדול ביותר (GCD) של שני המספרים.

שם הפעולה: `modinv`

טענת כניסה:

1. `e` - מספר

2. `phi` - מודולוס

טענת יציאה: מחזירה את ההופכי המודולרי של `e` ביחס ל `phi`, או `None` אם לא קיים.

שם הפעולה: `extended_gcd`

טענת כניסה:

1. `a` - מספר ראשון

2. `b` - מספר שני

טענת יציאה: מחזירה שלשה (gcd, x, y) המשמשת לחישוב ההופכי המודולרי.

שם הפעולה: `generate_keys`

טענת כניסה: אין

טענת יציאה: הפעולה מייצרת את זוג המפתחות (ציבורי ופרטי) על ידי יצירת שני מספרים ראשוניים שונים, חישוב $n = p \cdot q$ של `n`. בחירת `e` מתאים וחישוב `d`, ההופכי המודולרי של `e`.



שם הפעולה: encrypt

טענת כניסה:

1. Plaintext - מחרוזת להצפנה
2. e - המפתח הציבורי להצפנה
3. n - המודולוס להצפנה

טענת יציאה: מחזירה רשימת ערכים מוצפנים המייצגים את המחרוזת.

שם הפעולה: decrypt

טענת כניסה:

1. Ciphertext - רשימת ערכים מוצפנים

טענת יציאה: מחזירה את המחרוזת המקורית לאחר פענוח באמצעות המפתח הפרטי.

שם הפעולה: get_public_key

טענת כניסה: אין

טענת יציאה: מחזירה את המפתח הציבורי בפורמט (e, n).

שם הפעולה: str

טענת כניסה: אין

טענת יציאה: מחזירה מחרוזת המציגה את פרטי המפתחות שנוצרו – ציבורי, פרטי והמספרים הראשוניים.

שם המחלקה: communication

תפקיד המחלקה: לספק מערכת בדיקות לקישורים, כדי לזהות אם הם תקינים, או שמה עלולים להיות מסוכנים, כלומר חשודים. המחלקה מבצעת סדרת בדיקות על כתובת ה URL הנתונה ומחזירה סיבות אפשריות לחדש אם קיימות. המחלקה נועדה להחליף את השימוש בספרייה validators, על ידי מימוש מקומי ומותאם אישית לצרכי המערכת. בעצם, המחלקה היא גרסה משופרת של הספרייה validators.



תכונות המחלקה:

שם התכונה: `self.suspicious_keyword`

תפקידה: רשימה של מילות מפתח אשר אופייניות לאתרים חשודים או ניסיונות פשינג, כגון login, reset, verify, account ועוד.

פעולות המחלקה:

שם הפעולה: `__init__`

טענת כניסה: אין

טענת יציאה: פעולה בונה שמגדירה את רשימת מילות המפתח החשודות שישמשו לאיתור אתרים חשודים.

שם הפעולה: `validate`

טענת כניסה:

1. `url` – כתובת ה URL לבדיקה.

טענת יציאה: מחזירה True אם כתובת ה URL בנויה כראוי לפי תבנית תחבירית בסיסית של קישור, אחרת False.

שם הפעולה: `is_https`

טענת כניסה:

1. `url` – כתובת ה URL לבדיקה.

טענת יציאה: הפעולה מחזירה True אם כתובת ה URL מתחילה ב-"https", סימן לחיבור מאובטח. אחרת False.

שם הפעולה: `is_valid_domain`

טענת כניסה:

1. `url` – כתובת ה URL לבדיקה.

טענת יציאה: בודקת אם שם הדומיין (שם דומיין הוא השם שכותבים בשורת הכתובת כדי להיכנס לאתר. זה בעצם הכתובת של האתר באינטרנט, למשל google.com) של כתובת ה URL מכיל רק תווים חוקיים (אותיות, מספרים, מקפים ונקודות) ומחזירה True או False.



שם הפעולה: `has_ssl`

טענת כניסה:

1. כתובת ה-URL לבדיקה.

טענת יציאה: הפעולה מבצעת ניסיון להתחבר לכתובת ה-URL הנתונה, ובדקת האם יש לה תעודת SSL תקינה (כלומר האם קיימת תעודה בכלל, והאם בתוקף) מחזירה `True` אם החיבור הצליח עם אימות תעודת ה-SSL, אחרת מחזירה `False`. *תעודת SSL, קיצור של Secure Sockets Layer, היא סוג של פרוטוקול אבטחה שמבטיח שהמידע שמועבר בין המשתמש לאתר יהיה מוצפן ומוגן. זוהי בעצם תעודה דיגיטלית.

שם הפעולה: `is_suspicious_url`

טענת כניסה:

1. כתובת ה-URL לבדיקה.

טענת יציאה: בודקת אם כתובת ה-URL מכילה מילות מפתח המאפיינות אתרי פשינג', כלומר אם מופיע ברשימת המילים החשודות `suspicious_keywords` ומחזירה משתנה בוליאני בהתאם – `True` או `False`.

שם הפעולה: `check_url`

טענת כניסה:

1. כתובת ה-URL לבדיקה.

טענת יציאה: מריצה את כל הבדיקות (בדיקת התחביר של הקישור, HTTPS, דומיין, תעודת SSL והאם מכיל מילות מפתח חשודות) ומחזירה רשימת סיבות לחשד במידה ונמצאו בעיות.



שם המחלקה: Rating

תפקיד המחלקה: לנהל את מסך דירוג המשתמש במערכת. המחלקה מוסיפה (בעזרת מחלקת GUI) את כפתור "Rate me!", וברגע שנלחץ פותחת חלון שבו המשתמש מזין שם (אופציונלי), דירוג חווית השימוש בין 1 ל-5, ומשוב טקסטואלי. לאחר שליחת הדירוג מוצגת למשתמש הודעה על כך ותודה. השמירה למסד הנתונים מתבצעת בשרת.

תכונות המחלקה:

שם התכונה: self.parent

תפקיד: חלון הממשק הגרפי שמכיל את כפתור הדירוג.

פעולות המחלקה:

שם הפעולה: __init__

טענת כניסה:

1. Parent- חלון הממשק הגרפי.

טענת יציאה: פעולה בונה, שומרת את ההתייחסות לממשק לשימוש בפעולות המחלקה.

שם הפעולה: setup_rating_button

טענת כניסה: אין

טענת יציאה: מציגה בממשק למשתמש את כפתור "Rate Me!" המאפשר למשתמש לפתוח את מסך הדירוג.

שם הפעולה: open_rating_form

טענת כניסה: אין

טענת יציאה: הפעולה פותחת את חלון הדירוג, בו המשפּמֶש יכול להזים את שמו, את דירוג חוויתו מ 1 עד 5, ואת המשוב הטקסטואלי שלו. בנוסף, מוצג גם כפתור לשליחת הדירוג.



שם הפעולה: submit_rating

טענת כניסה:

1. Popup - חלון הדירוג
2. Name - שם המדרג (אופציונלי)
3. Rating - דירוג המשתמש מ 1 עד 5.
4. Feedback - משוב המשתמש.

טענת יציאה: הפעולה בודקת אם השדות מלאים (מלבד שדה השם) מציגה שגיאה אם יש צורך, ומציגה הודעת תודה.

שם המחלקה: HashManager

תפקיד המחלקה: לנהל את בדיקת סיסמת המנהל או המפתח הפרטי על ידי השוואת ערך הצפנת הHASH של ערך הסיסמה שהוזנה אל ערך HASH של סיסמה שנשמר מראש בתור משתנה סביבה (environment variable). המחלקה משתמש באלגוריתם בשם SHA-256 על מנת לחשב את הHASH של הסיסמה שהוזנה.

תכונות המחלקה:

שם התכונה: self.env_var

תפקיד: שם משתנה הסביבה שבו מאוחסן הHASH של סיסמת המנהל.

שם התכונה: self.stored_hash

תפקיד: ערך הHASH שנלקח ממשתנה הסביבה בעת אתחול המחלקה.

פעולות המחלקה:

שם הפעולה: __init__

טענת כניסה:

1. env_var – שם משתנה הסביבה שממנו ילקח ערך הHASH, מאותחל ל- "ADMIN HASH"

טענת יציאה: פעולה בונה אשר טוענת את ערך הHASH ממשתנה הסביבה ומאחסנת אותו. אם המשתנה לא מוגדר תופיע חריגה.



שם הפעולה: hash_password

טענת כניסה:

1. Password- הסיסמה שמזין המשתמש

טענת יציאה: מחזירה את ערך הHASH של הסיסמה לפי אלגוריתם SHA-256.

שם הפעולה: verify_password

טענת כניסה:

1. Password- סיסמה לבדיקה

טענת יציאה: הפעולה מחזירה True אם ערך הHASH של הסיסמה תואם לHASH השמור, אחרת מחזירה False. בנוסף, כאשר האדמין צריך לבצע אימות של מפתח פרטי, המערכת משתמשת גם במחלקה – בודקת אם הערך הHASH למפתח שהזין האדמין, זהה לערך הסיסמה המוגדר בתור משתנה הסביבה בשם "ADMIN_PRIVATE_KEY"

שם המחלקה: GUI

תפקיד המחלקה: לנהל את הממשק הגרפי של המערכת, כולל טופס להזנת קישור לבדיקה, הצגת תוצאות, טופס דירוג חווית השימוש, הצגת הסברים על סיבות לחשד ומסך פתיחה.

תכונות המחלקה:

שם התכונה: self.root

תפקיד: חלון ה-Tkinter הראשי של הממשק.

שם התכונה: self.client

תפקיד: אובייקט הלקוח המבצע את הבדיקות מול השרת.

שם התכונה: self.logo_image

תפקיד: תמונת הלוגו של המערכת.

שם התכונה: self.form_frame

תפקיד: מסגרת להזנת הקישור לבדיקה.



שם התכונה: self.result_frame

תפקיד: מסגרת להצגת תוצאות הבדיקה.

שם התכונה: self.entry_name

תפקיד: שדה להזנת הקישור לבדיקה.

פעולות המחלקה:

שם הפעולה: __init__

טענת כניסה:

1. Root - חלון tkinter הראשי.

2. Client – אובייקט הלקוח המטפל בתקשורת עם השרת.

טענת יציאה: פעולה בונה שמאתחלת את חלון הממשק ומפעילה את הגדרת רכיבי הממשק.

שם הפעולה: setup_gui

טענת כניסה: אין

טענת יציאה: בונה את רכיבי הממשק – לוגו phishido, טופס קלט לקישור, כפתור בדיקת קישור, כפתור פתיחת מסך דירוג וכפתור מיד על המערכת.

שם הפעולה: load_logo_image

טענת כניסה:

1. Path - נתיב לתמונת הלוגו.

2. Size - גודל התמונה, ברירת המחדל היא 80X80

טענת יציאה: טוענת את תמונת הלוגו ומחזירה אובייקט PhotoImage מותאם לממשק.

שם הפעולה: show_splash_screen_and_wait

טענת כניסה: אין

טענת יציאה: מציגה מסך פתיחה בו מוצגים הודעת "WELCOME BACK", לוגו phishido וכפתור המשך.



שם הפעולה: check_url_safety

טענת כניסה: אין

טענת יציאה: שולחת את הקישור שהוזן לבדיקה לשרת ומציגה את תוצאת הבדיקה בממשק.

שם הפעולה: show_reasons_for_link

טענת כניסה:

1. Link - כתובת ה URL לבדיקה.

טענת יציאה: מציגה את הסיבות לחשד עבור הקישור, אם קיימות מהמסד או מהשרת.

שם הפעולה: show_reasons

טענת כניסה:

1. Reasons – רשימת סיבות לחשד

טענת יציאה: מציגה את הסיבות בממשק (רשימת סיבות שכבר התקבלה)

שם הפעולה: display_results

טענת כניסה:

1. url - כתובת ה URL שנבדקה

2. Response - תוצאת הבדיקה מהשרת.

טענת יציאה: מציגה את תוצאת הבדיקה, דירוג הבטיחות וסיבות החשד.

שם הפעולה: open_rating_form

טענת כניסה: אין

טענת יציאה: פותחת טופס קופץ להזנת שם, דירוג כוכבים (1 עד 5) ומשוב טקסטואלי.

שם הפעולה: submit_rating

טענת כניסה:

1. Popup - חלון הדירוג

2. Name - שם המדרג

3. Rating - מספר הכוכבים

4. Feedback - משוב טקסטואלי

טענת יציאה: בודקת את תקינות הדירוג והמשוב ושולחת את הדירוג לשרת.



שם הפעולה : `set_rating`

טענת כניסה:

1. `Rating` - מספר הכוכבים שנבחר (בין 1 ל-5)

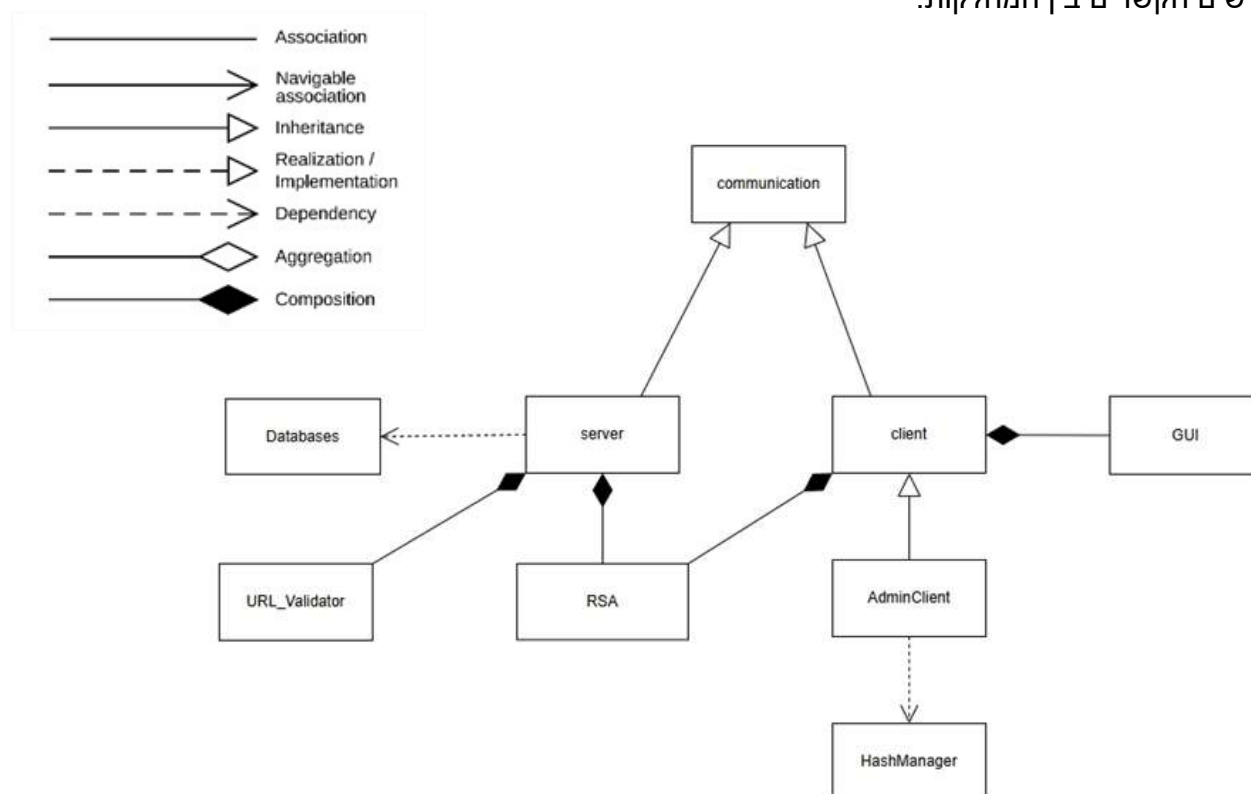
טענת יציאה: מעדכנת את תצוגת הכוכבים בהתאם לבחירה.

שם הפעולה: `show_info_window`

טענת כניסה: אין

טענת יציאה: מציגה חלון עם הסברים על כל אחת מהסיבות האפשריות לסימון קישור כחשוד.

תרשים הקשרים בין המחלקות:



התרשים מציג את הקשרים בין הרכיבים השונים במערכת בצורה ברורה. תחילה, קיימים קשרי ירשה (Inheritance) בין מחלקות: AdminClient ירש מ-client, ואילו המחלקות server, client, יורשות ממחלקת communication. המשמעות היא שכל מחלקה יורשת תכונות ופונקציות מהמחלקה שמעליה, ומרחיבה את ההתנהגות שלה.

בנוסף, קיימים קשרי קומפוזיציה (composition). בתוך מחלקת client, מתקיים קשר קומפוזיציה עם מחלקת GUI, כלומר ה-GUI נוצר כחלק מהלקוח ואינו יכול להתקיים בנפרד. בנוסף, גם מחלקת RSA נמצאת בקשר של קומפוזיציה עם client, אך גם עם server. כלומר היא נבנית כחלק מהמחלקות ומשמשת לכל אורכן. בנוסף, server כולל בקומפוזיציה גם את מחלקת URL_Validator שמשמשת לבדיקת תקינות ואבטחת קישורים.

לבסוף, קיימים גם קשרי תלות (Dependency), שהם קשרים חלשים יותר: server יוצר ומשתמש במחלקת Databases באופן זמני לצורך גישה למסד הנתונים, מבלי לשמור אותה כאובייקט קבוע. גם AdminClient תלוי במחלקת HashManager, הרי היא משמשת לאימות סיסמה ומפתח, אך אינה נשמרת כחלק מן האובייקט. קשרים אלה מצביעים על שימוש נקודתי במחלקות אחרות, לצורך פעולה מסוימת בלבד.

חלק ב' - הבעיה האלגוריתמית

הערכת רמת הסיכון של כתובת אינטרנטית מסויימת לצורך הגנה על המשתמש מפני אתרי פשינג וקישורים אשר נועדו למטרות זדוניות.

הסבר על היכולת: האלגוריתם הזה הוא הלב הפועם של כל המערכת. למעשה, כל הרעיון של Anti PhishIdo, עומד על יכולת הניתוח והזיהוי של קישורים חשודים. האלגוריתם הוא המנוע שיוצר את כל הערך למשתמש – בלי המנגנון הזה, המערכת הייתה בסך הכול מעטפת ריקה – ממשק שלא יודע להבדיל בין אתר בטוח למסוכן. כל ההחלטות של המערכת – האם להציג התרעה, האם להוסיף לרשימה השחורה, האם להמליץ להיכנס לאתר – כולן מתבססות על התוצאה של האלגוריתם הזה. זאת לא סתם בדיקה אחת שטחית, אלא סדרת בדיקות שנועדו לכסות מגוון רחב של שיטות פשינג. האלגוריתם נותן הסבר למשתמש מדוע האתר חשוד, ולא רק מודיעה לו אם מסוכן או לא ללא הסבר.

פירוט היכולת ומשמעות כל בדיקה:

1. בדיקת פורמט הכתובת: המערכת בודקת האם הכתובת כתובה בצורה חוקית, לפי תבנית של קישור תקני.
 2. בדיקת https: המערכת בודקת האם הכתובת מתחילה ב/https://, דבר המצביע על חיבור מוצפן.
 3. בדיקת שם הדומיין: האלגוריתם בודק האם הדומיין (דומיין הוא השם שכותבים בשורת הכתובת כדי להיכנס לאתר. זה בעצם הכתובת של האתר באינטרנט, למשל google.com) בנוי מאותיות מספרים ונקודות בלבד.
 4. בדיקת SSL: המערכת מנסה להתחבר לכתובת ולבדוק אם יש לה תעודת SSL תקינה, כלומר קיימת ובתוקף (תעודת SSL, קיצור של Secure Sockets Layer, היא סוג של פרוטוקול אבטחה שמבטיח שהמידע שמועבר בין המשתמש לאתר יהיה מוצפן ומוגן. זוהי בעצם תעודה דיגיטלית.)
 5. איתור מילות מפתח חשודות: מחפשת מילות מפתח נפוצות בפשינג, כגון login,password וכו'.
- האלגוריתם בנוי לגילוי מגוון רחב של בעיות אבטחה, החל מתבנית שגויה, דרך חוסר הצפנה ועד לניסיון התחזות מתוחכם יותר, ובכך מאפשר הגנה מקיפה מפני קישורים מסוכנים.

קטע הקוד המרכזי (מחלקת URL Validators):

```

58 def check_url(self, url):
59
60     reasons = [] # רשימת הסיבות החשודות
61
62     if not self.validate(url):
63         reasons.append("ה-URL לא תקין.")
64     if not self.is_https(url):
65         reasons.append("הקישור לא מאובטח (לא משתמש ב-HTTPS).")
66     if not self.is_valid_domain(url):
67         reasons.append("שם הדומיין לא תקין או מכיל תווים חשודים.")
68     if not self.has_ssl(url):
69         reasons.append("האתר לא משתמש בהצפנת SSL (הקישור לא מוצפן).")
70     if self.is_suspicious_url(url):
71         reasons.append("הקישור מכיל מילות מפתח חשודות.")
72
73     return reasons
74

```



קוד שאהבתי:

1. החלפת מפתחות

הסבר על היכולת מהי החלפת מפתחות ולמה היא קריטית: החלפת המפתחות היא חלק קריטית בהגנה על המידע ודואגת לכך שמידע אישי של המשתמש לא יעבור בתקשורת שבין השרת ללקוח כטקסט גלוי שעלול להיחשף. זהו הלב השני של המערכת: תהליך החלפת המפתחות הוא הלב הביטחוני של המערכת, בלעדיו כל ההודעות היו עוברות כטקסט קריא, וכל אחד היה יכול ליירט אותן לראות את המידע שמוכל בהן, כלומר, האמינות והביטחון של המערכת היו נפגעים קשות. המערכת אינה הייתה שווה כלום בלי ההגנה הזאת, המשתמשים אינם היו בטוחים.

פירוט היכולת: ראשית מהי הצפנת RSA? הצפנת RSA היא הצפנה א-סימטרית, המתממשת בעזרת מפתחות:

1. מפתח ציבורי – אפשר לשתף עם כולם, הוא מיועד להצפנה

2. מפתח פרטי – נשמר בסוד, הוא מיועד לפענוח

ברגע שמישהו מצפין הודעה עם המפתח הציבורי שלך, רק אתה יכול לפענח אותו עם המפתח הפרטי שלך.

במערכת, הלקוח יוצר זוג מפתחות: פרטי וציבורי. לאחר מכן, הלקוח לשרת את המפתח הציבורי שלו והשרת שולח לו בחזרה את המפתח הפרטי של עצמו. מכאן ואילך, כל צד מצפין את ההודעות במפתח הציבורי של הצד השני – כך שרק הצד השני יוכל לפענח את ההודעות בעזרת המפתח הפרטי שלו. כמו כן, בכל תחילת חיבור המפתחות משתנים על מנת להבטיח בטיחות מירבית.

דבר זה גורף הגנה על המידע, אשר מוצפן מקצה לקצה וכמו כן גם מניעת MITM, כלומר מניעת יירוט של ההודעה – גם אם מישהו יאזין לחיבור ויצליח ליירט את ההודעה העוברת, הוא יראה מספר תווים מוצפנים ולא את המידע האמיתי. עם תווים אלו לא יוכל לעשות כלום כמובן, כיוון שאין לו את המפתח הפרטי.

```
def connect_to_server(self):
```

```
    try:
```

```
        self.connect() # יצירת חיבור TCP לשרת
        # שליחת המפתח הציבורי של הלקוח לשרת:
        self.send_message(str(self.encryption.get_public_key()))
        # קבלת המפתח הציבורי של השרת:
        server_public_key_str = self.receive_message()
        server_e, server_n = eval(server_public_key_str)
        self.server_e, self.server_n = server_e, server_n
```

```
    except Exception as e:
```

```
        print(f"Failed to connect to server: {str(e)}")
```

קוד בצד הלקוח:

קוד בצד השרת:

```
def handle_client(self, client_socket, address):
    try:
        database = Databases(self.db_name)
        client_ip, client_port = address

        client_public_key = eval(client_socket.recv(1024).decode())
        client_e, client_n = client_public_key
        client_socket.send(str((self.encryption.e, self.encryption.n)).encode())
```

2. שימוש במשתנה סביבה לאימות

הסבר על היכולת: אחד מהפתרונות היצירתיים והאבטחתיים שביצעתי במערכת היה השימוש במשתני סביבה לשם הגנה על סיסמת גישת האדמין. עמדתי בפני דילמה: רציתי לאפשר גישה לפאנל ניהול רק למשתמשים מורשים, אך לא רציתי שהסיסמה תופיע בקוד עצמו, מחשש לחשיפתה למי שיצפה בקוד המקור. כדי לפתור את הבעיה, החלטתי שלא לאחסן את הסיסמה ישירות – אלא לאחסן את ערך הצפנת הhash שלה בתוך משתנה סביבה. בשלב הראשון יצרתי hash מוצפן מהסיסמה שבחרתי בעזרת אלגוריתם SHA-256. לאחר מכן, שמרתי את הערך הזה כמשתנה סביבה במערכת ההפעלה, בשם ADMIN_HASH. בעת הריצה, כאשר המשתמש מנסה להיכנס כאדמין, המערכת מבקשת ממנו להזין את הסיסמה. ערך זה עובר גם הוא הצפנת hash ומושווה מול המשתנה שנשמר. רק אם שני הערכים תואמים – המערכת מאפשרת כניסה. גישה זו מאפשרת להסתיר מידע רגיש מהקוד, להחליף סיסמאות בקלות ללא שינוי קוד, ולשמור על רמת אבטחה גבוהה מול תוקפים. לאחר מכן בקוד, ממשתי אותו הדבר גם בעת הזנת מפתח פרטי בצד האדמין לפני הצפייה בטבלת history, המכילה כתובות IP מוצפנות.

מחלקת HashManager - קריאה למשתנה הסביבה והשוואה:

```
class HashManager:
    def __init__(self, env_var="ADMIN_HASH"):
        self.env_var = env_var
        self.stored_hash = os.getenv(env_var)
        if not self.stored_hash:
            raise ValueError(f"לא הוגדר משתנה הסביבה {env_var}")

    def hash_password(self, password):
        hashed = hashlib.sha256(password.encode()).hexdigest()
        return hashed

    def verify_password(self, password):
        result = self.hash_password(password) == self.stored_hash
        return result
```




חלק ג' - מסמך בדיקות מלא:

הבדיקות שתכננו בשלב האפיון:

1. בדיקת מבנה ה-URL: בשלב האפיון לא הכרתי לעומק את מאפייני הקישור ואת הדרכים האמיתיות לבדוק אותו, לכן על פי החקר המינימלי שביצעתי, הדרכים העיקריות מבחינתי היו ראשית, לבדוק האם הקישור בעל מבנה חשוד, כגון תווים או מילים מעוררות חשד. בנוסף, בדיקה מול רשימה של קישורים שאובחנו כחשודים. כלומר, בדיקות על בסיס מאפייניו החיצוניים של הקישור
- מה בוצע בפועל: בפועל בוצעו בדיקות הרבה יותר מעמיקות:
 - בדיקה האם הקישור תקין תחבירית.
 - חיפוש תווים חשודים כמו -, *, %, @.
 - בדיקת הופעת מילות מפתח מוכרות מעולם ההונאות כמו login, password.
 - האם בעלת תעודת SSL
 - האם בעל תחילה של https://

תוצאות הבדיקה:

- הבדיקה עבדה – בכל פעם שהוזן קישור חשוד בגלל סיבה מסויימת, אכן הוצגו התוצאות הנכונות לאותו הקישור.

בעיות שנתקלתי בהן וכיצד נפתרו:

- ראשית, רציתי להוסיף מנגנון אשר אומר אם אתר קיים או לא על מנת לחסוך בדיקות. הוספתי את המנגנון, אבל אז הבנתי שלא אוכל לבדוק את פעילות המערכת. כלומר, הבנתי שאין לי מאיפה להביא קישורים שקיימים והם חשודים, כיוון שכל המקורות הקישורים המסוכנים שמצאתי ברחבי האינטרנט, הכילו קישורים של אתרים שכבר הוכחו כחשודים ונחסמו, לכן לא קיימים. כדי לפתור את הבעיה החלטתי שגם במידה והאתר לא באמת קיים, יש להציג את הסיבות התיאורטיות, מדוע היה חשוד אם היה קיים.
- שנית, ברשימת המילים החשודות היו גם שמות של אתרים מפורסמים. חשבתי שזה רעיון חכם, אשר יוכל לזהות ניסיון התחזות לאתרים האלה, למשל - <http://www.facebookok.com> (התחזות לפייסבוק), אך במהרה גיליתי שאם אכניס לדוגמה את המילה facebook לרשימת המילים החשודות, פייסבוק עצמו גם יחשב כחשוד. לכן החלטתי לא לכלול ברשימה שמות של אתרים מפורסמים

2. בדיקות סייבר: בשלב האפיון תוכננו מספר בדיוק סייבר, הראשונה שבהן, היא מניעה של SQL Injection, מתקפה בה תוקף מזין קלט זדוני שמכיל פקודות SQL על מנת להפעיל אותן על מסד הנתונים של המערכת ולחשוף או לשנות מידע. באמצעות ניהול שאילתות מאובטח. שנית, תוכנה הגנה מפני DDoS, מתקפה שמטרתה להעמיס על המערכת בכמות אדירה של בקשות כדי לגרום לה לקרוס או להפוך ללא זמינה. אשר כללה הגבלה של מספר הבקשות שניתן לשלוח מאותה כתובת IP בזמן נתון.

מה בוצע בפועל: בפועל, נתתי מענה לקוד שלי עבור שתי הסכנות הממשיות האלה: ראשית, יישמתי מנגון הגנה מפני מתקפת SQL, הנקרא prepared statements -זוהי שיטת עבודה שבה יש הפרדה בין הפקודה לקלט, כלומר, הקלט עובר עיבוד ואינו יכול להתפרש כחלק מהפקודה עצמה. כך נמנעת האפשרות להזריק קוד עוין לתוך השאילתות שמבצעות על מסד הנתונים כדי לבדוק בדיקת קופסה שחורה, ניסיתי להזין בשדות הקלט טקסט שמכיל ניסיונות הזרקה, כגון '1'='1' OR, אשר אמור להכניס כל ערך שבא לפניו אל הטבלה, כיוון שזוהי שאילתת SQL נכונה. שנית, במערכת שלי יישמתי מנגון הגבלת בקשות לפי כתובת IP, כלומר כל כתובת IP יכולה לבצע עד 10 בקשות ב-10 שניות. אם עוברת את המגבלה הזו, המערכת חוסמת את הכתובת. כדי לבדוק זאת הקטנתי את מספר החיבורים ל-3 בעשר שניות, לצורך הבדיקה וניסיתי להתחבר עם ארבעה משתמשים.

תוצאות הבדיקה:

- בדיקת ה-SQL עבדה: השאילתות נכנסו אל הטבלה כמו שהן ונרשמו סיבות לכך שהקישור לא תקין וכו'. דבר זה אומר בעצם שהדבר לא עקף את המערכת. -בדיקת DDoS עבדה גם היא. לאחר שלוש התחברויות פעילות, ההתחברות הרביעית לא אופשרה.

בעיות שנתקלתי בהן וכיצד נפתרו:

- היה מאתגר להגדיר מהו קצב נורמלי של בקשות, כלומר – כמה בקשות באמת נחשב לגיטימי וכמה נחשב לחשוד. לאחר ניסוי וטעיה מצאתי איזון סביר של 10 בקשות ל-10 שניות, שמאפשר גם שימוש תקין למשתמש רגיל, וגם חוסם מתקפות. - לגבי SQL Injection, בתחילה לא השתמשתי בPrepared Statements, ורק לאחר שבדקתי את המערכת מול מתקפות מזויפות הבנתי את הצורך בכך, ושיפרתי את הקוד.



פירוט בדיקות נוספות שבוצעו למערכת:

1. בדיקת החלפת מפתחות

מטרת הבדיקה: לבדוק שהמערכת אכן מצפינה ומחליפה מפתחות RSA בכל התחברות לקוח, כדי למנוע אפשרות לגניבת מפתחות או ניצול מפתח ישן.

מה בוצע בפועל: המערכת מתוכננת לבצע החלפת מפתחות RSA בכל התחברות מחדש. לצורך הבדיקה התחברנו עם לקוח מספר פעמים ברצף, ובכל התחברות בדקנו את המפתחות שהוחלפו – המפתח הציבורי שנשלח מהלקוח לשרת, והמפתח הציבורי שנשלח מהשרת ללקוח.

מה היו תוצאות הבדיקה: בכל התחברות התקבל מפתח חדש ושונה, ולא נעשה שימוש במפתחות ישנים.

התקשורת בוצעה בצורה תקינה לאחר ההחלפה בכל פעם.

במידה והתגלו בעיות – כיצד נפתרו: במהלך הפיתוח הראשוני המערכת לא ביצעה החלפה מלאה של מפתחות בכל התחברות, אלא שמרה אותם במצב סטטי. כשהבחנתי בכך, שיניתי את המימוש כך שמנגנון החילוף יתממש מחדש בכל התחברות, והבעיה נפתרה.

2. בדיקת אורך קישור

מטרת הבדיקה: לבדוק שהמערכת לא מאפשרת קבלת קישורים באורך קיצוני (מאות או אלפי תווים) שעלולים לשמש לתקיפות שונות.

מה בוצע בפועל: נשלחו למערכת קישורים באורך קצר ותקני, ולאחר מכן גם קישורים באורך חריג של מעל 2000 תווים (קישור תקני לרוב לא עובר 300-200 תווים). המערכת נדרשה לזהות זאת כהתנהגות חריגה.

תוצאות הבדיקה: קישורים קצרים עברו בהצלחה, נבדקו, וחזרו תשובות לגבי החדש לגביהם, בעוד כאשר הוקלד קישור ארוך מדי, התקבלה הודעה לכך.

בעיות שנתקלתי בהן וכיצד נפתרו: תחילה, כאשר הוקלד קישור ארוך מדי, הודפסה הודעה של "response from server". כדי לפתור זאת, הגדרתי מחדש הודעה שמודפסת כאשר אורך הלינק גדול יותר מ-2000.



מדריך למשתמש

התקנת המערכת

-הסביבה הנדרשת:

1. מערכת הפעלה, Windows 10/11
 2. python 3.9 (או גרסאות חדשות יותר)
- הכלים הנדרשים: ישנן רק שתי ספריות שיש להתקין דרך pip:

1. requests- לצורך בדיקת תעודת ה-SSL של האתר
 2. pillow- לצורך הצגת תמונת הלוגו ב-GUI
- שאר הספריות כבר מוגדרות בפיתון, לכן אין להתקין אותן בצורה ידנית.

-מיקומי הקבצים:

כל הקבצים ממוקמים באותה התיקייה.

-נתונים התחלתיים: יש להתקין את משתני הסביבה מראש:

1. ADMIN_HASH- ערך ה-HASH של סיסמת האדמין
2. ADMIN_PRIVATE_KEY- המפתח הפרטי של האדמין שמיועד לאפשר צפייה בהיסטוריה

-רשת:

1. פורט ברירת המחדל הוא 8002
 2. תקשורת בין שרת ללקוח מבוססת TCP
 3. אין צורך לחיבור אמיתי לאינטרנט, למעט בדיקת SSL.
- ארכיטקטורה מינימלית: מחשב לקוח, אשר מריץ את קובץ הלקוח או האדמין, מחשב שרת, אשר מריץ את השרת, קישור התקשורת, כלומר שהפורט הנוכחי יהיה פנוי, מסד נתונים, התקנה python 3.9 או גרסאות חדשות יותר, תמיכה בספריות צד שלישי, כלומר ספריות שלא מותקנות אוטומטית בפיתון - requests, pillow, מערכת הפעלה – windows.

משתמשי המערכת:

1. משתמש קצה:

- יכול לבדוק קישורים ולקבל תשובה אם הם חשודים או בטוחים.
- על הודעת הבדיקה מתקבלות סיבות חשד מפורטות.
- המשתמש יכול לשלוח פידבק, כלומר דירוג חווית שימוש.

2. מנהל מערכת:

- ראשית, למנהל קיימות כל אפשרויות משתמש הקצה + פיצ'רים שרק הוא יכול לממש.
- המנהל נדרש להזין סיסמת גישה
- המנהל יכול לרענן את הרשימה השחורה, כלומר להפעיל פעולה שבדקת האם יש קישורים בטוחים ברשימה השחורה, במקרה וכן תוציא אותה ותבטל את קשריה.
- המנהל יכול לצפות ברשימת הדירוגים
- המנהל יכול לצפות בממוצע הדירוגים
- המנהל יכול למחוק את מסדי הדירוגים והרשימה השחורה
- המנהל יכול לצפות בטבלה ההיסטוריה המפוענחת, בכפוף להזנת מפתח פרטי.

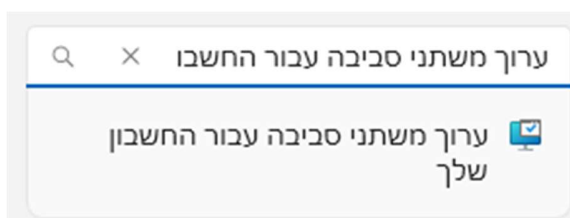
אופן הפעלת המערכת:

1. אם אתה המנהל, הגדר את משתני הסביבה (נמצאים בנספחים). רק לאחר מכן פתח את הקוד בסביבת ההרצה pycharm.

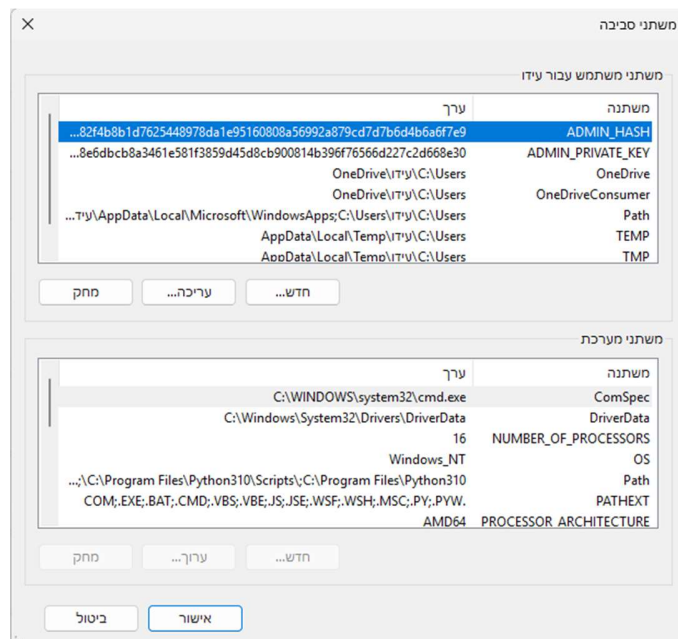
על מנת להגדירם:



היכנס להגדרות המחשב.

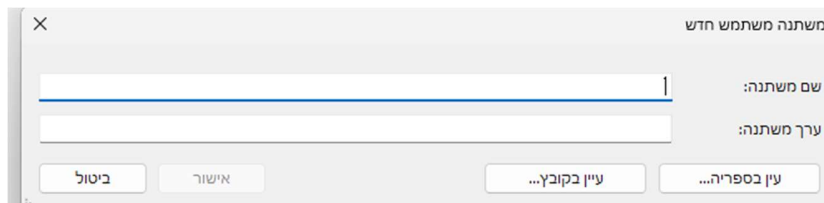


חפש בשורת החיפוש "ערוך משתני סביבה עבור החשבון שלך"



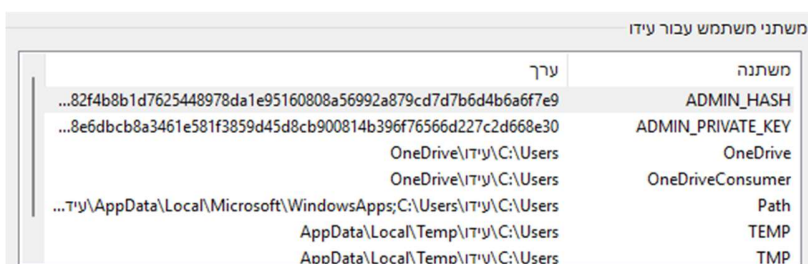
יפתח המסך הבא:

לחץ על הכפתור "חדש..". (העליון)



יפתח בפניך המסך הבא, אנא הגדר את משתני הסביבה הרצויים

כשסיימת, לחץ על "אישור"



ניתן לראות את משתני הסביבה המוגדרים



לחץ "אישור" על כל החלונות.

2. התקנת ספריות הצד השלישי – requests, pillow
על מנת לעשות זאת:

```

Microsoft Windows [Version 10.0.26100.4061]
(c) Microsoft Corporation. All rights reserved.
C:\Users\יודיע>
  
```

נפתח במחשב את
הCMD

```

Microsoft Windows [Version 10.0.26100.4061]
(c) Microsoft Corporation. All rights reserved.
C:\Users\יודיע>pip install request
  
```

נקליד: "pip install requests", ולאחר מכן
pillow"

(ביניהן נלחץ על הכפתור enter)

```

Microsoft Windows [Version 10.0.26100.4061]
(c) Microsoft Corporation. All rights reserved.
C:\Users\יודיע>pip install pillow
  
```

3. הרץ את הקובץ server.py, כלומר השרת.



4. הרץ את client.py, הלקוח, או AdminClient.py האדמין בצד המשתמש או המנהל.

5. וודא שהקו והפורט זהים בקוד הלקוח והשרת.

סיכום אישי

בתחילת העבודה על הפרוייקט, לא ידעתי לאן אני מכוון. רציתי מצד אחד לבחור נושא עם זיקה לסייבר, אך מהצד השני, שהוא יהיה נושא שאני מתעניין בו באופן אישי, כזה שכולל מחקר עצמאי ויאפשר לי להעמיק את הבנתי בתחום, תוך כדי הנאה ורצון אמיתי לדעת, ולא מחובת העבודה. לאחר חיפוש רב, והתייעצות עם מורי, הגעתי לנושא הנוכחי – זיהוי פישנינג, על מנת לאפשר את בטיחותם של המשתמשים באינטרנט באופן גורף. לא פעם ולא פעמיים, כל אחד מאיתנו גלש באינטרנט והגעתי לאתרי פישנינג, בעלי כוונות זדוניות. במהלכו של הפרוייקט נדרשתי להתמודד עם מספר שלבים מורכבים, החל מהבנה עקרונית של בעיית הפישנינג, בניית מערכת יעילה לזיהוי אתרים חשודים, דרך בניית תקשורת בין שרת ללקוח, ועד פיתוח ממשק גרפי ידידותי ונוח למשתמש. אחד מן הקשיים העיקריים שלי היו יצירת ההצפנה, כלומר, יצירת מערכת אשר עובדת באופן עקבי ובטוח אשר תבטיח את בטיחות המשתמש במערכת, הכוללת החלפת מפתחות ושמירה על דיוק מירבי.

במהלך הדרך, למדתי רבות בעצמי, בין היתר, למדתי כיצד פועל אלגוריתם RSA ואיך ניתן לממש אותו באופן ידני ומתמטי, כלומר, מבלי ללכת ב"דרך הקלה", שהיא שימוש בספריות מוכנות. למדתי מהי הצפנת hash, מדוע יש בה צורך, ואיך להשתמש בה, בשילוב אם למידה של מהו משתנה סביבה, ואיך כדאי לשלב ביניהם על מנת ליצור בטיחות מירבית. למדתי את מאפיינים כתובת ה-URL, ומכאן והאלה, בזכות הפרוייקט והחקר המעמיק, אדע לזהות קישור חשוד עוד לפני כניסתי אליו, באמצעות מאפייניו החיצוניים. כל הכלים הללו, אשר למדתי בעת העבודה על הפרוייקט, ילקחו איתי להמשך.

במהלך הדרך נעזרתי במומחים בתחום – מורי, בעלי שנים של ניסיון וידע נצבר בתחום, אשר הצליחו לתת לי מענה יעיל וטוב אל כל בעיה אשר נתקלתי בה. נעזרתי גם בחבריי לכיתה, אשר עזרו לי להבין עקרונות תיאורטיים ולהביע את דעתם על הממשק שלי.

בראייה לאחור, הייתי מוסיף פיצ'רים נוספים לפרוייקט, למשל, AI אשר חוזה קישורים חשודים אשר עלולים להיווצר. בנוסף, הייתי מתחיל מלכתחילה שימוש ב-costum tkinter. התחלתי לכתוב את הפרוייקט tkinter רגיל, כיוון שזהו מה שלמדתי וידעתי. חשבתי שאוכל ראשית להגיע לתוצר סיום, ורק אז לדאוג לממש. התברר לי שמאוד קשה להחליף את הממשק הקיים הכתוב בtkinter רגיל אל custom tkinter.

במידה והיו ברשותי משאבים נוספים, הייתי מוסיף מערכת AI אשר יודעת ליצור רשימה של חזיונים של קישורים מסוכנים אשר עתידים להיווצר, או מערכת שלומדת לנתח מבנה קישור. בנוסף, הייתי מעמיק את בדיקות הקישורים, לדוגמה בדיקה האם האתר מכיל פרסומות, או כמה זמן האתר קיים.

שאלות חקר עצמי: אילו עוד פעולות המשתמש היה רוצה שיהיו זמינות עבורו? איך ניתן לשפר את הממשק? או את חווית המשתמש?

תודות: תודה לכל מי שליווה אותי בתהליך, שעזר, תמך ונתן דעה. תודה למורים שהנחו ולימדו אותי לאורך כל השנה, וידעו תמיד להיות שם בשבילי בכל זמן: עידן פלדברג, רום רפסון, עודי רז ויעקב שוצמן.



ביבליוגרפיה

"What Is An SSL/TLS Certificate?", aws -amazon.

<https://aws.amazon.com/what-is/ssl-certificate/>

" HTTPS", Wikipedia

<https://en.wikipedia.org/wiki/HTTPS>

"הרקע המתמטי של RSA, או: איך הצפנת RSA עובדת?", בעז צבר, אוקטובר 2011

<https://www.digitalwhisper.co.il/files/Zines/0x19/DW25-4-MathBaseRSA.pdf>

"רשתות מחשבים", עומר רוזנבוים, ברק גונן ושלומי הוד, נובמבר 2020

<https://data.cyber.org.il/networks/networks.pdf>

כמובן שמספר המקורות בהם השתמשתי הוא בלתי ספיר, אך אלו העיקריים בהם נעשה שימוש, איך להשתמש בהצפנת RSA בצורה המתמטית, מהו HTTP/HTTPS, ומהי תעודת SSL, ביחד עם ספר "רשתות מחשבים" שנתן מענה לשאלות ותהיות רבות מכל התחומים השונים.

נספחים

1. הסבר מעמיק על סיבות החשד כלפי קישור:

- ה-URL לא תקין: הקישור אינו עומד בפורמט בסיסי של כתובת אינטרנט – למשל חסר בו דומיין תקף, סימנים מיוחדים במקומות לא חוקיים, או שהקידומת לא תקינה.
- הקישור לא מאובטח (לא משתמש ב-HTTPS): https, כלומר, Hyper Text Transfer Protocol (Secure) הוא פרוטוקול תקשורת שמעביר מידע בין דפדפן לשרת. חלק מהאתרים באינטרנט יתחילו בקידומת http://, וחלק ב-https://. מה סיומת ה-s נותנת בפועל? ראשית הצפנת מידע - כל מה שנשלח – סיסמאות, טפסים, קבצים – עובר מוצפן. גם אם מישו מאזין למידע שעובר, הוא לא יוכל להבין אותו. שמירה על שלמות המידע - מבטיח שהמידע לא שונה בדרך –אם מישו מנסה לשנות את התוכן בין השולח למקבל, הדפדפן יזהה זאת. בניגוד זאת, באתרים ללא סיומת s, כל מידע עובר כטקסט פתוח, מה שנותן אפשרות לגנוב פרטים בקלות, האקרים יכולים להתחזות לאתר מוכר ולבקש סיסמה.
- שם הדומיין לא תקין או מכיל תווים חשודים: הדומיין, כולל תווים חריגים, כגון סימנים לא חוקיים, למשל *, &, %, @. הרבה פעמים באתרי פישנינג נעשה שימוש בתווים על מנת ליצור דמיון עם האתר המקורי שמתחזים אליו, לדוגמה <https://www.q00gle.com>
- האתר לא משתמש בהצפנת SSL (הקישור לא מוצפן): SSL, כלומר, Secure Sockets Layer, היא שכבת התקשורת המאובטחת. זהו פרוטוקול אבטחה שנועד להצפין את התקשורת בין שני צדדים – למשל בין הדפדפן שלך לשרת של אתר אינטרנט. SSL יוצר "נתיב מוצפן" בין הלקוח (הדפדפן) לבין השרת, כלומר המידע עובר מוצפן בין השרת ללקוח ולא ניתן ליירט אותו. בנוסף, כל צד מאמת את הזהות של הצד השני, ומתקיים מנגנון של שמירה על שלמות המידע – לוודא שהנתונים לא שונו. תעודת SSL, היא תעודה אשר משמשת כחותמת דיגיטלית לכך שאותו האתר בטוח, מתקיימת בו הצפנת SSL. ניתן לזהותה על ידי סמל של מנעול בשורת כתובת האתר הנתון.
- הקישור מכיל מילות מפתח חשודות: הקישור כולל מילים נפוצות בהונאות פישנינג, בין המילים האלה ניתן למצוא את login, verify, password ועוד מילים נוספות. המילים נועדו לשדר דחיפות מסויימת או אמינות מן הצד השני, לכן משתמשים במילים האלו.

ההצפנות בפרוייקט

- הצפנת hash: ההצפנה היא אלגוריתם מתמטי, שמקבל קלט, ומחזיר מחרוזת קצרה, קבועת אורך, שנראית כמו קוד מוצפן. החישוב הוא חד כיווני, לכן אי אפשר לפענח אותו ולחזור לקלט המקורי. משתמשים בהצפנה זו בעיקר לאימות סיסמאות.

- הצפנת RSA: אלגוריתם ההצפנה אסימטרי, כלומר משתמש בשני מפתחות שונים, מפתח ציבורי, אשר מיועד להצפנה, ומפתח פרטי, אשר נועד לפענוח. המשמעות היא שאפשר לשלוח מידע פרטי, וגם אם אחרים רואים הודעה לא יבינו אותה. היחיד שיכול להבין אותה, הוא בעל המפתח הפרטי שיכול לפענחה. ההצפנה מבוססת על קושי מתמטי, קשה לפרק מספר גדול לגורמיו הראשוניים.

איך עובדת ההצפנה?

ראשית, נבחר שני מספרים ראשוניים גדולים, נסמן אותם על ידי האותיות p, q . הם יהיו הבסיס לשאר החישובים שלנו.
שנית, נכפיל את המספרים. תוצאת מכפלתם תיקרא n , כלומר: $n = p * q$. זהו מספר גדול מאוד, ושמו הוא מודולוס. כל מי שמצפין הודעה ישתמש בו.
לאחר מכן, נחשב את פונקציית אוילר ($\phi(n)$ פי של n): $\phi(n) = (p-1)(q-1)$ המספר שהתקבל עוזר לקבוע מה יהיה המפתח הפרטי. הוא נשאר סודי.
נבחר את המפתח הציבורי: e . כדי למצוא אותו, נצטרך למצוא מספר מסוים, אשר גדול מ-1 וקטן יותר מ $\phi(n)$, והוא זר לו (כלומר אין להם מחלקים משותפים מלבד 1). זהו המפתח הציבורי שלנו, והוא נכתב כך: (e, n)
כעת, נחשב את המפתח הפרטי: נמצא מספר, בשם d בצורה הבאה: $(d * e) \% \phi(n) = 1$ ערך זה של d , נקרא גם ההופכי המודולרי של e .
השלב הבא הוא ההצפנה: אם רוצים להצפין הודעה, m : $m^e \% n = c$, כאשר c היא ההודעה המוצפנת.
וכדי לבצע פענוח של c : $c^d \% n = m$, כאשר m היא ההודעה המקורית.

*הסימון במשוואה המתמטית, הוא פי, פשוט נכתב אחרת בדלל מגבלות word.



ערכים חשובים

הסיסמה לכניסה לצד האדמיין: IdolsAdmin1!

המפתח הפרטי לצפייה בטבלת ההיסטוריה: 2753

ערך הגדרת ADMIN_HASH:

f4b8b1d7625448978da1e95160808a56992a879cd7d7b6d4b6a6f7e95db8782
4

ערך הגדרת ADMIN_PRIVATE_KEY: 8 e6dbcb8a3461e581f3859d
45d8cb900814b396f76566d227c2d668e30c62828



הקוד

Server.py

```
import threading
from SQLITE import Databases
from communication import Communication
from rsa import RSA
from URL_Validators import URL_Validator

class Server(Communication):
    def __init__(self, ip, port):
        """
        הצפנה מפתחות יצירת, חייבור פתיחת: שרת אתחול,
        ודירוגים קישורים לעיבוד נדרשים פרמטרים והגדרת
        """
        super().__init__(ip, port, is_server=True)
        self.socket.bind((self.ip, self.port))
        self.socket.listen(100)
        print("Server running...")

        self.encryption = RSA(key_size=16)
        self.encryption.generate_keys()
        self.db_name = 'ido_databases.db'

        self.issue_weights = {
            "The URL is not valid.": 5,
            "The link is not secure (does not use HTTPS).": 4,
            "The domain name is invalid or contains suspicious characters.":
3,
            "The website does not use SSL (the link is not encrypted).": 4,
            "The link contains suspicious keywords": 2
        }

        self.url_validator = URL_Validator()
        self.connection_count = 0

    def send_large_message(self, client_socket, message):
        """
        ללקוח (chunks) בחתיכות ארוכה הודעה שליחת
        <END>-ב מסומן סיום עם
        """
        CHUNK_SIZE = 1024
        message_str = str(message)
        for i in range(0, len(message_str), CHUNK_SIZE):
            client_socket.send(message_str[i:i + CHUNK_SIZE].encode())
        client_socket.send(b"<END>")

    def check_url(self, url, database, update_database=True):
        """
        וסיבות דירוג ומחזיר URL כתובת על בדיקות מבצע
        בהתאם הנתונים מסד את יעדכן, update_database=True אם
        """
```



```

"""
if len(url) > 2000:
    return {
        "url": url,
        "rating": 1,
        "reasons": ["The URL is too long."]
    }
reasons = self.url_validator.check_url(url)
rating = self.rate_url(reasons) if reasons else 5

if update_database:
    if reasons:
        database.add_to_blacklist(url, reasons)
    else:
        database.remove_from_blacklist(url)

return {"reasons": reasons, "rating": rating}

def rate_url(self, reasons):
    """
    שנמצאו סיבות משקלי לפי (1-5) בטיחות דירוג חישוב.
    """
    total_weight = sum(self.issue_weights.get(reason, 1) for reason in
reasons)
    max_possible_weight = sum(self.issue_weights.values())
    safety_score = (1 - (total_weight / max_possible_weight)) * 100

    if safety_score >= 90:
        return 5
    elif safety_score >= 75:
        return 4
    elif safety_score >= 60:
        return 3
    elif safety_score >= 40:
        return 2
    else:
        return 1

def handle_client(self, client_socket, address):
    """
    בודד בלקון טיפול:
    - הצפנה מפתחות ניהול
    - בקשות פענוח
    - תגובות שליחת
    - הלקון לפקודות בהתאם הנתונים למסד גישה
    """
    try:
        database = Databases(self.db_name)
        client_ip, client_port = address
        client_public_key = eval(client_socket.recv(1024).decode())
        client_e, client_n = client_public_key
        client_socket.send(str((self.encryption.e,
self.encryption.n)).encode())

        while True:
            encrypted_data = client_socket.recv(1024).decode().strip()
            if not encrypted_data:

```



```

        break

    decrypted_data =
self.encrypted.decrypt(eval(encrypted_data))

    if decrypted_data.startswith("RATING:"):
        try:
            content = decrypted_data[len("RATING:"):]
            parts = content.split(",", 2)
            if len(parts) != 3:
                raise ValueError("שגוי דירוג פורמט.")
            name, rating_str, feedback = parts
            rating = int(rating_str)
            database.insert_rating(name.strip(), rating,
feedback.strip())

            response = {"message": "בהצלחה התקבל הדירוג"}
        except Exception as e:
            response = {"message": f"דירוג בעיבוד שגיאה"}

    {str(e)}"

        encrypted_response =
self.encrypted.encrypt(str(response), e=client_e, n=client_n)
        self.send_large_message(client_socket,
str(encrypted_response))
        continue

    if decrypted_data == "GET_ALL_BLACKLISTED_LINKS":
        links = database.get_all_blacklisted_links()
        response = {"links": links}
        encrypted_response =
self.encrypted.encrypt(str(response), e=client_e, n=client_n)
        self.send_large_message(client_socket,
str(encrypted_response))
        continue

    if decrypted_data.startswith("REMOVE_FROM_BLACKLIST:"):
        link_to_remove = decrypted_data.split(":", 1)[1]
        database.remove_from_blacklist(link_to_remove)
        response = {"message": f"{link_to_remove} הוסר
מהרשימה"}

        encrypted_response =
self.encrypted.encrypt(str(response), e=client_e, n=client_n)
        self.send_large_message(client_socket,
str(encrypted_response))
        continue

    if decrypted_data.startswith("UPDATE_BLACKLIST:"):
        parts = decrypted_data.split(":", 2)
        if len(parts) == 3:
            link_to_update = parts[1]
            new_reasons = eval(parts[2])
            database.add_to_blacklist(link_to_update,
new_reasons)

            response = {"message": f"עודכנו הסיבות"}
            {link_to_update)}"
        else:
            response = {"message": "שגוי פורמט"}

```




```

        encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
        self.send_large_message(client_socket,
str(encrypted_response))
        continue

        if decrypted_data.startswith("GET_REASONS:"):
            link = decrypted_data.split(":", 1)[1]
            reasons = database.get_link_reasons(link)
            response = {"reasons": reasons}
            encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
            self.send_large_message(client_socket,
str(encrypted_response))
            continue

        if decrypted_data == "GET_ALL_RATINGS":
            ratings = database.get_all_ratings()
            response = {"ratings": ratings}
            encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
            self.send_large_message(client_socket,
str(encrypted_response))
            continue

        if decrypted_data == "GET_AVERAGE_RATING":
            average = database.get_average_rating()
            response = {"average": average}
            encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
            self.send_large_message(client_socket,
str(encrypted_response))
            continue

        if decrypted_data == "GET_CONNECTION_COUNT":
            count = self.connection_count
            response = {"count": count}
            encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
            self.send_large_message(client_socket,
str(encrypted_response))
            continue

        if decrypted_data == "GET_LAST_10_HISTORY":
            history = database.get_last_10_history()
            response = {"history": history}
            encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
            self.send_large_message(client_socket,
str(encrypted_response))
            continue

        if decrypted_data == "RESET_BLACKLIST":
            database.clear_blacklist()
            response = {"message": "השחזור הרשימה"}
            encrypted_response =
self.encryption.encrypt(str(response), e=client_e, n=client_n)
            self.send_large_message(client_socket,

```



```

str(encrypted_response))
    continue

    if decrypted_data == "RESET_RATINGS":
        database.clear_ratings()
        response = {"message": "בהצלחה אופסה הדירוגים טבלת"}
        encrypted_response =
self.encrypted_response.encrypt(str(response), e=client_e, n=client_n)
        self.send_large_message(client_socket,
str(encrypted_response))
        continue

        result = self.check_url(decrypted_data, database,
update_database=False)
        response = {
            "blacklisted":
database.is_url_blacklisted(decrypted_data),
            "message": "",
            "reasons": result["reasons"],
            "rating": result["rating"]
        }

        is_suspicious = result["rating"] < 5

        rsa_for_ip = RSA()
        rsa_for_ip.e = 17
        rsa_for_ip.n = 3233

        encrypted_ip_chars = rsa_for_ip.encrypt(client_ip)
        encrypted_ip_str = ','.join(map(str, encrypted_ip_chars))

        database.log_to_history(encrypted_ip_str, client_port,
decrypted_data, is_suspicious)

        encrypted_response = self.encrypted_response.encrypt(str(response),
e=client_e, n=client_n)
        self.send_large_message(client_socket,
str(encrypted_response))

    except Exception as e:
        print(f"[SERVER ERROR] with {address}: {str(e)}")
    finally:
        if 'database' in locals():
            database.close()
            client_socket.close()
            self.connection_count = max(0, self.connection_count - 1)

def start_server(self):
    """
    לקוח לכל נפרד thread ומפעיל חדשים לחיבורים מאזין
    """
    while True:
        client, address = self.socket.accept()
        self.connection_count += 1
        print(f"New connection from {address} (Total connections:
{self.connection_count})")
        threading.Thread(target=self.handle_client, args=(client,

```



```
address)).start()
```

```
if __name__ == "__main__":  
    server = Server('192.168.8.140', 8002)  
    server.start_server()
```



client.py

```

from communication import Communication
from rsa import RSA
from gui import GUI

class Client(Communication):
    """
    הצפנה ניהול, השרת עם לתקשורת לקוח מחלקת,
    גרפי משתמש ממשק ובניית
    """

    def __init__(self, root):
        """
        לקוח אתחול:
        - לשרת חיבור יצירת
        - מפתחות RSA יצירת
        - ממשק אתחול GUI
        - לשרת התחברות
        """
        super().__init__('192.168.8.140', 8002)
        self.root = root
        self.encryption = RSA(key_size=16)
        self.encryption.generate_keys()
        print(f"Client keys generated - e: {self.encryption.e}, d: {self.encryption.d}, n: {self.encryption.n}")
        self.server_e = None
        self.server_n = None
        self.gui = GUI(root, self)
        self.connect_to_server()

    def connect_to_server(self):
        """
        לשרת התחברות מבצע:
        - הלקוח של ציבורי מפתח שולח
        - השרת מפתח את מקבל
        """
        try:
            self.connect()
            self.send_message(str((self.encryption.e, self.encryption.n)))

            server_key_str = self.receive_message()

            if not server_key_str:
                raise Exception("Did not receive server key from server")

            self.server_e, self.server_n = eval(server_key_str)

        except Exception as e:
            print(f"[ERROR] Failed to connect to server: {str(e)}")
            raise e

    def receive_large_message(self, timeout=10):
        """
        בחלקים מהשרת ארוכה הודעה קבלת (chunks),

```



```

        """
        תד <END> לסיומ.
        """
        import socket
        CHUNK_SIZE = 1024
        data = []
        self.socket.settimeout(timeout)

        while True:
            try:
                chunk = self.socket.recv(CHUNK_SIZE).decode()
                if chunk.endswith("<END>"):
                    data.append(chunk[:-5])
                    break
                data.append(chunk)
            except socket.timeout:
                print("Timeout while receiving message")
                break
            except Exception as e:
                print(f"Error reading message: {e}")
                break

        return ''.join(data) if data else None

    def send_url_check_request(self, url):
        """
        לשרת קישור בדיקת בקשת שליחת,
        ופיענוחה מוצפנת תשובה קבלת.
        """
        try:
            self.socket.settimeout(10)
            encrypted_url = self.encryption.encrypt(url, e=self.server_e,
n=self.server_n)
            self.send_message(str(encrypted_url))
            encrypted_response = self.receive_large_message(timeout=20)
            if not encrypted_response:
                raise ValueError("No response from server")
            return eval(self.encryption.decrypt(eval(encrypted_response)))
        finally:
            self.socket.settimeout(None)

    def request_link_reasons(self, link):
        """
        השחזרה מהרשימה מסוים קישור לחסימת סיבות בקשת.
        """
        try:
            self.socket.settimeout(10)
            encrypted_request =
self.encryption.encrypt(f"GET_REASONS:{link}", e=self.server_e,
n=self.server_n)
            self.send_message(str(encrypted_request))
            encrypted_response = self.receive_large_message(timeout=10)
            if not encrypted_response:
                raise ValueError("No response from server")
            return
eval(self.encryption.decrypt(eval(encrypted_response))).get("reasons", [])
        finally:
            self.socket.settimeout(None)

```



```

def submit_rating(self, name, rating, feedback):
    """
    לשרת (פידבק, דירוג, שם) משתמש דירוג שליחת
    """
    message = f"RATING:{name},{rating},{feedback}"
    encrypted_message = self.encrypted_message = self.encrypted_message.encrypt(message, e=self.server_e,
n=self.server_n)
    self.send_message(str(encrypted_message))

def disconnect_and_exit(self):
    """
    הגרפי הממשק וסגירת מהשרת ניתוק
    """
    try:
        self.disconnect()
    except Exception as e:
        print(f"Error during cleanup: {e}")
    finally:
        self.root.destroy()

if __name__ == "__main__":
    # הלקוח ממשיך והפעלת פתיחה מסך
    from gui import show_splash_screen_and_wait
    import tkinter as tk

    show_splash_screen_and_wait()
    root = tk.Tk()
    app = Client(root)
    root.protocol("WM_DELETE_WINDOW", app.disconnect_and_exit)
    root.mainloop()

```



communication.py

```
import socket

class Communication:
    """
    ללקוח השרת בין בסיסית תקשורת מחלקת.
    מפתחות והחלפת, הודעות קבלת, הודעות שליחת, חיבור ביצירת מטפלת.
    """
    def __init__(self, ip, port, is_server=False):
        # לקוח או שרת) האובייקט וסוג, הפורט, IP-ה כתובת את מאחסן
        self.ip = ip
        self.port = port
        self.is_server = is_server
        self.socket = socket.socket()
        self.connected = False

    def connect(self):
        """
        (לקוח הוא האובייקט אם) לשרת להתחבר מנסה.
        True-ל connected את מעדכן - הצליח אם
        """
        if not self.is_server:
            try:
                self.socket.connect((self.ip, self.port))
                self.connected = True
            except Exception:
                self.connected = False

    def send_message(self, message):
        """
        (לקוח או שרת) השני לצד הודעה שולח.
        מחדש להתחבר מנסה, חיבור אין אם
        """
        if not self.connected:
            self.connect()
        if self.connected and self.socket:
            try:
                self.socket.send(message.encode())
            except Exception:
                self.connected = False

    def receive_message(self, buffer_size=1024):
        """
        השני מהצד הודעה מקבל.
        נכשל אם None או, כמחזרת ההודעה את מחזיר
        """
        if self.connected and self.socket:
            try:
                message = self.socket.recv(buffer_size).decode()
                return message
            except Exception:
                self.connected = False
                return None
        else:
```



```
        return None

def send_public_key(self, public_key):
    """
    (שרת זה אם רק) ללקוח הציבורי המפתח את שולח
    """
    if self.is_server and self.socket:
        try:
            key_str = f"{public_key[0]},{public_key[1]}"
            self.socket.send(key_str.encode())
        except Exception:
            self.connected = False

def receive_public_key(self):
    """
    (לקוח זה אם רק) מהשרת ציבורי מפתח מקבל
    נכשל אם None או tuple-כ המפתח את מחזיר
    """
    if not self.is_server and self.socket:
        try:
            key_str = self.socket.recv(1024).decode()
            return eval(key_str)
        except Exception as e:
            print(f"Error receiving public key: {e}")
            return None

def disconnect(self):
    """
    הצדדים בן הציבור את סוגר
    """
    if self.socket:
        self.socket.close()
    self.connected = False
```




AdminClient.py

```

from client import Client
import tkinter as tk
from hash import HashManager
from rsa import RSA
from tkinter import simpledialog, messagebox, Toplevel, Text, WORD, BOTH

class AdminClient(Client):
    """
    הרגיל מהלקוח יורשת, מנהל לקוח מחלקת
    ודירוגים רשימות ניהול כמו לממשק ניהול יכולות מוסיפה
    """
    def __init__(self, root):
        """הצלחה לאחר המנהל ממשק את ומאתחלת סיסמה אימות מבצעת."""

        manager = HashManager()

        while True:
            password = simpledialog.askstring("Authentication", "Enter
password:")

            if password is None:
                messagebox.showinfo("Cancelled", "Login was cancelled by the
user.")
                root.destroy()
                return

            if manager.verify_password(password):
                break
            else:
                messagebox.showerror("Error", "Incorrect password, please try
again.")

        super().__init__(root)
        self.add_admin_controls()

    def add_admin_controls(self):
        """המנהל של לממשק נוספים שליטה ואזורי כפתורים מוסיף"""
        title_label = tk.Label(
            self.root,
            text="🔒 System Management - Admin Panel 🔒",
            bg="#f0f0f0",
            font=("Arial", 16, "bold")
        )
        title_label.pack(pady=15)

        # פעילים חייבורים תצוגת
        self.connection_count_label = tk.Label(
            self.root,
            text="Active connections: 0",
            bg="#f0f0f0",
            font=("Arial", 12)
        )
        self.connection_count_label.pack(pady=(0, 15))

```

```

בתחתית לכפתורים מסגרת
button_frame = tk.Frame(self.root, bg="#f0f0f0")
button_frame.pack(side=tk.BOTTOM, pady=15)

def create_button(text, color, command):
    return tk.Button(
        button_frame,
        text=text,
        bg=color,
        fg="white",
        font=("Arial", 10, "bold"),
        width=20,
        command=command
    )

# אופקי בריווח כפתורים הוספת
create_button("Refresh Blacklist", "firebrick",
self.refresh_blacklist).pack(side=tk.LEFT, padx=5)
create_button("Ratings Table", "steelblue",
self.show_ratings_window).pack(side=tk.LEFT, padx=5)
create_button("Average Rating", "slateblue",
self.show_average_rating).pack(side=tk.LEFT, padx=5)
create_button("Database Management", "darkred",
self.open_database_management_window).pack(side=tk.LEFT, padx=5)
create_button("Recent Checks", "darkorange",
self.show_last_10_checks).pack(side=tk.LEFT, padx=5)

# חיבורים ספירת התחלת
self.update_connection_count()

def open_database_management_window(self):
    """שחורה רשימה איפוס: הנתונים מסד על פעולות לניהול חלון פותח"""
    window = tk.Toplevel(self.root)
    window.title("Database Management")
    window.geometry("300x180")
    window.configure(bg="#f0f0f0")

    tk.Label(window, text="Select an action:", bg="#f0f0f0",
font=("Arial", 12, "bold")).pack(pady=10)

    tk.Button(window, text="Reset Blacklist and its connections",
bg="darkred", fg="white",
command=self.confirm_reset_blacklist).pack(pady=5)

    tk.Button(window, text="Reset Ratings", bg="darkred", fg="white",
command=self.confirm_reset_ratings).pack(pady=5)

def reset_blacklist(self):
    try:
        self.send_message(str(self.encryption.encrypt("RESET_BLACKLIST",
e=self.server_e, n=self.server_n)))
        messagebox.showinfo("Success", "Blacklist and links were
successfully cleared.")
    except Exception as e:
        messagebox.showerror("Error", f"Error clearing blacklist:

```



```
{str(e)}")

def reset_ratings(self):
    try:
        self.send_message(str(self.encrypted.encrypt("RESET_RATINGS",
e=self.server_e, n=self.server_n)))
        messagebox.showinfo("Success", "All ratings were successfully
deleted.")
    except Exception as e:
        messagebox.showerror("Error", f"Error clearing ratings:
{str(e)}")

def confirm_reset_blacklist(self):
    if messagebox.askyesno("Confirmation", "Are you sure you want to
delete the blacklist and its links?"):
        self.reset_blacklist()

def confirm_reset_ratings(self):
    if messagebox.askyesno("Confirmation", "Are you sure you want to
delete all ratings?"):
        self.reset_ratings()

def refresh_blacklist(self):
    try:
        # השחזורה הרשימה כל את לקבל בקשה שלח
        self.send_message(
            str(self.encrypted.encrypt("GET_ALL_BLACKLISTED_LINKS",
e=self.server_e, n=self.server_n)))
        encrypted_response = self.receive_large_message(timeout=20)
        response =
eval(self.encrypted.decrypt(eval(encrypted_response)))
        blacklisted_links = response.get("links", [])

        updated = False
        change_log = []

        for link in blacklisted_links:
            result = self.check_url_direct(link)

            if result and isinstance(result, dict):
                reasons = result.get("reasons", [])
                rating = result.get("rating", 0)
                is_available = rating > 1 # דמין לא אתר = 1 דירוג

                if is_available:
                    if len(reasons) == 0:
                        # מוחק - סיבות וללא חיי אתר
                        self.send_message(
str(self.encrypted.encrypt(f"REMOVE_FROM_BLACKLIST:{link}", e=self.server_e,
n=self.server_n)))

                        self.receive_large_message(timeout=10)
                        change_log.append(f"✅ Removed from blacklist:
{link}")

                        updated = True
```



```

else:
    # להשאיר - סיבות עם חיי אתר
    print(f"[INFO] {link} remained in the blacklist
with reasons: {reasons}")
else:
    # תמיד להשאיר - זמין לא אתר
    print(f"[INFO] {link} remained (site unavailable,
existing reasons: {reasons}")

if updated:
    log_message = "\n".join(change_log)
    print(log_message)
    messagebox.showinfo("Blacklist Refreshed", f"The blacklist
was updated:\n\n{log_message}")
else:
    messagebox.showinfo("Blacklist Refreshed", "No changes were
made to the blacklist.")

except Exception as e:
    messagebox.showerror("Error", f"Error updating blacklist:
{str(e)}")

def update_connection_count(self):
    try:
        # מהשרת חיבורים למספר בקשה

self.send_message(str(self.encrypted.encrypt("GET_CONNECTION_COUNT",
e=self.server_e, n=self.server_n)))
        encrypted_response = self.receive_large_message(timeout=10)
        response =
eval(self.encrypted.decrypt(eval(encrypted_response)))
        count = response.get("count", 0)

        # עצמו המנהל את לכלול לא כדי 1 של הפחתה
        display_count = max(0, count - 1)

        # למצב בהתאם צבע קבע
        color = "green" if display_count > 0 else "red"
        text = f"Active connections: {display_count}"

        self.connection_count_label.config(text=text, fg=color)
    except Exception as e:
        self.connection_count_label.config(text="Active connections:
Unavailable", fg="red")

    # שניות 5 בעוד מחדש רענון
    self.root.after(5000, self.update_connection_count)

def check_url_direct(self, url):
    """שרת מול ישירה בדיקה"""
    try:
        self.socket.settimeout(10)
        encrypted_url = self.encrypted.encrypt(url, e=self.server_e,
n=self.server_n)
        self.send_message(str(encrypted_url))

        encrypted_response = self.receive_large_message(timeout=20)

```



```

        if not encrypted_response:
            raise ValueError("No response from server")

        response =
eval(self.encrypted.decrypt(eval(encrypted_response)))
        return response

    except Exception as e:
        print(f"Error during direct URL check: {e}")
        return {"reasons": []}

    finally:
        self.socket.settimeout(None)

def show_ratings_window(self):
    try:
        # דירוגים לקבלת בקשה שליחת
        self.send_message(str(self.encrypted.encrypt("GET_ALL_RATINGS",
e=self.server_e, n=self.server_n)))

        # מהשרת תשובה קבלת
        encrypted_response = self.receive_large_message(timeout=10)
        response =
eval(self.encrypted.decrypt(eval(encrypted_response)))
        ratings = response.get("ratings", [])

        print("[DEBUG] Received ratings:", ratings) # ← כאן

        # הטבלה להצגת חלון יצירת
        window = tk.Toplevel(self.root)
        window.title("Ratings Table")
        window.geometry("600x400")

        text_area = tk.Text(window, wrap=tk.WORD)
        text_area.pack(fill=tk.BOTH, expand=True)

        # כותרות
        text_area.insert(tk.END, "ID | Name | Rating | Feedback\n")
        text_area.insert(tk.END, "-" * 50 + "\n")

        # הטבלה שורות הצגת
        for idx, (name, rating, feedback) in enumerate(ratings, start=1):
            text_area.insert(tk.END, f"{idx} | {name} | {rating} |
{feedback}\n")

        text_area.config(state=tk.DISABLED)

    except Exception as e:
        messagebox.showerror("Error", f"Error loading ratings: {str(e)}")

def show_average_rating(self):
    try:
self.send_message(str(self.encrypted.encrypt("GET_AVERAGE_RATING",
e=self.server_e, n=self.server_n)))
        encrypted_response = self.receive_large_message(timeout=10)
        response =

```



```

eval(self.encryption.decrypt(eval(encrypted_response)))
    average = response.get("average", 0)

    messagebox.showinfo("Average Rating", f"The current average is:
{average:.2f}/5")

    except Exception as e:
        messagebox.showerror("Error", f"Error fetching average rating:
{str(e)}")

def show_last_10_checks(self):
    try:
        # הפרטי המפתח לאימות שהגענו לראות דיבוג הדפסת
        print("[DEBUG] Starting private key verification using
ADMIN_PRIVATE_KEY")

        # יצירת HashManager עם ADMIN_PRIVATE_KEY
        manager = HashManager("ADMIN_PRIVATE_KEY")

        # מהמשתמש המפתח בקשת
        private_key_input = simplifiedialog.askstring("Verify Private Key",
"Enter the private key:")

        if private_key_input is None:
            messagebox.showinfo("Cancelled", "The action was cancelled by
the user.")
            return

        # מהסביבה ההאש מול המפתח אימות
        if not manager.verify_password(private_key_input):
            messagebox.showerror("Error", "Incorrect key.")
            return

        # תקין אימות אחרי למספר המרה
        private_key = int(private_key_input)
        rsa_for_ip = RSA()
        rsa_for_ip.d = private_key
        rsa_for_ip.n = 3233

        # היסטוריה לקבלת בקשה שליחת

self.send_message(str(self.encryption.encrypt("GET_LAST_10_HISTORY",
e=self.server_e, n=self.server_n)))
        encrypted_response = self.receive_large_message(timeout=10)
        response =
eval(self.encryption.decrypt(eval(encrypted_response)))
        history = response.get("history", [])

        # ההיסטוריה להצגת חלון יצירת
        window = Toplevel(self.root)
        window.title("Recent 10 Checks")
        window.geometry("600x400")

        text_area = Text(window, wrap=WORD)
        text_area.pack(fill=BOTH, expand=True)

        text_area.insert("end", "IP | Port | Link | Suspicious |

```



```

Timestamp\n")
    text_area.insert("end", "-" * 60 + "\n")

    # ה-IP ופענוח הרשומות כל על מעבר
    for record in history:
        encrypted_ip, port, link, is_suspicious, timestamp = record
        try:
            encrypted_ip_numbers = [int(num) for num in
encrypted_ip.split(',')]
            decrypted_ip = rsa_for_ip.decrypt(encrypted_ip_numbers)
        except Exception:
            decrypted_ip = "[Decryption Failed]"

            text_area.insert("end", f"{decrypted_ip} | {port} | {link} |
{is_suspicious} | {timestamp}\n")

            text_area.config(state="disabled")

    except Exception as e:
        messagebox.showerror("Error", f"Error fetching history:
{str(e)}")

if __name__ == "__main__":
    print("[DEBUG] Launching AdminClient UI...")
    root = tk.Tk()
    app = AdminClient(root)
    root.protocol("WM_DELETE_WINDOW", app.disconnect_and_exit)
    print("[DEBUG] Starting Tkinter mainloop")
    root.mainloop()

```



GUI.py

```
import tkinter as tk
from tkinter import messagebox, Toplevel, StringVar, IntVar
import webbrowser
from PIL import Image, ImageTk
def load_logo_image(path, size=(80, 80)):
    """
    -ב לשימוש ImageTk אובייקט ומחזיר, גודלה את משנה, לוגו תמונת טוען
    Tkinter.
    """
    try:
        resample = Image.Resampling.LANCZOS
    except AttributeError:
        resample = Image.ANTIALIAS
    img_raw = Image.open(path)
    img_resized = img_raw.resize(size, resample)
    return ImageTk.PhotoImage(img_resized)

def show_splash_screen_and_wait():
    """
    התוכנית - לוחץ לא המשתמש אם. המשיך וכפתור לוגו עם זמני פתיחה מסך מציג
    נסגרת.
    """
    splash = tk.Tk()
    splash.title("Welcome")
    splash.geometry("500x500+500+200")
    splash.configure(bg="#f0f0f0")

    user_continued = [False] # הפונקציות בתוך לשנות כדי ברשימה נשתמש
    הפנימיות

    def on_continue():
        user_continued[0] = True
        splash.destroy()

    def on_close():
        splash.destroy()

    splash.protocol("WM_DELETE_WINDOW", on_close)

    title_label = tk.Label(splash, text="WELCOME!", font=("Arial", 24,
"bold"), bg="#f0f0f0", fg="#333333")
    title_label.place(relx=0.5, rely=0.2, anchor="center")

    logo_image = load_logo_image("PhishIdoLogo.png", size=(150, 150))
    logo_label = tk.Label(splash, image=logo_image, bg="#f0f0f0")
    logo_label.place(relx=0.5, rely=0.45, anchor="center")

    continue_button = tk.Button(splash, text="continue", command=on_continue,
bg="#4CAF50", fg="white", padx=20, pady=10, font=("Arial", 12, "bold"))
```



```

continue_button.place(relx=0.5, rely=0.75, anchor="center")

splash.mainloop()

# התוכנית את נסיים - המשך לחץ לא המשתמש אם
if not user_continued[0]:
    exit()

class GUI:
    """
    שליחת, תוצאות הצגת, קישורים בדיקת מאפשר. הקצה למשתמש עיקרי גרפי ממשק
    מידע והצגת דירוגים.
    """
    def __init__(self, root, client):
        self.root = root
        self.client = client
        self.setup_gui()

    def setup_gui(self):
        self.root.title("Anti PhishIdo")
        self.root.geometry("900x400")
        self.root.configure(bg="#f0f0f0")

        self.logo_image = load_logo_image("PhishIdoLogo.png")
        logo_label = tk.Label(self.root, image=self.logo_image, bg="#f0f0f0",
bd=0)
        logo_label.place(x=10, y=10)

        info_button = tk.Button(self.root, text="i",
command=self.show_info_window, bg="#2196F3", fg="white", font=("Arial", 12,
"bold"), width=2, height=1, cursor="hand2")
        info_button.place(relx=1.0, rely=0.0, anchor="ne", x=-10, y=10)

        self.form_frame = tk.Frame(self.root, bg="#f0f0f0")
        self.form_frame.pack(pady=20)

        tk.Label(self.form_frame, text="enter link:", bg="#f0f0f0").pack()
        self.entry_name = tk.Entry(self.form_frame, width=50)
        self.entry_name.pack(pady=10)

        tk.Button(self.form_frame, text="Check safety",
command=self.check_url_safety, bg="#4CAF50", fg="white").pack()

        self.result_frame = tk.Frame(self.root, bg="#f0f0f0")
        self.result_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=10)

        rate_button = tk.Button(self.root, text="Rate your experience!",
command=self.open_rating_form, bg="blue", fg="white")
        rate_button.pack(anchor="ne", padx=10, pady=10)

    def check_url_safety(self):
        """בממשק הסריקה תוצאת את ומציג לבדיקה הקישור את שולח"""
        for widget in self.result_frame.winfo_children():
            widget.destroy()
        url = self.entry_name.get().strip()
        if not url:

```

```

        tk.Label(self.result_frame, text="Please enter a link to check",
fg="red", bg="#f0f0f0").pack()
        return
    try:
        response = self.client.send_url_check_request(url)
        if response.get("blacklisted", False):
            tk.Label(self.result_frame, text="⚠ The link is on the
blacklist!", fg="red", bg="#f0f0f0").pack()
            tk.Button(self.result_frame, text="View Reasons",
command=lambda: self.show_reasons_for_link(url), bg="#f0ad4e",
fg="white").pack(pady=10)
            return
        self.display_results(url, response)
    except Exception as e:
        tk.Label(self.result_frame, text=f"Error: {str(e)}", fg="red",
bg="#f0f0f0").pack()

    def show_reasons_for_link(self, link):
        reasons = self.client.request_link_reasons(link)
        self.show_reasons(reasons)

    def show_reasons(self, reasons):
        for widget in self.result_frame.winfo_children():
            widget.destroy()
        if not reasons:
            tk.Label(self.result_frame, text="No reasons found", fg="black",
bg="#f0f0f0").pack()
        else:
            tk.Label(self.result_frame, text="Reasons:", fg="blue",
font=("Arial", 12, "bold"), bg="#f0f0f0").pack()
            for reason in reasons:
                tk.Label(
                    self.result_frame,
                    text=f"- {reason}",
                    fg="black",
                    bg="#f0f0f0",
                    anchor="center",
                    justify="center"
                ).pack(anchor="center", pady=2)

    def display_results(self, url, response):
        for widget in self.result_frame.winfo_children():
            widget.destroy()

        reasons = response.get("reasons", [])
        rating = response.get("rating", 0)

        if not reasons:
            tk.Label(
                self.result_frame,
                text="✓ The link is safe",
                fg="green",
                bg="#f0f0f0",
                font=("Arial", 12)
            ).pack(pady=5)
        else:

```



```

tk.Label(
    self.result_frame,
    text="⚠ The link is dangerous",
    fg="red",
    bg="#f0f0f0",
    font=("Arial", 12, "bold")
).pack(pady=5)

for reason in reasons:
    tk.Label(
        self.result_frame,
        text=reason,
        fg="black",
        bg="#f0f0f0",
        font=("Arial", 10),
        anchor="center",
        justify="center"
    ).pack(anchor="center", pady=2)

stars = "★" * rating + "☆" * (5 - rating)
tk.Label(
    self.result_frame,
    text=f"Safety Rating: {stars}",
    fg="orange",
    bg="#f0f0f0",
    font=("Arial", 11, "bold")
).pack(pady=5)

if rating == 5:
    visit_link_button = tk.Label(
        self.result_frame,
        text="Click here to visit the site",
        fg="blue",
        cursor="hand2",
        bg="#f0f0f0",
        font=("Arial", 10, "underline")
    )
    visit_link_button.pack(pady=5)
    visit_link_button.bind("<Button-1>", lambda e:
webbrowser.open(url))

def open_rating_form(self):
    """החוויה על דעת וחוות דירוג לתת למשתמש המאפשר פופאפ חלון פותח"""
    popup = Toplevel(self.root)
    popup.title("Rate Us")
    popup.geometry("350x300")
    popup.configure(bg="#f0f0f0")

    tk.Label(popup, text="Name(optional)", bg="#f0f0f0").pack(pady=5)
    name_var = StringVar()
    tk.Entry(popup, textvariable=name_var, width=30).pack(pady=5)

    tk.Label(popup, text="Rate your experience!",
bg="#f0f0f0").pack(pady=5)
    star_frame = tk.Frame(popup, bg="#f0f0f0")

```

```

star_frame.pack()
self.rating_var = IntVar(value=0)
self.star_buttons = []
for i in range(1, 6):
    star_btn = tk.Button(star_frame, text="☆", font=("Arial", 24),
command=lambda val=i: self.set_rating(val), bg="#f0f0f0", fg="gold", bd=0)
    star_btn.pack(side=tk.LEFT, padx=2)
    self.star_buttons.append(star_btn)

tk.Label(popup, text="Your feedback", bg="#f0f0f0").pack(pady=5)
feedback_var = StringVar()
tk.Entry(popup, textvariable=feedback_var, width=30).pack(pady=5)

tk.Button(popup, text="Submit Rating", command=lambda:
self.submit_rating(popup, name_var.get(), self.rating_var.get(),
feedback_var.get(), bg="#4CAF50", fg="white").pack(pady=10)

def set_rating(self, rating):
    self.rating_var.set(rating)
    for i, star_btn in enumerate(self.star_buttons):
        star_btn.config(text="★" if i < rating else "☆", fg="gold")

def submit_rating(self, popup, name, rating, feedback):
    if not feedback or rating == 0:
        messagebox.showerror("Error", "Please provide feedback and select
a rating!")
    return
    self.client.submit_rating(name or "Anonymous", rating, feedback)
    messagebox.showinfo("Thank you!", "Your rating has been submitted!")
    popup.destroy()

def show_info_window(self):
    explanations = {
        "The URL is invalid.": "The URL format is incorrect.",
        "The link is not secure (does not use HTTPS).": "The link is not
encrypted.",
        "The domain name is invalid or contains suspicious characters.":
"The domain appears suspicious.",
        "The website does not use SSL encryption (the link is not
encrypted).": "No security certificate (SSL).",
        "The link contains suspicious keywords.": "The keywords may
indicate phishing (e.g., login, password, account).",
        "The site is unavailable or not responding.": "The site is
unresponsive or does not exist."
    }
    info_win = Toplevel(self.root)
    info_win.title("Information")
    info_win.geometry("480x400")
    info_win.configure(bg="#ffffff")
    tk.Label(info_win, text="Explanation of reasons for marking a link as
suspicious:", font=("Arial", 12, "bold"), bg="#ffffff").pack(pady=10)
    for reason, explanation in explanations.items():
        tk.Label(info_win, text=f"• {reason}: {explanation}",
bg="#ffffff", wraplength=450, justify="left").pack(anchor="w", padx=15,
pady=5)

```



SQLITE.py

```
import sqlite3
import datetime

class Databases:
    VALID_REASONS = {
        1: "The URL is not valid.",
        2: "The link is not secure (does not use HTTPS).",
        3: "The domain name is invalid or contains suspicious characters.",
        4: "The website does not use SSL (the link is not encrypted).",
        5: "The link contains suspicious keywords"
    }

    def __init__(self, db_name):
        self.conn = sqlite3.connect(db_name)
        self.cursor = self.conn.cursor()
        self.create_tables()
        self.initialize_reasons()

    def create_tables(self):
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS BLACKLIST (
            LINK_ID INTEGER PRIMARY KEY AUTOINCREMENT,
            LINK TEXT UNIQUE NOT NULL)''')
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS REASONS (
            REASON_ID INTEGER PRIMARY KEY,
            NAME TEXT UNIQUE NOT NULL)''')
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS LINKS_REASONS (
            LINK_ID INTEGER,
            REASON_ID INTEGER,
            FOREIGN KEY(LINK_ID) REFERENCES BLACKLIST(LINK_ID),
            FOREIGN KEY(REASON_ID) REFERENCES REASONS(REASON_ID),
            UNIQUE(LINK_ID, REASON_ID))''')
        self.cursor.execute('''CREATE TABLE IF NOT EXISTS RATINGS (
            ID INTEGER PRIMARY KEY AUTOINCREMENT,
            NAME TEXT NOT NULL,
            RATING INTEGER NOT NULL CHECK(RATING >= 1 AND RATING <= 5),
            FEEDBACK TEXT)''')
        self.cursor.execute("""CREATE TABLE IF NOT EXISTS history (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            ip TEXT,
            port INTEGER,
            link TEXT,
            is_suspicious TEXT,
            timestamp TEXT
        )""")
        self.conn.commit()

    def initialize_reasons(self):
        """
        במילון מראש המוגדרות הסיבות לפי הנתונים במסד הסיבות טבלת את מאתחלת
        VALID_REASONS.
        עדכניות להבטיח כדי הקיימות הסיבות כל את תחילה מוחקת.
        """
```



```

"""
self.cursor.execute("DELETE FROM REASONS")
for reason_id, reason_name in self.VALID_REASONS.items():
    self.cursor.execute(
        "INSERT INTO REASONS (REASON_ID, NAME) VALUES (?, ?)",
        (reason_id, reason_name)
    )
self.conn.commit()

def add_to_blacklist(self, url, reasons):
    try:
        self.cursor.execute("SELECT LINK_ID FROM BLACKLIST WHERE LINK =
?", (url,))
        existing = self.cursor.fetchone()

        if existing:
            link_id = existing[0]
        else:
            self.cursor.execute("INSERT INTO BLACKLIST (LINK) VALUES
(?)", (url,))
            self.conn.commit()
            self.cursor.execute("SELECT LINK_ID FROM BLACKLIST WHERE LINK
= ?", (url,))
            link_id = self.cursor.fetchone()[0]

        self.cursor.execute("DELETE FROM LINKS_REASONS WHERE LINK_ID =
?", (link_id,))

        for reason in reasons:
            if reason not in self.VALID_REASONS.values():
                continue

            self.cursor.execute("SELECT REASON_ID FROM REASONS WHERE NAME
= ?", (reason,))
            reason_id_row = self.cursor.fetchone()

            if not reason_id_row:
                continue

            reason_id = reason_id_row[0]
            self.cursor.execute(
                "INSERT OR IGNORE INTO LINKS_REASONS (LINK_ID, REASON_ID)
VALUES (?, ?)",
                (link_id, reason_id)
            )

            self.conn.commit()

    except Exception as e:
        self.conn.rollback()

def get_link_reasons(self, url):
    '''כפרטמר המתקבל הקישור של הסיבות שליפת'''
    self.cursor.execute("SELECT LINK_ID FROM BLACKLIST WHERE LINK = ?",
(url,))
    result = self.cursor.fetchone()
    if not result:

```



```

        return []
    link_id = result[0]

    self.cursor.execute("""
        SELECT R.NAME FROM LINKS_REASONS LR
        JOIN REASONS R ON LR.REASON_ID = R.REASON_ID
        WHERE LR.LINK_ID = ?
    """, (link_id,))
    return [row[0] for row in self.cursor.fetchall()]

def remove_from_blacklist(self, url):
    self.cursor.execute("SELECT LINK_ID FROM BLACKLIST WHERE LINK = ?",
    (url,))
    result = self.cursor.fetchone()
    if result:
        link_id = result[0]
        self.cursor.execute("DELETE FROM LINKS_REASONS WHERE LINK_ID =
    ?", (link_id,))
        self.cursor.execute("DELETE FROM BLACKLIST WHERE LINK_ID = ?",
    (link_id,))
        self.conn.commit()

def insert_rating(self, name, rating, feedback):
    self.cursor.execute(
        "INSERT INTO RATINGS (NAME, RATING, FEEDBACK) VALUES (?, ?, ?)",
        (name, rating, feedback)
    )
    self.conn.commit()

def get_all_blacklisted_links(self):
    self.cursor.execute("SELECT LINK FROM BLACKLIST")
    return [row[0] for row in self.cursor.fetchall()]

def get_all_ratings(self):
    self.cursor.execute("SELECT NAME, RATING, FEEDBACK FROM RATINGS")
    return self.cursor.fetchall()

def get_average_rating(self):
    self.cursor.execute("SELECT AVG(RATING) FROM RATINGS")
    result = self.cursor.fetchone()
    return round(result[0], 2) if result and result[0] is not None else 0

def get_last_10_history(self):
    self.cursor.execute("SELECT ip, port, link, is_suspicious, timestamp
    FROM history ORDER BY id DESC LIMIT 10")
    return self.cursor.fetchall()

def is_url_blacklisted(self, url):
    self.cursor.execute("SELECT 1 FROM BLACKLIST WHERE LINK = ?", (url,))
    return self.cursor.fetchone() is not None

def log_to_history(self, ip, port, link, is_suspicious):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    self.cursor.execute("""
        INSERT INTO history (ip, port, link, is_suspicious, timestamp)
        VALUES (?, ?, ?, ?, ?)
    """, (ip, port, link, str(is_suspicious), timestamp))

```



```
self.conn.commit()

def clear_blacklist(self):
    self.cursor.execute("DELETE FROM LINKS_REASONS")
    self.cursor.execute("DELETE FROM BLACKLIST")
    self.conn.commit()

def clear_ratings(self):
    self.cursor.execute("DELETE FROM RATINGS")
    self.conn.commit()

def close(self):
    self.conn.close()
```




URL Validators.py

```
import re
import requests
from urllib.parse import urlparse

class URL_Validator:
    def __init__(self):
        # לפישינג חשודות מפתח מילות
        # והתחברויות גישות
        self.suspicious_keywords = [
            "login", "log-in", "signin", "sign-in", "signon", "sign-on",
            "logout", "log-out", "signout", "sign-out",
            "auth", "reauth", "authentication", "authenticate",
            "verify", "verification", "validate", "validation",

            # חשבון פרטי
            "account", "myaccount", "account-info", "user", "username",
            "userid",
            "credentials", "password", "passcode", "change-password", "reset-
password",
            "security-question", "security-code", "2fa", "two-factor", "otp",
            "token",

            # אישי מידע
            "profile", "ssn", "social", "personal", "fullname", "dob",
            "birthdate",

            # פיננסי מידע
            "bank", "credit", "card", "debit", "paypal", "transaction",
            "billing",
            "payment", "invoice", "statement", "balance", "iban", "swift",
            "wire",

            # שחזור תהליכי
            "reset", "recovery", "unlock", "restore", "reactivate", "restore-
access",

            # מטעים או דחופים ניסיונות
            "urgent", "update", "suspend", "alert", "important", "warning",
            "confirm", "click-here", "secure", "unusual", "problem", "issue",

            # נפוצות פשינג שיטות
            "secure-login", "secure-verification", "validate-session",
            "session-expired",
            "limited-access", "verify-now", "update-info", "confirm-email",
            "login-attempt"
        ]

    def validate(self, url):
        """
        URL-ה פורמט את בודק.
        """
        regex = re.compile(r'^((https?:/)?)([a-zA-Z0-9-]+\.[a-zA-Z]{2,})$')
        return re.match(regex, url) is not None
```



```

def is_https(self, url):
    """
    https-ב מתחיל URL-ה אם בודק
    """
    return url.startswith("https://")

def is_valid_domain(self, url):
    """
    חוקית בצורה בנוי הדומיין אם בודק
    """
    try:
        parsed_url = urlparse(url)
        domain = parsed_url.netloc
        return re.match(r"^[a-zA-Z0-9.-]+$", domain) is not None
    except Exception:
        return False

def has_ssl(self, url):
    """
    תקינה SSL תעודת לו שיש ולוודא URL-ל בקשה לבצע מנסה
    """
    try:
        requests.get(url, timeout=5, verify=True)
        return True # ש - חריגה בלי הצלחנו אם
    except requests.exceptions.SSLError:
        return False
    except Exception:
        return False

def is_suspicious_url(self, url):
    """
    חשודות מפתח מילות מכיל URL-ה אם בודק
    """
    lower_url = url.lower()
    return any(keyword in lower_url for keyword in
self.suspicious_keywords)

def check_url(self, url):
    reasons = [] # החשודות הסיבות רשימת
    if not self.validate(url):
        reasons.append("The URL is not valid.")
    if not self.is_https(url):
        reasons.append("The link is not secure (does not use HTTPS).")
    if not self.is_valid_domain(url):
        reasons.append("The domain name is invalid or contains suspicious
characters.")
    if not self.has_ssl(url):
        reasons.append("The website does not use SSL (the link is not
encrypted).")
    if self.is_suspicious_url(url):
        reasons.append("The link contains suspicious keywords")

    return reasons

```



RSA.py

```
import random
import math

class RSA:
    def __init__(self, key_size=16):
        # (ביט 16: מחדל ברירת) בביטים המפתח גודל
        self.key_size = key_size
        self.p = None # ראשון ראשוני מספר
        self.q = None # שני ראשוני מספר
        self.n = None #  $p-q$  מכפלת
        self.phi_n = None # פונקציית  $n$  של אוילר
        self.e = None # ציבורי מעריך
        self.d = None # פרטי מעריך

    def is_prime(self, num):
        if num < 2:
            return False
        for i in range(2, int(math.sqrt(num)) + 1):
            if num % i == 0:
                return False
        return True

    def generate_prime(self):
        """
        המפתח לגודל המתאימים בגבולות ראשוני מספר ממש
        """
        lower = 2 ** (self.key_size - 1)
        upper = 2 ** self.key_size - 1
        while True:
            candidate = random.randint(lower, upper)
            if self.is_prime(candidate):
                return candidate

    def gcd(self, a, b):
        """
        מספרים שני בין ביותר הגדול המשותף המחלק את מחשב
        """
        while b != 0:
            a, b = b, a % b
        return a

    def modinv(self, e, phi):
        """
         $\phi(n)$  מודולו  $e$  של ההופכי את מחשב
        """
        g, x, y = self.extended_gcd(e, phi)
        if g != 1:
            return None # No modular inverse exists
        else:
            return x % phi

    def extended_gcd(self, a, b):
```



```

"""
המודולרי ההופכי למציאת אוקלידס של המורחב האלגוריתם.
"""
if a == 0:
    return (b, 0, 1)
else:
    g, y, x = self.extended_gcd(b % a, a)
    return (g, x - (b // a) * y, y)

def generate_keys(self):
    # Generate two distinct primes
    self.p = self.generate_prime()
    self.q = self.generate_prime()
    while self.q == self.p:
        self.q = self.generate_prime()

    # Calculate n and φ(n)
    self.n = self.p * self.q
    self.phi_n = (self.p - 1) * (self.q - 1)

    # Choose e (public exponent)
    self.e = random.randrange(2, self.phi_n)
    while self.gcd(self.e, self.phi_n) != 1:
        self.e = random.randrange(2, self.phi_n)

    # Calculate d (private exponent)
    self.d = self.modinv(self.e, self.phi_n)

def encrypt(self, plaintext, e=None, n=None):
    """Encrypt with specified public key or own public key"""
    e = e or self.e
    n = n or self.n
    if e is None or n is None:
        raise ValueError("Public key not available for encryption")

    encrypted = []
    for char in plaintext:
        m = ord(char)
        c = pow(m, e, n)
        encrypted.append(c)
    return encrypted

def decrypt(self, ciphertext):
    """Decrypt with own private key"""
    if self.d is None or self.n is None:
        raise ValueError("Private key not available for decryption")

    decrypted = []
    for num in ciphertext:
        m = pow(num, self.d, self.n)
        decrypted.append(chr(m))
    return ''.join(decrypted)

def get_public_key(self):
    return (self.e, self.n)

def __str__(self):

```



```
return (f"RSA Key Pair:\n"  
        f"Public Key (e, n): ({self.e}, {self.n})\n"  
        f"Private Key (d, n): ({self.d}, {self.n})\n"  
        f"Primes (p, q): ({self.p}, {self.q})")
```



Rating.py

```
import tkinter as tk
from tkinter import Toplevel, StringVar, IntVar, messagebox

class Rating:
    def __init__(self, parent):
        self.parent = parent

    def setup_rating_button(self):
        """Add the 'Rate Me!' button to the GUI."""
        rate_button = tk.Button(
            self.parent,
            text="Rate Me!",
            command=self.open_rating_form,
            bg="blue",
            fg="white"
        )
        rate_button.pack(anchor="ne", padx=10, pady=10)

    def open_rating_form(self):
        """Opens the rating form popup with star buttons"""
        popup = Toplevel(self.root)
        popup.title("Rate Us")
        popup.geometry("350x300")
        popup.configure(bg="#f0f0f0")

        # Name Entry (optional)
        tk.Label(popup, text="Your Name (optional):",
            bg="#f0f0f0").pack(pady=5)
        name_var = StringVar()
        name_entry = tk.Entry(popup, textvariable=name_var, width=30)
        name_entry.pack(pady=5)

        # Star Rating
        tk.Label(popup, text="Rate Us:", bg="#f0f0f0").pack(pady=5)

        star_frame = tk.Frame(popup, bg="#f0f0f0")
        star_frame.pack()

        self.rating_var = IntVar(value=0)
        self.star_buttons = [] # שפותחים פעם כל לאפס

        for i in range(1, 6):
            star_btn = tk.Button(
                star_frame,
                text="★",
                font=("Arial", 24),
                command=lambda val=i: self.set_rating(val),
                bg="#f0f0f0",
                fg="gold",
                bd=0,
                activebackground="#f0f0f0",
                cursor="hand2"
```



```

    )
    star_btn.pack(side=tk.LEFT, padx=2)
    self.star_buttons.append(star_btn)

    # Feedback
    tk.Label(popup, text="Your Feedback:", bg="#f0f0f0").pack(pady=5)
    feedback_var = StringVar()
    feedback_entry = tk.Entry(popup, textvariable=feedback_var, width=30)
    feedback_entry.pack(pady=5)

    # Submit button
    submit_button = tk.Button(
        popup,
        text="Submit Rating",
        command=lambda: self.submit_rating(
            popup,
            name_var.get().strip() or "Anonymous",
            self.rating_var.get(),
            feedback_var.get()
        ),
        bg="#4CAF50",
        fg="white",
        padx=10,
        pady=5
    )
    submit_button.pack(pady=10)

    def submit_rating(self, popup, name, rating, feedback):
        """Saves the rating to the database."""
        if not name or not feedback:
            messagebox.showerror("Error", "Name and Feedback cannot be
empty!")
            return

        # For now, just print the data as the "database" is None
        print(f"Rating Submitted - Name: {name}, Rating: {rating}, Feedback:
{feedback}")
        popup.destroy()
        messagebox.showinfo("Thank You!", "Your rating has been submitted!")

```



Hash.py

```
import hashlib
import os

class HashManager:
    def __init__(self, env_var="ADMIN_HASH"):
        self.env_var = env_var
        self.stored_hash = os.getenv(env_var)
        if not self.stored_hash:
            raise ValueError(f"הוגדר לא {env_var} הסביבה משתנה")

    def hash_password(self, password):
        hashed = hashlib.sha256(password.encode()).hexdigest()
        return hashed

    def verify_password(self, password):
        result = self.hash_password(password) == self.stored_hash
        return result
```